student@ubuntu-lab:~$ cat README

Linux+ Hands-On Lab Book

Step-by-step labs for every CompTIA Linux+ exam objective, explained in plain English.

EXAM XK0-006 · OBJECTIVES V5.0

ubuntu-lab	**rocky-lab**
Ubuntu Server 24.04 LTS	Rocky Linux 9
main lab system	RHEL-family topics

```
$ ls ~/labs
Part 1  System Management                          23%   7 labs
Part 2  Services and User Management               20%   6 labs
Part 3  Security                                   18%   6 labs
Part 4  Automation, Orchestration, and Scripting   17%   5 labs
Part 5  Troubleshooting                            22%   5 labs
$ ./start --lab 0
```

Unofficial study guide. CompTIA and Linux+ are trademarks of CompTIA, Inc.

Contents

How to Use This Book 3

Lab 0 Build Your Lab 6

Part 1: System Management 18

- Lab 1.1 Explore Basic Linux Concepts 19
- Lab 1.5 Use Common Shell Operations 41

Part 2: Services and User Management 69

- Lab 2.1 Manage Files and Directories 70

Part 5: Troubleshooting 91

- Lab 5.1 Monitoring Concepts and Configurations 92

Appendices 110

- Appendix A: Answer Key 111
- Appendix B: Exam Corner Review Sheet 113
- Appendix C: Exam Objective Coverage Map 116
- Appendix E: Acronym List 118

How to Use This Book

This book teaches the CompTIA Linux+ XK0-006 exam objectives by having you **do** them. Every exam objective has its own lab. In each lab you type real commands on real Linux systems, watch what happens, and read an explanation of why it happened. By the end you will have done, at least once, nearly everything the exam asks about.

Who this book is for

You do not need any Linux experience. You need:

- A computer that can run two virtual machines (VMs) at the same time. In practice that means 8 GB of RAM (16 GB is more comfortable) and about 80 GB of free disk space.
- Virtualization software. Any of these work: VirtualBox (free, all platforms), VMware Workstation Pro or Fusion (free for personal use), Hyper-V (built into Windows Pro), or UTM (macOS on Apple silicon).
- Patience to type commands carefully and read what comes back.

How the book is organized

- **Lab 0** builds your lab: an Ubuntu Server 24.04 LTS VM named `ubuntu-lab` and a Rocky Linux 9 VM named `rocky-lab`. Do this first.
- **Parts 1 to 5** match the five exam domains. Each part has one lab per exam objective, numbered the same way as the objective. Lab 2.4, for example, covers objective 2.4.
- **The appendices** contain the answers to every review question, a coverage map that shows where each exam topic is practiced, and the official acronym list.

You can do the labs in order, which is the easiest path, or jump to the objective you want to study. Every lab cleans up after itself, so labs do not depend on each other.

What is inside every lab

Section	What it gives you
Lab header	The official objective, exam weight, time needed, which VM to use, and whether to take a snapshot first.
What you'll practice	The exam topics this lab covers, in the exam's own words.
How it works	A plain-English explanation of the concepts before you touch the keyboard.
Tasks	Numbered, step-by-step instructions. Each command is followed by sample output and a **What just happened** box.
Checkpoints	Quick checks that prove each task worked before you move on.
Challenge	Optional extra tasks with no step-by-step help, to test yourself.
Clean up	Steps that undo the lab's changes so the next lab starts fresh.
Exam corner	The facts to memorize and review questions. Answers are in Appendix A.

Conventions

Commands you type are shown in code blocks like this. Type them exactly, or copy and paste them:

```
lsblk
```

Commands run on **ubuntu-lab** unless the first line of the block says otherwise:

```
# on rocky-lab
sudo dnf install -y httpd
```

Output that the computer prints back is shown in a shaded block. Your output will differ in small details such as dates, sizes, process IDs, and IP addresses. That is normal.

```
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda      8:0    0   25G  0 disk
```

Inside commands, words in CAPITALS such as `ROCKY_IP` are placeholders. Replace them with your own values.

The boxes in the text mean:

What just happened: explains the command you just ran and how to read its output.

TIP
A shortcut, a good habit, or a difference between Ubuntu and Rocky Linux.

> **CAUTION**
> Something that can break your VM or lose data. Read it before you continue.

> **Checkpoint:** a quick test that proves the task worked.

> **ON THE EXAM**
> A fact that is likely to appear on the exam.

Safety rules

1. **Never do these labs on a computer you care about.** Use only the lab VMs.
2. **Take snapshots.** A snapshot saves the whole state of a VM so you can roll back in seconds. Take one called `clean` at the end of Lab 0, and take another whenever a lab header says "Snapshot first: Yes". If something goes wrong, restore the snapshot and start the lab again.
3. **Keep a second session open** when you change SSH, firewall, sudo, or login settings. If your change locks you out, the open session lets you undo it. The VM console in your virtualization software always works as a last resort.
4. **Read before you press Enter**, especially for commands that start with `sudo`, `rm`, `dd`, `mkfs`, `wipefs`, or `fdisk`.

A note on the exam

Linux+ XK0-006 has up to 90 questions, both multiple-choice and performance-based, and you get 90 minutes. Performance-based questions put you in front of a simulated terminal or configuration screen and ask you to complete a task. The hands-on practice in this book is designed for exactly that kind of question.

This is an unofficial study guide. CompTIA and Linux+ are trademarks of CompTIA, Inc. The objective titles are quoted from the official CompTIA Linux+ XK0-006 Exam Objectives, version 5.0.

Lab 0 Build Your Lab

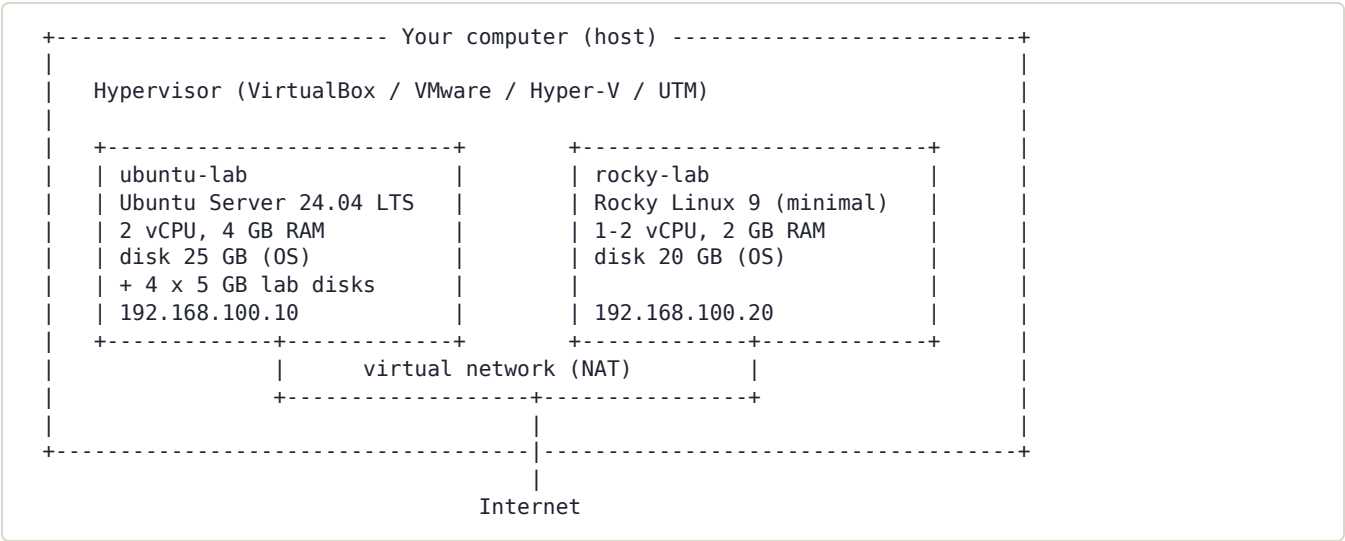
Goal: Build the two virtual machines used in every lab.
Time: about 90 minutes (most of it is waiting for installers)
You will end with: `ubuntu-lab` (Ubuntu Server 24.04 LTS) and `rocky-lab` (Rocky Linux 9) on the same network, each with a snapshot named `clean`

The labs in this book need two Linux servers. Most labs use Ubuntu Server, the most popular Linux server distribution. A few labs need a Red Hat-style system, because the exam covers tools that only exist there: SELinux, firewalld, dnf, rpm, dracut, and Kickstart. For those labs you will use Rocky Linux, a free rebuild of Red Hat Enterprise Linux (RHEL). Anything you learn on Rocky works the same way on RHEL.

How it works

A **virtual machine (VM)** is a computer simulated in software. Your real computer, the **host**, runs a program called a **hypervisor** (VirtualBox, VMware, Hyper-V, or UTM). The hypervisor carves out some of the host's CPU, memory, and disk and presents them to the VM as if they were real hardware. The operating system inside the VM, the **guest**, does not know the difference.

VMs are perfect for practice because you can break them without consequences. A **snapshot** freezes the complete state of a VM's disks (and optionally its memory). If a lab goes wrong, you restore the snapshot and you are back where you started.



Both VMs sit on the same virtual network. They can reach each other and the internet, and you can reach both of them from your host with SSH.

The four extra 5 GB disks on `ubuntu-lab` are for the storage labs (partitions, LVM, RAID, encryption, and disk troubleshooting). They start empty, and every lab that uses them wipes them clean again at the end.

What you need

Item	Where to get it
Hypervisor	VirtualBox (virtualbox.org), VMware Workstation Pro or Fusion (free for personal use), Hyper-V (Windows Pro and up), or UTM (Apple silicon Macs)
Ubuntu Server 24.04 LTS ISO	ubuntu.com/download/server. Choose the latest 24.04.x point release.
Rocky Linux 9 minimal ISO	rockylinux.org/download. Choose the latest 9.x **Minimal** ISO for your CPU type.
Free resources on the host	About 6 GB of RAM and 80 GB of disk for both VMs

TIP

Apple silicon Macs (M1 and later): download the **ARM64** (aarch64) ISOs of both distributions. Everything in the book works the same way. Where a lab downloads an `x86_64` file, such as a cloud image in Lab 1.7, choose the `aarch64` version instead.

Tasks

Task 1: Prepare a virtual network

Both VMs need to be on a network where they can see each other and reach the internet. The simplest choice depends on your hypervisor.

1. Set up the network for your hypervisor:
   - **VirtualBox:** open **File > Tools > Network Manager**, select the **NAT Networks** tab, and click **Create**. Name it `LabNet`, set the IPv4 prefix to `192.168.100.0/24`, and keep **Enable DHCP** checked. You will attach both VMs to this NAT network. (The plain "NAT" mode does **not** let VMs talk to each other.)
   - **VMware Workstation or Fusion:** use the default **NAT** network (VMnet8, or "Share with my Mac"). Its subnet is chosen at random when VMware is installed, for example `192.168.47.0/24`. You will find yours in Task 4.
   - **Hyper-V:** use the **Default Switch**. It provides NAT and DHCP. Its subnet can change when the host reboots, so in Task 4 you will keep DHCP and update `/etc/hosts` if the addresses change.
   - **UTM:** use the default **Shared Network** mode for both VMs.

What just happened: You created (or chose) a private virtual network. **NAT** (Network Address Translation) lets the VMs reach the internet through your host's connection, while hiding them from the outside world. **DHCP** hands each VM an IP address automatically when it boots.

Task 2: Create and install ubuntu-lab

1. Create a new VM with these settings:

Setting	Value
Name	`ubuntu-lab`
Type	Linux, Ubuntu (64-bit), or ARM64 on Apple silicon
CPUs	2
Memory	4096 MB
Disk	25 GB, dynamically allocated (thin provisioned)
Network	The network from Task 1
Firmware	UEFI if offered (BIOS also works)
Install media	The Ubuntu Server 24.04 ISO

> **TIP**
> **VirtualBox:** turn **off** "Unattended Installation" (check "Skip Unattended Installation"), so that you see and learn the installer screens.

Do **not** add the four extra lab disks yet. Adding them after the installation guarantees the installer cannot touch them, and that your system disk is the first disk (`sda` or `vda`).

2. Start the VM and walk through the installer. Use the arrow keys, Tab, Space, and Enter to move around.

Installer screen	What to choose
Language	English
Installer update	“Continue without updating” is fine
Keyboard	Your layout
Type of installation	**Ubuntu Server** (not minimized)
Network	Accept the DHCP address it found. Write it down.
Proxy	Leave empty
Ubuntu archive mirror	Accept the default
Guided storage	**Use an entire disk**, keep **Set up this disk as an LVM group** checked
Storage configuration	See the tip below, then **Done** and **Continue**
Profile	Your name: `Student`, server name: `ubuntu-lab`, username: `student`, and a password you will remember
Ubuntu Pro	Skip for now
SSH configuration	**Install OpenSSH server**: checked. Do not import keys.
Featured server snaps	Select none

> **TIP**
> On the storage summary, the installer usually gives the root logical volume (`ubuntu-lv`) only about half of the disk and leaves the rest free in the volume group. That is fine. You will grow it to full size in Task 3 with one command, which is also a sneak preview of Lab 1.3.

3. Wait for the installation to finish, choose **Reboot Now**, and remove the ISO if your hypervisor asks you to. Log in at the console as `student`.

Checkpoint: you see a prompt like `student@ubuntu-lab:~$`, and `hostnamectl` shows `Static hostname: ubuntu-lab` and `Operating System: Ubuntu 24.04.x LTS`.

Task 3: First steps on ubuntu-lab

1. Update the system. Updating takes a few minutes the first time.

```
sudo apt update
sudo apt full-upgrade -y
```

> **What just happened:** `sudo` runs a command as the administrator (root). It asks for **your** password the first time. `apt update` downloads the latest list of available packages from Ubuntu's servers. `apt full-upgrade` installs newer versions of everything already installed, and `-y` answers "yes" to the confirmation prompt.

2. Grow the root filesystem to use the whole disk:

```
sudo lvextend -r -l +100%FREE /dev/ubuntu-vg/ubuntu-lv
df -h /
```

```
Filesystem                         Size  Used Avail Use% Mounted on
/dev/mapper/ubuntu--vg-ubuntu--lv   23G  5.1G   17G  24% /
```

> **What just happened:** The installer used **LVM** (Logical Volume Manager), which lets you resize storage on the fly. `lvextend` grew the logical volume into all the free space (`-l +100%FREE`), and `-r` also resized the filesystem inside it, so the extra space is usable immediately. `df -h /` shows the size of the root filesystem in human-readable units. You will learn exactly how this works in Lab 1.3.

3. Install a few tools that many labs use:

```
sudo apt install -y vim nano curl wget git tree bash-completion \
  net-tools dnsutils traceroute tcpdump unzip
```

4. Set your time zone so log timestamps match your clock. List zones with `timedatectl list-timezones`, then set yours (this example uses US Central time):

```
sudo timedatectl set-timezone America/Chicago
timedatectl
```

5. Reboot to load the updated kernel:

```
sudo reboot
```

Task 4: Give ubuntu-lab a fixed IP address (recommended)

Servers normally keep the same IP address forever, so other machines can find them. A fixed (static) address also means the `/etc/hosts` entries you create in Task 7 never go stale.

> TIP
> **Hyper-V Default Switch users:** skip this task and keep DHCP. If your VMs' addresses change after a host reboot, update `/etc/hosts` on both VMs (Task 7).

1. Find your current address, network, and gateway:

```
ip -br address
ip route
```

```
lo               UNKNOWN        127.0.0.1/8 ::1/128
enp0s3           UP             192.168.100.4/24 fe80::a00:27ff:fe4e:66a1/64
default via 192.168.100.1 dev enp0s3 proto dhcp src 192.168.100.4 metric 100
192.168.100.0/24 dev enp0s3 proto kernel scope link src 192.168.100.4 metric 100
```

What just happened: `ip -br address` lists your network interfaces briefly (`-br`). Here the interface is named `enp0s3` and has the address `192.168.100.4` on a `/24` network (the first three numbers identify the network). `ip route` shows the routing table. The `default via 192.168.100.1` line is your **default gateway**, the router used to reach everything outside your own network.

Write down three things: your **interface name** (for example `enp0s3` , `ens33` , or `eth0`), your **network** (for example `192.168.100`), and your **gateway** (for example `192.168.100.1` ; on VMware NAT it usually ends in `.2`).

2. Look at the current Netplan configuration. Netplan is Ubuntu's network configuration tool, and it reads YAML files in `/etc/netplan/` :

```
ls /etc/netplan/
sudo cat /etc/netplan/50-cloud-init.yaml
```

The file name may differ slightly. Whatever its name, that file currently tells the interface to use DHCP.

3. Create a new configuration file that sets a static address. Replace `enp0s3` , the address, and the gateway with **your** values. Keep the address in your own network and end it in `.10` :

```
cat <<'EOF' | sudo tee /etc/netplan/60-static.yaml
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: false
      addresses:
        - 192.168.100.10/24
      routes:
        - to: default
          via: 192.168.100.1
      nameservers:
        addresses: [1.1.1.1, 8.8.8.8]
EOF
sudo chmod 600 /etc/netplan/60-static.yaml
```

What just happened: `cat <<'EOF' ... EOF` is a **here document**. Everything between the two `EOF` markers is fed to `sudo tee` , which writes it into the file with root privileges. You will use this trick often, because it avoids typos in an editor. The YAML says: for this interface, turn DHCP off, use this address, send everything else to this gateway, and use these DNS servers. Netplan files are combined in alphabetical order, so `60-static.yaml` overrides the DHCP setting from `50-cloud-init.yaml` . `chmod 600` makes the file readable only by root, because Netplan warns about files that other users can read.

4. Apply the change safely with `netplan try`. It applies the new settings and **automatically rolls back after 120 seconds** unless you press Enter to confirm:

```
sudo netplan try
```

   If the network still works (for example `ping -c 2 8.8.8.8` succeeds in another session or after you confirm), press **Enter** to keep the configuration.

> **CAUTION**
> Do this step from the VM's own console, not over SSH. If you mistype the gateway or interface name, you can lose network access. The console always works.

5. Prevent cloud-init from rewriting the network configuration at a later boot:

```
echo 'network: {config: disabled}' | \
  sudo tee /etc/cloud/cloud.cfg.d/99-disable-network-config.cfg
```

Checkpoint: `ip -br address` shows your `.10` address, and both `ping -c 2 8.8.8.8` and `ping -c 2 ubuntu.com` succeed.

Task 5: Create and install rocky-lab

1. Create a second VM:

Setting	Value
Name	`rocky-lab`
Type	Linux, Red Hat (64-bit), or ARM64 on Apple silicon
CPUs	1 (2 if you can spare it)
Memory	2048 MB
Disk	20 GB, dynamically allocated
Network	The same network as ubuntu-lab
Install media	The Rocky Linux 9 minimal ISO

2. Boot the VM and choose **Install Rocky Linux 9**. The graphical installer, Anaconda, shows an **Installation Summary** hub. Configure each item:

Item	What to choose
Language	English
Installation Destination	Select the 20 GB disk, keep **Automatic** storage, click **Done**
Software Selection	**Minimal Install**
Network & Host Name	Switch the Ethernet adapter **ON**. Set Host Name to `rocky-lab` and click **Apply**.
Root Password	Leave root **locked** ("Lock root account" checked)
User Creation	Full name `Student`, user name `student`, check **Make this user administrator**, set a password
Time & Date	Your time zone

3. Click **Begin Installation**. When it finishes, click **Reboot System** and log in at the console as `student`.

> **What just happened:** "Make this user administrator" adds `student` to the `wheel` group. On Red Hat-style systems, members of `wheel` may use `sudo`. Ubuntu uses a group named `sudo` for the same purpose. Both distributions lock the root account by default, so you always work as a normal user and use `sudo` for admin tasks. That is a security best practice you will study in Lab 3.4.

4. Update the system and install a few tools:

```
# on rocky-lab
sudo dnf -y upgrade
sudo dnf -y install vim nano curl wget git tree bash-completion \
  bind-utils traceroute tcpdump tar unzip policycoreutils-python-utils
sudo reboot
```

> **What just happened:** `dnf` is the package manager on Red Hat-style systems. It does the same job as `apt` on Ubuntu. `upgrade` updates all installed packages, and `-y` answers yes to prompts. `bind-utils` provides `dig` and `nslookup`, and `policycoreutils-python-utils` provides `semanage`, which the SELinux labs use.

Task 6: Give rocky-lab a fixed IP address (recommended)

Rocky Linux manages networking with **NetworkManager**, controlled from the command line with `nmcli`.

1. Find the connection name and current settings:

```
# on rocky-lab
nmcli connection show
ip -br address
ip route
```

```
NAME    UUID                                  TYPE      DEVICE
enp0s3  5fb06bd0-0bb0-7ffb-45f1-d6edd65f3e03  ethernet  enp0s3
lo      2f2b8e4a-5c3a-4b2f-9a1f-0c3b2a1d0e9f  loopback  lo
```

2. Set a static address ending in `.20` in the same network. Replace the connection name, address, and gateway with your values:

```
# on rocky-lab
sudo nmcli connection modify enp0s3 \
  ipv4.method manual \
  ipv4.addresses 192.168.100.20/24 \
  ipv4.gateway 192.168.100.1 \
  ipv4.dns "1.1.1.1 8.8.8.8"
sudo nmcli connection up enp0s3
```

> **What just happened:** `nmcli connection modify` changed the saved connection profile: `ipv4.method manual` turns off DHCP, and the other settings set the address, gateway, and DNS servers. `nmcli connection up` re-activates the profile so the change takes effect. NetworkManager stores the profile in `/etc/NetworkManager/system-connections/`, so it survives reboots.

Checkpoint: `ip -br address` on rocky-lab shows the `.20` address, and `ping -c 2 rockylinux.org` works.

Task 7: Let the VMs find each other by name

DNS turns names into IP addresses, but your lab network has no DNS server for your own VMs. Instead you will use `/etc/hosts`, a simple local lookup file that Linux checks before DNS.

1. On **both** VMs, add these lines to `/etc/hosts`. Use your real addresses:

```
echo "192.168.100.10  ubuntu-lab" | sudo tee -a /etc/hosts
echo "192.168.100.20  rocky-lab"  | sudo tee -a /etc/hosts
```

```
# on rocky-lab
echo "192.168.100.10  ubuntu-lab" | sudo tee -a /etc/hosts
echo "192.168.100.20  rocky-lab"  | sudo tee -a /etc/hosts
```

> **What just happened:** `tee -a` **appends** (`-a`) to the file instead of overwriting it. Running `tee` through `sudo` is the standard way to write to a root-owned file from a pipe. `sudo echo ... >> /etc/hosts` would fail, because the `>>` redirection is done by your own shell, which is not running as root.

2. Test name resolution and connectivity in both directions:

```
ping -c 2 rocky-lab
ssh student@rocky-lab hostname
```

The first time you connect with SSH, you are asked to confirm the server's **host key fingerprint**. Type `yes`, then enter student's password on rocky-lab. The command prints `rocky-lab`.

Checkpoint: `ping rocky-lab` works from ubuntu-lab, and `ping ubuntu-lab` works from rocky-lab.

Task 8: Connect from your host with SSH

Typing in the VM console works, but a terminal on your host is far more comfortable: you get copy and paste, scrollback, and several windows at once.

1. From your host, open a terminal (macOS or Linux Terminal, or Windows Terminal/PowerShell, which include an SSH client) and connect:

```
ssh student@192.168.100.10
```

   - **VirtualBox NAT Network:** the host cannot reach VMs on a NAT Network directly. Add port forwarding: in **Network Manager > NAT Networks > LabNet > Port Forwarding**, add rule `ubuntu-ssh`, host port `2210`, guest IP `192.168.100.10`, guest port `22`, and rule `rocky-ssh`, host port `2220`, guest IP `192.168.100.20`, guest port `22`. Then connect with `ssh -p 2210 student@127.0.0.1` and `ssh -p 2220 student@127.0.0.1`.
   - **VMware, Hyper-V, and UTM:** the host can reach the VM addresses directly.

2. Open a second terminal and connect to rocky-lab the same way. From now on you can work in host terminals.

> **TIP**
> Many labs ask you to keep a second session open while you change login or network settings. Two host terminal windows connected to the same VM give you exactly that.

Task 9: Add the four lab disks to ubuntu-lab

1. Shut down ubuntu-lab:

```
sudo poweroff
```

2. In your hypervisor's settings for ubuntu-lab, add **four new virtual hard disks**, each **5 GB** and dynamically allocated. Put them on the same controller as the system disk (SATA, SCSI, or VirtIO).

3. Start ubuntu-lab, log in, and list the disks:

```
lsblk -d -o NAME,SIZE,TYPE,MODEL
```

```
NAME  SIZE TYPE MODEL
sda    25G disk VBOX HARDDISK
sdb     5G disk VBOX HARDDISK
sdc     5G disk VBOX HARDDISK
sdd     5G disk VBOX HARDDISK
sde     5G disk VBOX HARDDISK
```

What just happened: `lsblk` lists **block devices** (storage devices). `-d` shows only whole disks, not partitions, and `-o` chooses the columns. Linux names SATA and SCSI disks `sda`, `sdb`, `sdc`, and so on, in the order they are detected. VirtIO disks are named `vda`, `vdb`, and so on, and NVMe disks `nvme0n1`, `nvme0n2`, and so on.

CAUTION

The labs call the extra disks `/dev/sdb`, `/dev/sdc`, `/dev/sdd`, and `/dev/sde`. If your names differ, for example `vdb` or `nvme0n2`, **substitute your names every time**. Always run `lsblk` before any command that writes to a disk, and make sure you never touch the 25 GB system disk.

Task 10: Enable nested virtualization (for Lab 1.7)

Lab 1.7 runs a small VM **inside** ubuntu-lab. That is called **nested virtualization**, and it is much faster if the hypervisor passes the CPU's virtualization features through to the VM. Lab 1.7 also works without nesting, just more slowly, so do not worry if your setup does not support it.

1. With ubuntu-lab powered off, enable nesting:
   - **VirtualBox:** Settings > System > Processor > check **Enable Nested VT-x/AMD-V**. If it is greyed out, run this on the host: `VBoxManage modifyvm ubuntu-lab --nested-hw-virt on`.
   - **VMware:** VM Settings > Processors > check **Virtualize Intel VT-x/EPT or AMD-V/RVI**.
   - **Hyper-V:** in an administrator PowerShell on the host: `Set-VMProcessor -VMName ubuntu-lab -ExposeVirtualizationExtensions $true`
   - **UTM on Apple silicon:** not available on most macOS versions. Lab 1.7 uses emulation instead.
2. Start ubuntu-lab and check:

```
grep -cE 'vmx|svm' /proc/cpuinfo
```

What just happened: `vmx` (Intel) and `svm` (AMD) are CPU flags that mean "hardware virtualization is available". `grep -c` counts the matching lines in `/proc/cpuinfo`, one per CPU core. A number greater than 0 means nesting works. `0` means Lab 1.7 will use software emulation.

Task 11: Take the clean snapshots

1. Shut down both VMs (`sudo poweroff`).
2. In your hypervisor, take a snapshot of each VM and name it `clean`.
   - **VirtualBox:** select the VM, open the **Snapshots** view, and click **Take**.
   - **VMware:** VM > Snapshot > Take Snapshot.
   - **Hyper-V:** right-click the VM > Checkpoint.
   - **UTM:** UTM has no snapshot button. Instead, shut down the VM, right-click it, and choose **Clone** to keep a backup copy.
3. Start both VMs again.

Checkpoint: each VM has a snapshot named `clean`, and you can log in to both from your host.

TIP

Restoring a snapshot: shut down the VM (or power it off), select the snapshot, and choose **Restore** (VirtualBox), **Revert to Snapshot** (VMware), or **Apply** (Hyper-V). The VM returns to exactly the state it was in when you took the snapshot.

Your lab at a glance

	ubuntu-lab	rocky-lab
Operating system	Ubuntu Server 24.04 LTS	Rocky Linux 9 (minimal)
Address (example)	192.168.100.10	192.168.100.20
Login	`student` (in the `sudo` group)	`student` (in the `wheel` group)
Package manager	`apt`, `dpkg`	`dnf`, `rpm`
Network configuration	Netplan (`/etc/netplan/`)	NetworkManager (`nmcli`)
Firewall	`ufw` (off by default)	`firewalld` (on by default)
Mandatory access control	AppArmor	SELinux (enforcing)
Extra disks	`sdb`, `sdc`, `sdd`, `sde` (5 GB each)	none

You are ready. Start with Lab 1.1, or go to any objective you want to practice.

PART 1 · DOMAIN 1.0

Part 1: System Management

23% of the exam · 2 labs

1.1	Explain basic Linux concepts.
1.5	Given a scenario, manage a Linux system using common shell operations.

Lab 1.1 Explore Basic Linux Concepts

Objective: 1.1 Explain basic Linux concepts.
Domain: 1.0 System Management (23% of the exam)
Time: about 75 minutes
VM: ubuntu-lab and rocky-lab
Snapshot first: Yes. This lab edits the bootloader configuration and reboots ubuntu-lab.

In this lab you follow a Linux system from power-on to login prompt, change a bootloader setting and a kernel parameter and watch the change take effect, and then tour the standard directory layout. You also find out which CPU architecture, distribution family, desktop components, and software licenses your two VMs use. These are the "vocabulary" topics of Linux+: every later lab assumes you know them, and the exam asks about them directly.

What you'll practice

- Basic boot process: bootloader (configuration files), kernel (parameters), initrd, PXE
- Filesystem Hierarchy Standard (FHS): `/`, `/bin`, `/boot`, `/dev`, `/etc`, `/home`, `/lib`, `/proc`, `/sbin`, `/tmp`, `/usr`, `/var`
- Server architectures: AArch64, RISC-V, x86, x86_64/AMD64
- Distributions: RPM-based, dpkg-based
- Graphical user interface (GUI): display managers, window managers, X Server, Wayland
- Software licensing: open source, free software, proprietary, copyleft

How it works

The boot process. Starting a Linux computer is a relay race. Each runner does one job and hands the baton to the next:

```
Power on
   |
Firmware (BIOS or UEFI)      tests hardware, finds a boot device
   |                         (local disk, USB, or the network = PXE)
Bootloader (GRUB 2)          shows a menu, reads its configuration file,
   |                         loads the kernel + initrd into memory
Kernel (vmlinuz)             starts with its parameters (the "command line"),
   |                         detects hardware, mounts the initrd
initrd / initramfs           tiny temporary root filesystem with the drivers
   |                         needed to find and mount the real root disk
Real root filesystem  /
   |
systemd (PID 1)              starts services, reaches a target
   |
Login prompt (text or graphical display manager)
```

1. **Firmware.** The BIOS (older) or UEFI (modern) firmware is built into the motherboard or the VM. It checks the hardware and picks a boot device.
2. **Bootloader.** A bootloader is a small program whose only job is to load the operating system. Linux almost always uses **GRUB 2** (GRand Unified Bootloader). GRUB reads a generated **configuration file**, `/boot/grub/grub.cfg` on Ubuntu or `/boot/grub2/grub.cfg` on Rocky. You never edit that file by hand. Instead you edit `/etc/default/grub` (the settings you choose) and run a tool that rebuilds `grub.cfg`: `update-grub` on Ubuntu, `grub2-mkconfig` on Rocky.
3. **Kernel.** The **kernel** is the core of the operating system. It talks to the hardware and shares the CPU and memory between programs. Its file is `/boot/vmlinuz-<version>`. GRUB passes it **kernel parameters**, such as `root=` (where the real root filesystem is), `ro` (mount it read-only at first), or `quiet` (show fewer messages). You can read the parameters of the running kernel in `/proc/cmdline`.
4. **initrd.** The kernel needs a driver to read the root disk, but the drivers live on that disk. The **initrd** (initial RAM disk), today called an **initramfs**, solves this chicken-and-egg problem. It is a compressed archive that GRUB loads into memory next to the kernel. It holds just enough drivers and scripts to find and mount the real root filesystem, and then it hands over control. On Ubuntu it is `/boot/initrd.img-<version>` (built by `update-initramfs`); on Rocky it is `/boot/initramfs-<version>.img` (built by `dracut`).
5. **systemd.** The first real program, process ID 1, is `systemd`. It starts all services and brings the system to a **target** (a group of services), such as `multi-user.target` (text login) or `graphical.target` (desktop login).

PXE (Preboot eXecution Environment, said "pixie") is booting from the network instead of a disk. The network card's firmware asks a **DHCP** server for an IP address and is told "your boot file is on this **TFTP** server." It downloads a network bootloader (for example `pxelinux.0` or `grubnetx64.efi`), which downloads its configuration, a kernel, and an initrd, and boots them. Data centers use PXE to install hundreds of servers without touching a USB stick.

The Filesystem Hierarchy Standard (FHS) is the agreed map of where things live on a Linux system. Think of it as the floor plan every distribution follows, so an administrator can find the logs or the configuration on any system:

Directory	What lives there
`/`	The root of the single directory tree. Everything hangs below it.
`/bin`	Essential user commands (`ls`, `cp`). On modern systems a link to `/usr/bin`.
`/sbin`	Essential system administration commands (`fdisk`, `ip`). A link to `/usr/sbin`.
`/boot`	The kernel, the initrd, and GRUB files.
`/dev`	Device files, such as `/dev/sda` (a disk) or `/dev/null`.
`/etc`	System-wide configuration files (text).
`/home`	Personal directories for users, such as `/home/student`.
`/lib`	Shared libraries and kernel modules. A link to `/usr/lib`.
`/proc`	A virtual filesystem: live information from the kernel, not stored on disk.
`/tmp`	Temporary files that anyone can write. Often cleared at boot.
`/usr`	Installed programs, libraries, and documentation (read-mostly).
`/var`	Variable data that grows: logs, caches, mail, databases, print queues.

Architectures. A CPU architecture is the instruction language a processor understands. A program compiled for one architecture does not run on another. **x86** is the original 32-bit Intel family. **x86_64**, also called **AMD64** (AMD invented the 64-bit extension) or `amd64` in Debian/Ubuntu package names, is the 64-bit version used by most PCs and servers. **AArch64** (also called ARM64, `arm64` in Ubuntu) is 64-bit ARM, used in phones, Apple silicon Macs, Raspberry Pi, and cloud servers such as AWS Graviton. **RISC-V** is a newer, open (royalty-free) instruction set used in embedded boards and growing in servers.

Distribution families. A distribution is the Linux kernel plus tools, a package format, and an installer. Two big families matter for the exam. **dpkg-based** distributions (Debian, Ubuntu, Mint) use `.deb` packages, the low-level tool `dpkg`, and the high-level tool `apt`. **RPM-based** distributions (Red Hat Enterprise Linux, Rocky, AlmaLinux, Fedora, openSUSE) use `.rpm` packages, the low-level tool `rpm`, and high-level tools such as `dnf` or `zypper`.

The GUI stack. A Linux desktop is built from separate layers. The **display server** draws pixels and handles the keyboard and mouse. The classic one is the **X Server** (X11, Xorg); the modern replacement is **Wayland**, a protocol where the compositor itself is the display server. A **window manager** decides where windows go and draws their borders and title bars (examples: Mutter in GNOME, KWin in KDE, Openbox, i3). A **display manager** is the graphical login screen that starts your session (GDM, SDDM, LightDM). A desktop environment (GNOME, KDE Plasma, Xfce) bundles all of this with apps. Servers usually have no GUI at all.

Software licensing. Open source means the source code is available and you may study, change, and share it under the license. **Free software** (as defined by the Free Software Foundation) means "free as in freedom": the four freedoms to run, study, change, and redistribute. In practice most open source software is also free software. **Copyleft**

licenses such as the GNU GPL say "if you distribute a changed version, you must share your source under the same license." Permissive licenses (MIT, BSD, Apache) let others make closed versions. **Proprietary** software is owned and controlled by a company; you get a license to use it, usually without source code, and you may not redistribute it (examples: some NVIDIA drivers, many commercial databases).

Tasks

Task 1: Look at the boot files on ubuntu-lab

1. Log in to ubuntu-lab as `student` and create the lab working directory. `mkdir -p` creates the folder (and any missing parent folders) without complaining if it already exists, and `cd` moves you into it.

```
mkdir -p ~/labs/1.1 && cd ~/labs/1.1
```

2. Find out whether the VM booted with UEFI or legacy BIOS firmware. UEFI systems expose the folder `/sys/firmware/efi`; BIOS systems do not.

```
[ -d /sys/firmware/efi ] && echo "UEFI boot" || echo "Legacy BIOS boot"
```

```
UEFI boot
```

> **What just happened:** `[ -d PATH ]` is a test that is true if `PATH` exists and is a directory. `&&` runs the next command only if the test succeeded, and `||` runs its command only if the test failed. Either answer is fine for this lab. On UEFI systems, the firmware loads GRUB from a small FAT partition mounted at `/boot/efi`.

3. List the contents of `/boot`. This is where GRUB looks for the kernel and the initrd.

```
ls -lh /boot
```

```
-rw-r--r-- 1 root root 287K Aug  9 config-6.8.0-45-generic
drwx------ 3 root root 4.0K Jan  1  1970 efi
drwxr-xr-x 5 root root 4.0K Oct 10 09:12 grub
lrwxrwxrwx 1 root root   28 Oct 10 09:10 initrd.img -> initrd.img-6.8.0-45-generic
-rw-r--r-- 1 root root  67M Oct 10 09:11 initrd.img-6.8.0-45-generic
-rw------- 1 root root 8.9M Aug  9 System.map-6.8.0-45-generic
lrwxrwxrwx 1 root root   25 Oct 10 09:10 vmlinuz -> vmlinuz-6.8.0-45-generic
-rw------- 1 root root  15M Aug  9 vmlinuz-6.8.0-45-generic
```

What just happened: `ls -lh` shows a long listing (`-l`) with human-readable sizes (`-h`). Read it like this:

- `vmlinuz-6.8.0-45-generic` is the compressed **kernel**. The "z" means compressed.
- `initrd.img-6.8.0-45-generic` is the **initrd** (initramfs) for that exact kernel version. It is much larger than the kernel because it contains drivers and tools.
- `config-...` is the list of options the kernel was compiled with, and `System.map-...` is a table of kernel symbol addresses used for debugging.
- `vmlinuz` and `initrd.img` without a version are **symbolic links** (shortcuts, shown by `->`) that always point at the newest kernel.
- `grub/` holds GRUB's modules and its generated configuration file. `efi/` is the EFI system partition (only on UEFI systems).

Your version numbers will differ. You may see two kernels if updates were installed; Ubuntu keeps the previous one as a fallback.

4. Compare the file names with the kernel that is running now. `uname -r` prints the running kernel release.

```
uname -r
```

```
6.8.0-45-generic
```

Checkpoint: You can name the kernel file and the initrd file in `/boot` that match the output of `uname -r` . Run `ls /boot/*$(uname -r)*` and you should see the `config` , `initrd.img` , `System.map` , and `vmlinuz` files for your running kernel.

Task 2: Read the bootloader configuration files and the kernel command line

1. Display GRUB's settings file. This is the file **you** edit.

```
cat /etc/default/grub
```

```
GRUB_DEFAULT=0
GRUB_TIMEOUT_STYLE=hidden
GRUB_TIMEOUT=0
GRUB_DISTRIBUTOR=`( . /etc/os-release; echo ${NAME:-Ubuntu} ) 2>/dev/null || echo Ubuntu`
GRUB_CMDLINE_LINUX_DEFAULT=""
GRUB_CMDLINE_LINUX=""
```

What just happened: `cat` prints a file. The comment lines (starting with `#`) were left out of the sample. The important settings are:

- `GRUB_DEFAULT=0` boots the first menu entry by default.
- `GRUB_TIMEOUT_STYLE=hidden` and `GRUB_TIMEOUT=0` hide the menu and boot immediately. (Holding **Shift** on BIOS, or pressing **Esc** on UEFI, during boot still shows the menu.)
- `GRUB_CMDLINE_LINUX_DEFAULT` holds kernel parameters for normal boots only. Ubuntu Desktop puts `quiet splash` here.
- `GRUB_CMDLINE_LINUX` holds kernel parameters added to **every** boot entry, including recovery mode.

TIP

Ubuntu also reads files in `/etc/default/grub.d/` after `/etc/default/grub`, and they can override it. Cloud images often have one there. Check with `ls /etc/default/grub.d/`. If a file there sets `GRUB_TIMEOUT`, that value wins.

2. Look at the generated configuration file that GRUB actually reads at boot. It is long, so search it for the lines that load the kernel and initrd. `grep -E` searches using an extended regular expression, and `-m 4` stops after four matches.

```
sudo grep -E -m 4 '^\s*(menuentry|linux|initrd)' /boot/grub/grub.cfg
```

```
menuentry 'Ubuntu' --class ubuntu --class gnu-linux --class gnu --class os ... {
        linux   /vmlinuz-6.8.0-45-generic root=/dev/mapper/ubuntu--vg-ubuntu--lv ro
        initrd  /initrd.img-6.8.0-45-generic
menuentry 'Ubuntu, with Linux 6.8.0-45-generic' --class ubuntu ...
```

What just happened: Each `menuentry` is one choice in the GRUB menu. Inside it, the `linux` line names the kernel file and its **parameters** (`root=` and `ro` here), and the `initrd` line names the initrd file. Notice the top of the real file says `DO NOT EDIT THIS FILE`: it is rebuilt from `/etc/default/grub` and the scripts in `/etc/grub.d/` every time `update-grub` runs, so manual edits would be lost.

3. Read the parameters the running kernel actually received.

```
cat /proc/cmdline
```

```
BOOT_IMAGE=/vmlinuz-6.8.0-45-generic root=/dev/mapper/ubuntu--vg-ubuntu--lv ro
```

What just happened: `/proc/cmdline` is not a real file on disk. `/proc` is a virtual filesystem, and reading this "file" asks the kernel to print the command line it was started with. `BOOT_IMAGE=` is added by GRUB to say which kernel file it loaded. This matches the `linux` line in `grub.cfg`, which proves which configuration was used.

Checkpoint: You can explain the difference between `/etc/default/grub` (you edit it), `/boot/grub/grub.cfg` (generated, GRUB reads it), and `/proc/cmdline` (what the kernel received). `cat /proc/cmdline` should show a `root=` parameter.

Task 3: Read the story of the last boot

1. Ask systemd how long the last boot took.

```
systemd-analyze
```

```
Startup finished in 3.512s (firmware) + 2.104s (loader) + 2.311s (kernel)
+ 9.874s (userspace) = 17.802s
graphical.target reached after 9.801s in userspace.
```

What just happened: `systemd-analyze` splits the boot into the stages from the diagram: **firmware** (UEFI only), **loader** (GRUB), **kernel** (kernel plus initrd), and **userspace** (systemd starting services). On a BIOS VM the firmware and loader times are not shown. Do not worry about "graphical.target" on a server; on Ubuntu Server it is an alias that adds nothing visible.

2. See which services took the longest, and the chain of units that the boot waited for. Press `q` to leave the pager if one opens.

```
systemd-analyze blame | head -5
systemd-analyze critical-chain
```

What just happened: `blame` lists units sorted by how long each took to start; `head -5` keeps the first five lines. `critical-chain` shows the path of units that the final target had to wait for, with times after `@` (when it started) and `+` (how long it took). Slow boots are usually explained by the top entries.

3. Read the kernel's own messages from the current boot. `journalctl` reads the systemd journal; `-b` limits it to the current boot and `-k` to kernel messages. `--no-pager` prints straight to the screen.

```
journalctl -b -k --no-pager | head -8
```

```
Oct 10 09:20:01 ubuntu-lab kernel: Linux version 6.8.0-45-generic ...
Oct 10 09:20:01 ubuntu-lab kernel: Command line: BOOT_IMAGE=/vmlinuz-6.8.0-45-generic
root=/dev/mapper/ubuntu--vg-ubuntu--lv ro
Oct 10 09:20:01 ubuntu-lab kernel: KERNEL supported cpus:
Oct 10 09:20:01 ubuntu-lab kernel:   Intel GenuineIntel
Oct 10 09:20:01 ubuntu-lab kernel:   AMD AuthenticAMD
...
```

What just happened: The very first kernel messages show the kernel version and, again, the command line. If you run `journalctl --list-boots`, you see a numbered list of earlier boots; `journalctl -b -1` shows the previous boot. Ubuntu keeps the journal on disk in `/var/log/journal`, so these survive reboots.

4. Find the moment the initrd handed over to the real root filesystem.

```
journalctl -b --no-pager | grep -i -m 3 -E 'initrd|switch root'
```

What just happened: You should see lines such as `Reached target initrd-switch-root.target` or `Switching root.`. That is systemd inside the initramfs finishing its job and switching to your real `/`. On some Ubuntu builds the initramfs uses shell scripts instead of systemd and the wording differs; any line mentioning initrd or switching root is fine.

Checkpoint: You can name the four stages `systemd-analyze` reports and find the kernel command line in `journalctl -b -k`.

Task 4: Change GRUB and add a kernel parameter (then revert)

CAUTION

Take a snapshot of ubuntu-lab now if you have not already. A typo in `/etc/default/grub` can stop the VM from booting. You will make a backup copy first, and you should work at the VM console (not only over SSH) so you can see the GRUB menu.

1. Make a backup copy of the settings file. `cp` copies a file; `sudo` is needed because only root can write in `/etc`.

```
sudo cp /etc/default/grub /etc/default/grub.lab-backup
```

2. Show the GRUB menu for 10 seconds and add a harmless made-up kernel parameter, `labtest=1`. You can edit the file with `sudo nano /etc/default/grub`, or use the three `sed` commands below, which do the same edits without typos.

```
sudo sed -i 's/^GRUB_TIMEOUT_STYLE=.*/GRUB_TIMEOUT_STYLE=menu/' /etc/default/grub
sudo sed -i 's/^GRUB_TIMEOUT=.*/GRUB_TIMEOUT=10/' /etc/default/grub
sudo sed -i 's/^GRUB_CMDLINE_LINUX_DEFAULT="\(.*\)"/GRUB_CMDLINE_LINUX_DEFAULT="\1
labtest=1"/' \
  /etc/default/grub
```

What just happened: `sed` is a stream editor. `-i` edits the file in place. The expression `s/OLD/NEW/` substitutes text. `^` means "start of the line" and `.*` means "anything", so `s/^GRUB_TIMEOUT=.*/GRUB_TIMEOUT=10/` replaces the whole `GRUB_TIMEOUT` line. In the third command, `\(.*\)` captures whatever was already between the quotes, and `\1` puts it back, followed by `labtest=1`. That way any existing parameters (such as `quiet splash`) are kept.

3. Check your edits before going further.

```
grep -E '^GRUB_(TIMEOUT|CMDLINE)' /etc/default/grub
```

```
GRUB_TIMEOUT_STYLE=menu
GRUB_TIMEOUT=10
GRUB_CMDLINE_LINUX_DEFAULT=" labtest=1"
GRUB_CMDLINE_LINUX=""
```

What just happened: The leading space inside the quotes is harmless; the kernel ignores extra spaces. If any line looks wrong, fix it with `sudo nano /etc/default/grub` before continuing.

4. Rebuild `grub.cfg` from your settings.

```
sudo update-grub
```

```
Sourcing file `/etc/default/grub'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-6.8.0-45-generic
Found initrd image: /boot/initrd.img-6.8.0-45-generic
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
Adding boot menu entry for UEFI Firmware Settings ...
done
```

What just happened: `update-grub` is an Ubuntu/Debian shortcut for `grub-mkconfig -o /boot/grub/grub.cfg`. It reads `/etc/default/grub`, runs each script in `/etc/grub.d/` (for example `10_linux` finds kernels in `/boot`), and writes a fresh `grub.cfg`. The os-prober warning only means it will not look for other operating systems such as Windows. That is expected.

5. Confirm the new parameter is now in the generated file, then reboot.

```
sudo grep -m 1 'labtest' /boot/grub/grub.cfg
sudo reboot
```

6. Watch the VM console. You now see the GRUB menu with a 10-second countdown. Use the arrow keys to look at the entries (including **Advanced options for Ubuntu**), but do not press `e` yet. Let it boot the default entry.

TIP

At the GRUB menu, pressing `e` opens the selected entry for a **one-time** edit. You can add or remove kernel parameters on the `linux` line and press **Ctrl+X** to boot. Nothing is saved. Administrators use this to rescue a system, for example by adding `systemd.unit=rescue.target` to boot into single-user rescue mode.

7. Log back in and check what the kernel received.

```
cat /proc/cmdline
sudo dmesg | grep -i labtest
```

```
BOOT_IMAGE=/vmlinuz-6.8.0-45-generic root=/dev/mapper/ubuntu--vg-ubuntu--lv ro labtest=1
[    0.071234] Unknown kernel command line parameters "BOOT_IMAGE=/vmlinuz-6.8.0-45-generic
labtest=1", will be passed to user space.
```

What just happened: The parameter traveled the whole chain: `/etc/default/grub` to `grub.cfg` (via `update-grub`) to GRUB to the kernel. The kernel did not recognize `labtest=1`, so it printed a notice and passed it on to user space (programs can still read it in `/proc/cmdline`). That is why it is harmless. Real parameters work the same way, for example `nomodeset` (do not load graphics drivers), `systemd.unit=multi-user.target`, or `mitigations=off`. `dmesg` prints the kernel's message buffer; `sudo` is needed because Ubuntu restricts it to root.

8. Revert to the original settings and rebuild `grub.cfg` again.

```
sudo cp /etc/default/grub.lab-backup /etc/default/grub
sudo update-grub
grep -c labtest /boot/grub/grub.cfg
```

```
0
```

What just happened: You restored the backup and regenerated the file. `grep -c` counts matching lines; `0` proves the parameter is gone from the configuration. It disappears from `/proc/cmdline` at the next reboot. Leave the backup file for now; you delete it in Clean up.

Checkpoint: Before the revert, `cat /proc/cmdline` showed `labtest=1`. After the revert, `grep -c labtest /boot/grub/grub.cfg` prints `0` and `grep ^GRUB_TIMEOUT= /etc/default/grub` shows the original value.

Task 5: Compare GRUB tools and look inside the initrd on both systems

1. On ubuntu-lab, list what is inside the initrd. `lsinitramfs` (from the `initramfs-tools-core` package, installed by default) lists the files in an initramfs image.

```
sudo lsinitramfs /boot/initrd.img-$(uname -r) | wc -l
sudo lsinitramfs /boot/initrd.img-$(uname -r) | grep -E 'bin/(busybox|lvm|udevadm)$'
```

```
3417
usr/bin/busybox
usr/bin/udevadm
usr/sbin/lvm
```

What just happened: `$(uname -r)` is **command substitution**: the shell runs `uname -r` and puts its output into the file name, so the command always targets the running kernel. `wc -l` counts lines, so you see the initrd holds thousands of files. The `grep` finds three key tools: `busybox` (a small all-in-one shell and command set), `udevadm` (detects devices), and `lvm` (needed here because Ubuntu's default install puts `/` on an LVM volume). Without `lvm` in the initrd, the kernel could not find `root=/dev/mapper/ubuntu--vg-ubuntu--lv`.

2. Switch to rocky-lab (log in at its console or run `ssh student@rocky-lab`). Rocky uses the same GRUB 2, but the commands have a `2` in their names and the files are in different places.

```
# on rocky-lab
cat /etc/default/grub
ls /boot
ls /boot/loader/entries/
```

```
GRUB_TIMEOUT=5
GRUB_DISTRIBUTOR="$(sed 's, release .*$,,g' /etc/system-release)"
GRUB_DEFAULT=saved
GRUB_DISABLE_SUBMENU=true
GRUB_TERMINAL_OUTPUT="console"
GRUB_CMDLINE_LINUX="crashkernel=1G-4G:192M,4G-64G:256M,64G-:512M resume=/dev/mapper/rl-swap
rd.lvm.lv=rl/root rd.lvm.lv=rl/swap"
GRUB_DISABLE_RECOVERY="true"
GRUB_ENABLE_BLSCFG=true
...
config-5.14.0-427.13.1.el9_4.x86_64
efi
grub2
initramfs-5.14.0-427.13.1.el9_4.x86_64.img
loader
vmlinuz-5.14.0-427.13.1.el9_4.x86_64
...
a1b2c3d4e5f6-5.14.0-427.13.1.el9_4.x86_64.conf
```

What just happened: Rocky's initrd is named `initramfs-<version>.img`. `GRUB_ENABLE_BLSCFG=true` means Rocky uses the **Boot Loader Specification** (BLS): each kernel gets its own small entry file in `/boot/loader/entries/` instead of a long `menuentry` block in `grub.cfg`. The `rd.lvm.lv=` parameters tell the initramfs (`rd` means "RAM disk") which LVM volumes to activate.

3. On Rocky, the recommended tool for changing kernel parameters is `grubby`, which edits the BLS entries directly. Look at the default entry, add the test parameter, check it, and remove it again.

```
# on rocky-lab
sudo grubby --default-kernel
sudo grubby --update-kernel=ALL --args="labtest=1"
sudo grubby --info=DEFAULT | grep ^args
sudo grubby --update-kernel=ALL --remove-args="labtest=1"
sudo grubby --info=DEFAULT | grep ^args
```

```
/boot/vmlinuz-5.14.0-427.13.1.el9_4.x86_64
args="ro crashkernel=1G-4G:192M,4G-64G:256M,64G-:512M resume=/dev/mapper/rl-swap
rd.lvm.lv=rl/root rd.lvm.lv=rl/swap labtest=1"
args="ro crashkernel=1G-4G:192M,4G-64G:256M,64G-:512M resume=/dev/mapper/rl-swap
rd.lvm.lv=rl/root rd.lvm.lv=rl/swap"
```

What just happened: `--default-kernel` shows the kernel GRUB boots by default. `--update-kernel=ALL` applies a change to every installed kernel, and `--args` / `--remove-args` add or remove parameters. `--info=DEFAULT` prints the default entry, and its `args=` line is what goes into `/proc/cmdline` at the next boot. You did not reboot, so nothing changed in the running system.

TIP

If you change `/etc/default/grub` on Rocky (for example `GRUB_TIMEOUT`), rebuild with `sudo grub2-mkconfig -o /boot/grub2/grub.cfg`. That path is correct on both BIOS and UEFI systems in Rocky 9. On Rocky 9.3 and later, `grub2-mkconfig` only copies `GRUB_CMDLINE_LINUX` into existing BLS entries if you add `--update-bls-cmdline`, which is one more reason to prefer `grubby` for kernel parameters.

4. Look inside Rocky's initrd. `lsinitrd` comes with `dracut`; with no file name it inspects the initramfs of the running kernel.

```
# on rocky-lab
sudo lsinitrd | head -15
sudo lsinitrd -m
```

What just happened: The first lines of `lsinitrd` show the image size and the dracut version, followed by the **dracut modules** included (such as `lvm`, `systemd`, `kernel-modules`) and then the file list. `-m` lists only the dracut modules. Lab 1.2 shows how to rebuild this image with `dracut -f`.

Checkpoint: You can name the bootloader configuration tool on each system (`update-grub` on Ubuntu, `grub2-mkconfig` or `grubby` on Rocky), the initrd file name on each, and the command that lists its contents (`lsinitramfs` vs `lsinitrd`). On rocky-lab, `sudo grubby --info=DEFAULT | grep -c labtest` prints `0`.

Task 6: Explore PXE network booting

TIP

Explore/simulate: A real PXE boot needs a DHCP server, a TFTP server, and a client VM with network boot enabled, and a second DHCP server on your lab network could break your other VMs. Instead, you will write the configuration files a PXE server would use, so you can recognize each piece on the exam. Nothing is installed or started.

1. Here is the conversation a PXE client has when it powers on:

```
PXE client (no OS)                  Servers on the network
------------------                  ----------------------
DHCPDISCOVER  "I need an IP + boot info"  ---> DHCP server
              <--- DHCPOFFER  IP 192.168.100.50
                   next-server 192.168.100.10  (TFTP server)
                   filename "grubnetx64.efi"   (or pxelinux.0 for BIOS)
TFTP GET grubnetx64.efi                 ---> TFTP server
TFTP GET grub/grub.cfg                  ---> TFTP server
TFTP/HTTP GET vmlinuz + initrd          ---> TFTP or HTTP server
boot kernel with parameters from the config
```

What just happened: DHCP (Dynamic Host Configuration Protocol) gives out addresses and can also say where the boot file is. TFTP (Trivial File Transfer Protocol, UDP port 69) is a tiny file transfer protocol simple enough to fit in network card firmware. After the bootloader is running, larger files are often fetched over HTTP, which is faster.

2. Write an example `dnsmasq` configuration. `dnsmasq` is a small program that can be both the DHCP and TFTP server. This lab uses a **heredoc**: everything between `<<'EOF'` and the line `EOF` is fed to `cat` as input, and `cat` writes it into the file named after `>`. The quotes around `'EOF'` stop the shell from changing anything inside.

```
mkdir -p ~/labs/1.1/pxe/pxelinux.cfg
cat <<'EOF' > ~/labs/1.1/pxe/dnsmasq-pxe.conf
# Example only: DHCP + TFTP for PXE clients on the lab network
interface=enp0s3
dhcp-range=192.168.100.50,192.168.100.99,12h
enable-tftp
tftp-root=/srv/tftp
# BIOS clients get pxelinux.0, UEFI x86_64 clients get GRUB
dhcp-match=set:efi64,option:client-arch,7
dhcp-boot=tag:efi64,grubnetx64.efi
dhcp-boot=tag:!efi64,pxelinux.0
EOF
```

3. Write the menu file a BIOS client would download after `pxelinux.0`. PXELINUX looks for `pxelinux.cfg/default` when no host-specific file exists.

```
cat <<'EOF' > ~/labs/1.1/pxe/pxelinux.cfg/default
DEFAULT install
PROMPT 0
TIMEOUT 50
LABEL install
  MENU LABEL Install Ubuntu Server 24.04 over the network
  KERNEL ubuntu/vmlinuz
  APPEND initrd=ubuntu/initrd ip=dhcp url=http://192.168.100.10/ubuntu-24.04.iso
EOF
cat ~/labs/1.1/pxe/pxelinux.cfg/default
```

What just happened: This file has the same parts as a local GRUB entry: a **kernel** (`KERNEL`), an **initrd** (`initrd=`), and **kernel parameters** (`ip=dhcp` tells the initrd to get an address, `url=` tells the Ubuntu installer where to download the ISO). The only difference from a disk boot is that every file arrives over the network. A UEFI client would read a `grub.cfg` with `linux` and `initrd` lines instead.

Checkpoint: `ls -R ~/labs/1.1/pxe` shows `dnsmasq-pxe.conf` and `pxelinux.cfg/default`, and you can explain the order: DHCP, then TFTP download of the bootloader, then its configuration, then kernel and initrd.

Task 7: FHS scavenger hunt

Answer each question by running the command. Try to guess the answer before you run it.

Start with a look at the top of the tree. `ls /` lists the directories directly under the root directory `/`.

```
ls /
```

```
bin                boot  etc   lib    lib64       media  opt   root  sbin  srv  tmp  var
bin.usr-is-merged  dev   home  lib.usr-is-merged  lost+found  mnt   proc  run   sbin.usr-is-
merged
snap  sys  usr
```

What just happened: Every name the FHS table lists is here, plus a few more: `/opt` (add-on software from vendors), `/srv` (data served by this system, such as websites), `/mnt` and `/media` (mount points for temporary and removable storage), `/run` (runtime data such as process ID files, kept in RAM), `/root` (the root user's home directory, not to be confused with `/`), and `/sys` (another virtual filesystem, for devices and drivers). The `*.usr-is-merged` entries are markers Ubuntu leaves to record that the merged `/usr` layout is in place. Now answer the hunt questions.

1. Which top-level directories are really links? Ubuntu 24.04 and Rocky 9 both use the "merged /usr" layout.

```
ls -l / | grep -- '->'
```

```
lrwxrwxrwx   1 root root     7 Apr 22 13:08 bin -> usr/bin
lrwxrwxrwx   1 root root     7 Apr 22 13:08 lib -> usr/lib
lrwxrwxrwx   1 root root     9 Apr 22 13:08 lib64 -> usr/lib64
lrwxrwxrwx   1 root root     8 Apr 22 13:08 sbin -> usr/sbin
```

What just happened: `/bin`, `/sbin`, and `/lib` still exist for compatibility, but they point into `/usr`. The `--` tells `grep` that `->` is a pattern, not an option. On the exam, still remember the classic meaning: `/bin` holds essential user commands, `/sbin` essential system administration commands, and `/lib` essential libraries.

2. Where do kernel modules (drivers) live, and where does the `ip` command live?

```
ls /lib/modules/
command -v ip ls fdisk
```

What just happened: `/lib/modules/` has one folder per installed kernel version. `command -v` prints the full path of each command. You will see `/usr/bin/ls` and `/usr/sbin/ip` and `/usr/sbin/fdisk`, which shows the `/sbin` = system administration convention.

3. What is in `/dev`? Find your disks and the "black hole" device.

```
ls -l /dev/sd* /dev/null
echo "this disappears" > /dev/null
```

```
brw-rw---- 1 root disk 8,  0 Oct 10 09:20 /dev/sda
brw-rw---- 1 root disk 8, 16 Oct 10 09:20 /dev/sdb
crw-rw-rw- 1 root root 1,  3 Oct 10 09:20 /dev/null
```

What just happened: The first letter of the permissions shows the device type: `b` is a **block** device (stores data in blocks, like a disk) and `c` is a **character** device (a stream of bytes, like a terminal or `/dev/null`). Anything written to `/dev/null` is thrown away. `/dev` is created in memory by the kernel and `udev` at every boot. If your disks are named `vda` or `nvme0n1`, use `/dev/vd*` or `/dev/nvme*` instead.

4. What does `/proc` know about your CPU and memory, and which process is number 1?

```
grep -m 1 'model name' /proc/cpuinfo
grep MemTotal /proc/meminfo
cat /proc/1/comm
```

What just happened: `/proc` files are generated by the kernel on demand. `cpuinfo` and `meminfo` describe hardware; each numbered folder is a running process, and `/proc/1/comm` prints the name of process 1, which is `systemd`. Run `df -h /proc` and you will see it is type `proc` with size 0: it uses no disk.

5. Where are configuration files, home directories, and logs?

```
cat /etc/hostname
ls /home
ls /var/log | head
sudo du -sh /var/log /var/cache
```

What just happened: `/etc/hostname` is a plain text configuration file. `/home` contains `student`. `/var/log` holds logs and `/var/cache` holds downloaded package files: both grow over time, which is why some administrators put `/var` on its own partition. `du -sh` shows the total size (`-s` summary, `-h` human-readable).

6. Why can everyone write to `/tmp`, but not delete each other's files?

```
ls -ld /tmp
findmnt /tmp
```

```
drwxrwxrwt 14 root root 4096 Oct 10 09:30 /tmp
```

What just happened: `-d` lists the directory itself instead of its contents. `rwxrwxrwx` means everyone can write. The final `t` is the **sticky bit**: in a sticky directory, users can delete only their own files. On Ubuntu 24.04, `findmnt /tmp` usually prints nothing because `/tmp` is a normal folder on the root disk that is cleaned at boot; on some systems (such as Fedora) `/tmp` is a `tmpfs` in RAM.

7. What is under `/usr`?

```
ls /usr
```

```
bin  games  include  lib  lib64  libexec  local  sbin  share  src
```

What just happened: `/usr/bin` and `/usr/sbin` hold programs installed by the package manager, `/usr/share` holds architecture-independent data such as documentation and licenses, and `/usr/local` is for software you install by hand, so the package manager never overwrites it.

Checkpoint: Without looking, say which directory you would check for: a log file (`/var/log`), an SSH server setting (`/etc/ssh`), a kernel image (`/boot`), a disk device (`/dev`), live process information (`/proc`), and a user's documents (`/home`).

Task 8: Identify the architecture and the distribution family

1. Ask for the CPU architecture three different ways.

```
uname -m
lscpu | head -4
dpkg --print-architecture
```

```
x86_64
Architecture:             x86_64
  CPU op-mode(s):         32-bit, 64-bit
  Address sizes:          39 bits physical, 48 bits virtual
  Byte Order:             Little Endian
amd64
```

What just happened: `uname -m` prints the machine hardware name the kernel uses: `x86_64` . `lscpu` shows CPU details; "op-mode(s) 32-bit, 64-bit" means this x86_64 CPU can also run old 32-bit **x86** code. `dpkg --print-architecture` shows the Debian name for the same thing: `amd64` . So **x86_64** and **AMD64** are two names for one architecture. If your VM runs on an Apple silicon Mac, you will see `aarch64` and `arm64` instead: two names for **AArch64**. A **RISC-V** system shows `riscv64` .

2. Check which architecture a program was compiled for.

```
sudo apt install -y file
file -L /bin/ls
```

```
/bin/ls: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, ...
```

What just happened: `file` identifies a file's type; `-L` follows the symbolic link. ELF is the Linux program format, and `x86-64` is the target CPU. A program built for `ARM aarch64` would not run on this VM. This is why distributions publish separate downloads per architecture.

3. Identify the distribution on ubuntu-lab.

```
grep -E '^(NAME|VERSION_ID|ID|ID_LIKE)=' /etc/os-release
dpkg -l | head -7
dpkg -l | grep -c '^ii'
dpkg -S /usr/bin/ls
```

```
NAME="Ubuntu"
VERSION_ID="24.04"
ID=ubuntu
ID_LIKE=debian
Desired=Unknown/Install/Remove/Purge/Hold
| Status=Not/Inst/Conf-files/Unpacked/halF-conf/Half-inst/trig-aWait/Trig-pend
|/ Err?=(none)/Reinst-required (Status,Err: uppercase=bad)
||/ Name           Version        Architecture Description
+++-==============-==============-============-================================
ii  adduser        3.137ubuntu1   all          add and remove users and groups
ii  apparmor       4.0.1-0ubuntu0 amd64        user-space parser utility for AppArmor
612
coreutils: /usr/bin/ls
```

What just happened: `/etc/os-release` exists on every modern distribution. `ID_LIKE=debian` tells you Ubuntu belongs to the Debian (**dpkg-based**) family. `dpkg -l` lists installed packages; `ii` means "desired: install, status: installed". The Architecture column shows `amd64` or `all` (no CPU-specific code). `grep -c '^ii'` counts installed packages. `dpkg -S` searches for the package that owns a file.

4. Run the RPM equivalents on rocky-lab.

```
# on rocky-lab
grep -E '^(NAME|VERSION_ID|ID|ID_LIKE)=' /etc/os-release
rpm -qa | head -3
rpm -qa | wc -l
rpm -qf /usr/bin/ls
uname -m
```

```
NAME="Rocky Linux"
VERSION_ID="9.4"
ID="rocky"
ID_LIKE="rhel centos fedora"
libgcc-11.4.1-3.el9.x86_64
crypto-policies-20240202-1.git283706d.el9.noarch
tzdata-2024a-1.el9.noarch
402
coreutils-8.32-35.el9.x86_64
x86_64
```

What just happened: `ID_LIKE="rhel centos fedora"` marks Rocky as Red Hat family (**RPM-based**). `rpm -qa` queries (`-q`) all (`-a`) installed packages. RPM package names end with the architecture: `x86_64` , or `noarch` for packages with no CPU-specific code (Debian calls this `all`). `rpm -qf` finds the package that owns a file, like `dpkg -S` .

Checkpoint: You can fill in this table from your own output: architecture name from `uname -m` (`x86_64`), Debian name (`amd64`), family of ubuntu-lab (dpkg-based), family of rocky-lab (RPM-based), and the "who owns this file" command on each (`dpkg -S` / `rpm -qf`).

Task 9: Explore the GUI stack and software licenses

TIP

Explore/simulate: Your VMs are servers without a desktop, and installing one takes gigabytes. You will instead ask the system what it would use and read the package descriptions of the GUI pieces.

1. Check which systemd target the system boots into and what kind of session you are using.

```
systemctl get-default
echo "$XDG_SESSION_TYPE"
systemctl status display-manager --no-pager
```

```
graphical.target
tty
Unit display-manager.service could not be found.
```

What just happened: `systemctl get-default` shows the boot target. Ubuntu Server may print `graphical.target` or `multi-user.target` ; with no desktop installed, both end at a text login. Rocky minimal prints `multi-user.target` . `XDG_SESSION_TYPE` is `tty` for a text console or SSH; on a desktop it would be `x11` or `wayland` . `display-manager.service` is an alias that points to whichever **display manager** (GDM, SDDM, LightDM) is installed. Here none is, so the unit is not found.

2. Read what each GUI layer does, using the package descriptions. `apt show` describes a package without installing it; `2>/dev/null` hides a harmless warning about the command-line interface.

```
apt show gdm3 2>/dev/null | grep -A1 '^Description'
apt show xserver-xorg-core 2>/dev/null | grep -A1 '^Description'
apt show openbox 2>/dev/null | grep -A1 '^Description'
apt show libwayland-server0 2>/dev/null | grep -A1 '^Description'
```

What just happened: `grep -A1` prints each match plus one line after it. You should see: `gdm3` is the GNOME Display Manager (the graphical login screen, a **display manager**); `xserver-xorg-core` is the Xorg **X Server** (the classic display server); `openbox` is a standalone **window manager**; and `libwayland-server0` is the library for **Wayland** compositors. On Ubuntu Desktop 24.04, GNOME uses GDM and runs on Wayland by default, with "Ubuntu on Xorg" as a login option. On a desktop you would switch with `sudo systemctl set-default graphical.target`.

3. Look at the license texts Ubuntu ships, and at the copyright file of one package.

```
ls /usr/share/common-licenses/
grep -m 3 -i 'license' /usr/share/doc/bash/copyright
```

```
Apache-2.0  CC0-1.0  GFDL-1.2  GPL    GPL-2  LGPL    LGPL-2.1  MPL-1.1
Artistic    GFDL     GFDL-1.3  GPL-1  GPL-3  LGPL-2  LGPL-3    MPL-2.0
BSD
License: GPL-3+
```

What just happened: `/usr/share/common-licenses/` stores one copy of each common license so packages do not each carry it. Every Debian/Ubuntu package documents its license in `/usr/share/doc/<package>/copyright`. Bash is under the GPL version 3 or later (`GPL-3+`), a **copyleft** license: anyone who distributes a modified bash must release the source under the GPL. Apache-2.0, BSD, and MIT are permissive **open source** licenses.

4. See where Ubuntu keeps non-free software.

```
grep -h '^Components' /etc/apt/sources.list.d/ubuntu.sources
```

```
Components: main restricted universe multiverse
Components: main restricted universe multiverse
```

What just happened: Ubuntu splits its archive into components. `main` and `universe` contain **free software**. `restricted` contains **proprietary** device drivers (such as some GPU firmware), and `multiverse` contains software with license or legal restrictions. You see two lines because the file has one section for the main archive and one for security updates. Your file may list fewer components.

5. On rocky-lab, the license is a field in each RPM package's header.

```
# on rocky-lab
rpm -qi bash | grep -E '^(Name|License)'
rpm -q --qf '%{NAME}: %{LICENSE}\n' openssh zlib
rpm -qa --qf '%{LICENSE}\n' | sort | uniq -c | sort -rn | head -5
```

```
Name        : bash
License     : GPLv3+
openssh: BSD
zlib: zlib and Boost
...
```

What just happened: `rpm -qi` prints a package's information block, including **License**. `--qf` (query format) prints only chosen fields. The last command lists the license of every installed package, then `sort | uniq -c | sort -rn` counts each license and sorts by frequency. You will see GPL variants (copyleft) near the top, along with permissive licenses such as MIT and BSD.

Checkpoint: You can name the display manager, window manager, and display server layers with an example of each, and find a package's license on both systems (`/usr/share/doc/<pkg>/copyright` on Ubuntu, `rpm -qi <pkg>` on Rocky).

Challenge

CHALLENGE

1. On ubuntu-lab, use a one-time edit at the GRUB menu (press `e`) to boot once with `systemd.unit=multi-user.target` added to the `linux` line. After logging in, prove it with `cat /proc/cmdline`, then reboot and prove it is gone.
2. Write a UEFI version of the PXE menu in `~/labs/1.1/pxe/grub.cfg` using `menuentry`, `linux`, and `initrd` lines that boot the same kernel and initrd as your `pxelinux.cfg/default`.
3. Find three packages on rocky-lab whose license is not GPL and describe whether each license is permissive or copyleft.

Clean up

1. On ubuntu-lab, confirm the original GRUB settings are back, delete the backup, and rebuild the configuration one last time to be safe.

```
sudo diff /etc/default/grub /etc/default/grub.lab-backup && echo "identical"
sudo rm /etc/default/grub.lab-backup
sudo update-grub
```

2. Reboot ubuntu-lab and confirm `labtest` is no longer on the kernel command line.

```
sudo reboot
```

```
grep -c labtest /proc/cmdline
```

The second command, run after you log in again, should print `0`.

3. On rocky-lab, confirm no entry still has the test parameter.

```
# on rocky-lab
sudo grubby --info=ALL | grep -c labtest
```

It should print `0`. If not, run `sudo grubby --update-kernel=ALL --remove-args="labtest=1"`.

4. Remove the lab working directory on ubuntu-lab. The `file` package you installed is small and harmless; leave it.

```
rm -rf ~/labs/1.1
```

Exam corner

- Boot order: firmware (BIOS/UEFI) then bootloader (GRUB 2) then kernel then initrd/initramfs then systemd (PID 1) then the default target.
- Edit `/etc/default/grub`, never `grub.cfg`. Rebuild with `update-grub` or `grub-mkconfig -o /boot/grub/grub.cfg` (Debian/Ubuntu) or `grub2-mkconfig -o /boot/grub2/grub.cfg` (RHEL/Rocky). `grubby` changes kernel parameters on RHEL-family systems.
- `GRUB_CMDLINE_LINUX` applies to all entries; `GRUB_CMDLINE_LINUX_DEFAULT` to normal boots only. `GRUB_TIMEOUT` and `GRUB_DEFAULT` control the menu.
- The running kernel's parameters are in `/proc/cmdline`. Press `e` at the GRUB menu for a one-time change.
- The initrd/initramfs holds drivers needed to mount `/`. Ubuntu: `update-initramfs`, `lsinitramfs`. Rocky: `dracut`, `lsinitrd`.
- PXE = network boot: DHCP gives an address and the boot file name, TFTP delivers the bootloader, then the kernel and initrd.
- FHS: `/etc` config, `/var` variable data and logs, `/proc` virtual kernel and process info, `/dev` device files, `/boot` kernel and initrd, `/usr` installed software, `/tmp` temporary (sticky bit), `/home` user files, `/bin` and `/sbin` and `/lib` essential commands and libraries.
- x86_64 = AMD64 = `amd64` (Debian). AArch64 = ARM64 = `arm64` (Debian). RISC-V is an open instruction set. `uname -m` shows the architecture.
- dpkg-based: `.deb`, `dpkg`, `apt` (Debian, Ubuntu). RPM-based: `.rpm`, `rpm`, `dnf` / `yum` / `zypper` (RHEL, Rocky, Fedora, SUSE).
- Display manager = graphical login (GDM, SDDM, LightDM). Window manager = places and decorates windows. X Server (Xorg) and Wayland are display servers; Wayland is the modern default.
- Copyleft (GPL) requires derivative works to keep the same license. Permissive (MIT, BSD, Apache) does not. Proprietary = closed source, restricted redistribution. Free software = the four freedoms.

Review questions

1. An administrator added `nomodeset` to `GRUB_CMDLINE_LINUX_DEFAULT` in `/etc/default/grub` on an Ubuntu server and rebooted, but `/proc/cmdline` does not show it. What step was missed?

   A. Running `update-initramfs -u`
   B. Running `update-grub`
   C. Editing `/proc/cmdline`
   D. Running `systemctl daemon-reload`

2. Which part of the boot process contains the drivers needed to mount the real root filesystem?

   A. The display manager
   B. `/etc/fstab`
   C. The initrd (initramfs)
   D. The UEFI system partition

3. A server must be installed automatically with no local media. The network card asks a DHCP server for a boot file name and then downloads it. Which technology is this, and which file transfer protocol does it typically use first?

4. Which directory contains files that are generated by the kernel and use no disk space, such as `cpuinfo`?

   A. `/dev`
   B. `/var`
   C. `/proc`
   D. `/sys/firmware`

5. `uname -m` prints `aarch64` on a cloud server. Which Debian architecture name do its packages use?

   A. `amd64`
   B. `arm64`
   C. `i386`
   D. `riscv64`

6. A company modifies a GPL-licensed program and sells it to customers. What does the copyleft clause require?

   A. Nothing, because the GPL is a permissive license
   B. They must release the modified source code under the GPL to those customers
   C. They must pay a royalty to the original author
   D. They must convert the program to a proprietary license

Lab 1.5 Use Common Shell Operations

Objective: 1.5 Given a scenario, manage a Linux system using common shell operations.
Domain: 1.0 System Management (23% of the exam)
Time: about 150 minutes (you can split it at any task boundary)
VM: ubuntu-lab
Snapshot first: Optional. This lab only changes files in your home directory, and the clean-up restores them.

The shell is where a Linux administrator spends most of the day. In this lab you build a small practice dataset (a fake web server log and a server inventory in CSV format) and then use it to practise the everyday shell skills: moving around the file system, working with environment variables, customizing your prompt and `PATH`, redirecting input and output, slicing text with classic tools like `grep`, `awk`, and `sed`, reusing your command history, and editing files in `vim` and `nano`. The exam tests these skills with "which command would…" scenarios, so the goal is that each tool feels familiar.

What you'll practice

- Environment variables: `DISPLAY`, `HOME`, `PATH`, `PS1`, `SHELL`, `USER`
- Paths: absolute (`~`, `/`) and relative (`.`, `..`, `-`)
- Shell environment configs: `.bashrc`, `.bash_profile`, `.profile` (login and non-login shells)
- Redirection: `<`, `>`, `<<`, `>>`, `|`; standard output, standard error, standard input; here docs; here strings `<<<`
- Utilities: `!`, `!!`, `alias`, `awk`, `bc`, `cat`, `cut`, `echo`, `grep`, `head`, `history`, `less`, `more`, `printf`, `sed`, `sort`, `source`, `tail`, `tee`, `tr`, `uname`, `uniq`, `wc`, `xargs`
- Editors: `vi` / `vim` and `nano`

How it works

The shell. When you log in, Linux starts a program called a *shell*. The shell reads the line you type, works out what you mean (expanding `~`, `*`, and `$VARIABLES`), finds the program, and runs it. On Ubuntu and Rocky the default shell is **Bash** (`/bin/bash`). Everything in this lab is Bash.

Paths. Every file lives somewhere in one big tree that starts at `/` (the *root* directory). A path tells the shell where a file is:

- An **absolute** path starts at the root, so it always works no matter where you are: `/etc/hostname`, `/home/student`. The shortcut `~` means "my home directory", so `~/labs` is also absolute (the shell turns it into `/home/student/labs`).
- A **relative** path starts from your *current working directory* (where you are now). `.` means "this directory", `..` means "the parent directory", and `cd -` means "go back to the directory I was just in".

Think of an absolute path as a full postal address and a relative path as “two doors down from here”.

Variables and the environment. The shell keeps named values called *variables*. Some of them are *environment variables*: they are copied into every program the shell starts, so programs can read them. Important ones:

Variable	Holds	Example
`HOME`	Your home directory	`/home/student`
`USER`	Your user name	`student`
`SHELL`	Your login shell	`/bin/bash`
`PATH`	Directories searched for commands, separated by `:`	`/usr/local/bin:/usr/bin:/bin...`
`PS1`	The text of your prompt	`\u@\h:\w\$`
`DISPLAY`	Which X11 graphical display GUI programs should draw on	`:0` or `localhost:10.0` (empty on a server)

When you type `ls`, the shell does not search the whole disk. It looks in each directory in `PATH`, left to right, and runs the first `ls` it finds. That is why a script in a directory that is not in `PATH` gives “command not found” until you type its path or add the directory to `PATH`.

Startup files. Variables you set at the prompt disappear when you log out. To make a setting permanent you put it in a *startup file* that Bash reads when it starts. Which file depends on the type of shell:

```
LOGIN shell (SSH login, console login, "bash -l")
  /etc/profile
  then the FIRST one that exists of:
    ~/.bash_profile  ->  ~/.bash_login  ->  ~/.profile
                                              |
                      (Ubuntu's ~/.profile sources ~/.bashrc)

NON-LOGIN interactive shell (typing "bash", a new terminal tab in a GUI)
  /etc/bash.bashrc
  ~/.bashrc
```

Ubuntu ships `~/.profile` and `~/.bashrc` for every new user but **no** `~/.bash_profile`. Its `~/.profile` runs `~/.bashrc`, so both kinds of shell end up reading `~/.bashrc`. The usual rule is: put **environment variables** such as `PATH` in `~/.profile` (read once at login, then inherited by every child process), and put **aliases, prompt settings, and functions** in `~/.bashrc` (read by every interactive shell, because aliases are *not* inherited). If you create `~/.bash_profile`, Bash reads it *instead of* `~/.profile`, which surprises many people. You will prove that in Task 4. The `source` command (or its short form `.`) reads a file into the *current* shell, so you can apply changes without logging out.

Streams and redirection. Every program starts with three open “pipes” called *streams*:

```
                    +-------------+  stdout (1) --> screen
keyboard --> stdin  |   program   |
             (0)    +-------------+  stderr (2) --> screen
```

- **standard input** (stdin, number 0): where the program reads input. Normally the keyboard.
- **standard output** (stdout, number 1): normal results. Normally the screen.
- **standard error** (stderr, number 2): error messages. Also the screen, but a separate stream.

Redirection reconnects these streams to files: `>` sends stdout to a file (replacing it), `>>` appends, `2>` sends stderr, `<` feeds a file into stdin, `<<` (a *here doc*) feeds lines you type, and `<<<` (a *here string*) feeds one string. A *pipe* `|` connects the stdout of one program to the stdin of the next, which lets you build a "pipeline" of small tools that each do one job. That idea, small tools joined by pipes, is the heart of Unix text processing.

Editors. Sooner or later you will edit a config file on a server with no GUI. `nano` is simple and shows its shortcuts on screen. `vi` / `vim` is on almost every Unix system ever made, is very powerful, and is *modal*: keys mean different things in different modes. You will practise both.

Tasks

Task 1: Prepare the lab and build a practice dataset

1. Log in to ubuntu-lab as `student` (over SSH or on the console). Install the tools this lab uses. Most are already present; `apt` will say so.

```
sudo apt update
sudo apt install -y bc vim nano less
```

> **What just happened:** `apt update` refreshes the list of available packages, and `apt install -y` installs the named packages, answering "yes" to the confirmation. `bc` is a command-line calculator, `vim` is the improved `vi` editor, `nano` is a simple editor, and `less` is a pager (a program that shows long text one screen at a time). Tools like `grep`, `sed`, `awk`, `sort`, and `cut` are part of every Ubuntu install.

2. Create the lab working directory and move into it.

```
mkdir -p ~/labs/1.5 && cd ~/labs/1.5
```

`mkdir -p` creates the directory and any missing parent directories without complaining if they already exist. `&&` means "run the second command only if the first one succeeded".

3. Create a fake web server access log using a **here doc**. Copy the whole block, including the final `EOF` line.

```
cat > access.log <<'EOF'
192.168.100.20 - - [10/Oct/2026:08:01:12 +0000] "GET /index.html HTTP/1.1" 200 5120
192.168.100.20 - - [10/Oct/2026:08:01:13 +0000] "GET /style.css HTTP/1.1" 200 1830
10.0.0.15 - - [10/Oct/2026:08:02:40 +0000] "GET /login HTTP/1.1" 200 2310
10.0.0.15 - - [10/Oct/2026:08:02:55 +0000] "POST /login HTTP/1.1" 302 0
203.0.113.7 - - [10/Oct/2026:08:03:01 +0000] "GET /admin HTTP/1.1" 403 512
203.0.113.7 - - [10/Oct/2026:08:03:02 +0000] "GET /wp-login.php HTTP/1.1" 404 320
203.0.113.7 - - [10/Oct/2026:08:03:03 +0000] "GET /.env HTTP/1.1" 404 320
10.0.0.22 - - [10/Oct/2026:08:05:10 +0000] "GET /index.html HTTP/1.1" 200 5120
10.0.0.22 - - [10/Oct/2026:08:05:11 +0000] "GET /images/logo.png HTTP/1.1" 200 20480
192.168.100.20 - - [10/Oct/2026:08:07:45 +0000] "GET /reports HTTP/1.1" 500 128
10.0.0.15 - - [10/Oct/2026:08:09:30 +0000] "GET /dashboard HTTP/1.1" 200 8800
203.0.113.7 - - [10/Oct/2026:08:10:00 +0000] "GET /phpmyadmin HTTP/1.1" 404 320
198.51.100.4 - - [10/Oct/2026:08:12:19 +0000] "GET /index.html HTTP/1.1" 200 5120
198.51.100.4 - - [10/Oct/2026:08:12:20 +0000] "GET /style.css HTTP/1.1" 304 0
10.0.0.22 - - [10/Oct/2026:08:15:42 +0000] "GET /reports HTTP/1.1" 500 128
EOF
```

What just happened: `cat > access.log` means "run `cat` and send its standard output into the file `access.log`". Normally `cat` would read from the keyboard, but `<<'EOF'` starts a **here doc**: the shell collects every following line until it sees a line containing only `EOF`, and feeds those lines to `cat` as standard input. `cat` copies them to its output, which lands in the file. `EOF` is just a conventional marker word; any word works. The quotes around `'EOF'` tell the shell *not* to expand `$variables` or backticks inside the text, so it is copied exactly. Each log line has: client IP, two unused fields (`-`), a timestamp, the HTTP request, the status code (200 = OK, 404 = not found, 500 = server error), and the number of bytes sent.

4. Create a server inventory in CSV (comma-separated values) format the same way.

```
cat > servers.csv <<'EOF'
hostname,os,ram_gb,role,site
web01,ubuntu,4,web,london
web02,ubuntu,4,web,london
db01,rocky,16,database,paris
db02,rocky,16,database,berlin
mon01,ubuntu,2,monitoring,paris
app01,debian,8,app,berlin
app02,ubuntu,8,app,london
EOF
```

5. See the difference between a quoted and an unquoted here doc marker.

```
cat <<EOF
Unquoted: my home is $HOME
EOF
cat <<'EOF'
Quoted: my home is $HOME
EOF
```

```
Unquoted: my home is /home/student
Quoted: my home is $HOME
```

What just happened: With an unquoted marker the shell expands variables inside the here doc, which is handy for generating config files that contain the current user or hostname. With a quoted marker the text is taken literally. When you write scripts or config files that contain `$`, quote the marker.

6. Check which system you are on with `uname`.

```
uname -a
uname -r
uname -m
```

```
Linux ubuntu-lab 6.8.0-45-generic #45-Ubuntu SMP PREEMPT_DYNAMIC ... x86_64 x86_64 x86_64
GNU/Linux
6.8.0-45-generic
x86_64
```

What just happened: `uname` prints system information. `-a` prints all of it: kernel name (`Linux`), host name, kernel release, kernel build version, machine hardware type, and operating system. `-r` prints only the kernel release, which you check before installing drivers or after a kernel update. `-m` prints the CPU architecture (`x86_64` or `aarch64`). Other options: `-s` kernel name, `-n` host name, `-o` operating system. Your kernel number will differ.

What just happened: You now have two small but realistic files to practise on. `access.log` imitates the log an Apache or Nginx web server writes to `/var/log/apache2/access.log` or `/var/log/nginx/access.log`, and `servers.csv` imitates the kind of inventory export you get from a spreadsheet or a monitoring system. Every command in the rest of the lab works the same way on real files, only with more lines.

Checkpoint: `ls -l ~/labs/1.5` shows `access.log` and `servers.csv`. `wc -l access.log servers.csv` reports 15 and 8 lines.

Task 2: Navigate with absolute and relative paths

1. Find out where you are, then jump around using absolute paths.

```
pwd
cd /
pwd
cd /var/log
pwd
cd ~
pwd
```

```
/home/student/labs/1.5
/
/var/log
/home/student
```

> **What just happened:** `pwd` (print working directory) shows your current directory. `cd /` moves to the root of the tree. `cd /var/log` uses an absolute path: it starts with `/`, so it works from anywhere. `cd ~` goes home; plain `cd` with no argument does the same thing.

2. Now use relative paths.

```
cd labs/1.5
pwd
cd ..
pwd
cd ./1.5
pwd
cd ../..
pwd
```

```
/home/student/labs/1.5
/home/student/labs
/home/student/labs/1.5
/home/student
```

> **What just happened:** `labs/1.5` has no leading `/`, so the shell looks for `labs` inside the current directory (`/home/student`). `..` is the parent directory, so `cd ..` goes up one level, and `../..` goes up two. `./1.5` means "`1.5` in this directory"; the `./` is optional for `cd` but required when you run a program from the current directory (for example `./myscript.sh`), because `.` is not in `PATH`.

3. Use `cd -` to bounce between two directories.

```
cd /etc
cd ~/labs/1.5
cd -
cd -
```

```
/etc
/home/student/labs/1.5
```

> **What just happened:** `cd -` switches to the previous directory and prints its name. Bash stores the previous directory in the variable `OLDPWD`. This is very useful when you are editing a config in `/etc` and checking results in another directory.

4. See `.` and `..` as real directory entries, and use `~` with another user name.

```
ls -a ~/labs/1.5
ls -ld ~root
```

```
.  ..  access.log  servers.csv
drwx------ 5 root root 4096 Oct 10 07:55 /root
```

What just happened: `ls -a` shows all entries, including hidden ones whose names start with a dot. Every directory contains `.` (itself) and `..` (its parent). `~root` expands to the home directory of the user `root`, which is `/root`. `ls -ld` shows the directory itself instead of its contents.

Checkpoint: Starting from `/tmp`, you can reach `~/labs/1.5` with one absolute `cd` and get back to `/tmp` with `cd -`. Try it, and run `pwd` after each step.

Task 3: Explore environment variables

1. Print the key variables.

```
echo "$HOME"
echo "$USER"
echo "$SHELL"
echo "$PATH"
echo "$PS1"
```

```
/home/student
student
/bin/bash
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/sn
ap/bin
\[\e]0;\u@\h: \w\a\]${debian_chroot:+($debian_chroot)}\[\033[01;32m\]\u@\h\[\033[00m\]:...
```

What just happened: `echo` prints its arguments. A `$` in front of a name tells the shell to replace it with the variable's value *before* `echo` runs. The double quotes keep the value as one argument even if it contains spaces. `PATH` is a list of directories separated by colons. Ubuntu's `PS1` looks scary because it contains color codes and a window-title code; you will replace it with something readable in Task 4. Your `PATH` may also contain `/home/student/.local/bin` or `/home/student/bin` if those directories existed when you logged in.

2. List all environment variables and pick out a few.

```
printenv | sort | head -n 15
printenv HOME
env | grep -E '^(USER|SHELL|LANG)='
```

What just happened: `printenv` (and `env` with no arguments) prints every *environment* variable as `NAME=value`. You piped the output into `sort` to alphabetize it and `head -n 15` to see only the first 15 lines. `printenv HOME` prints one variable without the `$`. `grep -E` searches using an extended regular expression; `^` means "start of line", so this matches lines that begin with exactly `USER=`, `SHELL=`, or `LANG=`.

3. Learn the difference between a shell variable and an environment variable.

```
COLOR=blue
echo "$COLOR"
bash -c 'echo "child shell sees: [$COLOR]"'
export COLOR
bash -c 'echo "child shell sees: [$COLOR]"'
unset COLOR
```

```
blue
child shell sees: []
child shell sees: [blue]
```

What just happened: `COLOR=blue` (no spaces around `=`) creates a *shell variable* that only the current shell knows. `bash -c '...'` starts a child shell that runs one command; it saw nothing because shell variables are not passed to children. `export COLOR` marks the variable for export, turning it into an *environment variable*, so the child now sees it. You can also do both in one step: `export COLOR=blue`. `unset` deletes the variable. This is why `PATH` must be exported (it already is): programs started by your shell need it.

4. Look at `DISPLAY`. ubuntu-lab is a server with no graphical desktop.

```
echo "DISPLAY is: [$DISPLAY]"
```

```
DISPLAY is: []
```

What just happened: `DISPLAY` tells graphical (X11) programs where to draw their windows. The format is `host:display.screen`; `:0` means "the first display on this machine". On a server with no GUI it is empty, so a graphical program such as `xeyes` or `virt-manager` would fail with "cannot open display". The fix is *X11 forwarding*: if you connect with `ssh -X student@ubuntu-lab` from a computer that runs an X server (a Linux desktop, macOS with XQuartz, or Windows with an X server such as VcXsrv or WSLg), SSH sets `DISPLAY` to something like `localhost:10.0` and tunnels the windows back to your screen. The server needs `X11Forwarding yes` in `/etc/ssh/sshd_config` (Ubuntu's default) and the `xauth` package.

TIP

Explore/simulate: If your workstation has an X server, try `ssh -X student@ubuntu-lab`, then `echo $DISPLAY` (you should see `localhost:10.0`), then `sudo apt install -y x11-apps` and `xeyes`. Without an X server, just check the server side: `grep -i x11forwarding /etc/ssh/sshd_config` shows `X11Forwarding yes`. Remove `x11-apps` afterwards if you installed it.

Checkpoint: You can explain why `bash -c 'echo $COLOR'` printed nothing until you ran `export`, and `echo $DISPLAY` prints an empty line on ubuntu-lab.

Task 4: Customize your shell with PS1, PATH, aliases, and startup files

In this task you change your prompt and `PATH`, first temporarily, then permanently. Start by backing up the two startup files so the clean-up can restore them exactly.

1. Back up your startup files.

```
cp ~/.bashrc ~/.bashrc.lab15.bak
cp ~/.profile ~/.profile.lab15.bak
ls ~/.bash_profile
```

```
ls: cannot access '/home/student/.bash_profile': No such file or directory
```

The error is expected: Ubuntu does not create `~/.bash_profile`. (If yours exists, back it up too.)

2. Change the prompt for the current shell only.

```
PS1='[\u@\h \W]\$ '
```

Your prompt becomes `[student@ubuntu-lab 1.5]$`. Now try a two-line prompt with the time:

```
PS1='\t \u@\h:\w\n\$ '
```

> **What just happened:** `PS1` is the *primary prompt string*. Bash replaces special backslash codes each time it draws the prompt: `\u` user name, `\h` short host name, `\w` full current directory (with `~` for home), `\W` only the last part of the directory, `\t` time (24-hour HH:MM:SS), `\n` a new line, and `\$` shows `$` for normal users and `#` for root. Single quotes stop the shell from interpreting the backslashes when you set the variable. Because you did not save it anywhere, the change disappears when this shell exits.

3. Make a personal tools directory with a small script and try to run it.

```
mkdir -p ~/labtools
cat > ~/labtools/hello-lab <<'EOF'
#!/bin/bash
echo "Hello from $(hostname), I am running as $USER"
EOF
chmod +x ~/labtools/hello-lab
hello-lab
```

```
hello-lab: command not found
```

> **What just happened:** You wrote a two-line script. `#!/bin/bash` (the "shebang") tells the kernel which program runs the file, and `chmod +x` makes it executable. Typing `hello-lab` fails because the shell only searches the directories in `PATH`, and `~/labtools` is not one of them. You could still run it with its full path: `~/labtools/hello-lab`.

4. Add the directory to `PATH` for this shell and test it.

```
export PATH="$PATH:$HOME/labtools"
hello-lab
which hello-lab
```

```
Hello from ubuntu-lab, I am running as student
/home/student/labtools/hello-lab
```

What just happened: `"$PATH:$HOME/labtools"` builds a new value from the old `PATH` plus a colon and your directory, so nothing existing is lost. Putting your directory at the *end* means system commands win if names clash; putting it at the *front* (`"$HOME/labtools:$PATH"`) would let your version override a system command. Never replace `PATH` with only your directory, or commands like `ls` will stop working in that shell. `which` shows which file the shell would run.

5. Make the changes permanent. `PATH` belongs in `~/.profile`; the prompt and aliases belong in `~/.bashrc`. The `>>` operator *appends* to a file instead of overwriting it, which is critical here: a single `>` would wipe out your existing `.bashrc`.

```
cat >> ~/.profile <<'EOF'

# >>> lab 1.5 >>>
export PATH="$PATH:$HOME/labtools"
# <<< lab 1.5 <<<
EOF
```

```
cat >> ~/.bashrc <<'EOF'

# >>> lab 1.5 >>>
PS1='[\t] \u@\h:\w\$ '
alias ll='ls -alF'
alias lab='cd ~/labs/1.5'
alias grep='grep --color=auto'
# <<< lab 1.5 <<<
EOF
tail -n 8 ~/.bashrc
```

What just happened: You appended a marked block to each file. The marker comments make the block easy to find and remove later. An `alias` is a short name that the shell replaces with longer text: after this, typing `ll` runs `ls -alF`. Ubuntu's `~/.bashrc` already defines `ll`, `la`, `l`, and a colorized `grep`; defining them again is harmless, and the later definition wins. You added your lines at the *end* of `~/.bashrc` on purpose: near the top, Ubuntu's file stops early ("return") for non-interactive shells, so settings at the end only affect interactive shells, which is what you want for prompts and aliases. You could make the same edits with `nano ~/.bashrc`; the here doc just avoids typos.

6. Apply the changes without logging out by sourcing the files.

```
source ~/.bashrc
alias
lab
pwd
```

```
...
alias grep='grep --color=auto'
alias l='ls -CF'
alias la='ls -A'
alias lab='cd ~/labs/1.5'
alias ll='ls -alF'
alias ls='ls --color=auto'
...
/home/student/labs/1.5
```

What just happened: `source ~/.bashrc` runs the file's commands *inside the current shell*, so the new prompt and aliases take effect right now. (Running `bash ~/.bashrc` would not work: it would apply them in a child shell that exits immediately.) `. ~/.bashrc` is the same as `source ~/.bashrc`. `alias` with no arguments lists all aliases; `alias ll` shows one; `unalias name` removes one; and `\ls` or `command ls` runs the real command, skipping any alias. Your prompt now looks like `[14:05:31] student@ubuntu-lab:~/labs/1.5$`.

7. Tell login shells from non-login shells.

```
echo $0
shopt login_shell
bash
echo $0
shopt login_shell
exit
```

```
-bash
login_shell     on
bash
login_shell     off
```

What just happened: Your SSH or console session is a *login shell*. Bash shows this by putting a dash in front of its name (`-bash`) and setting the `login_shell` option. Typing `bash` started a *non-login* interactive shell inside it, which reads only `/etc/bash.bashrc` and `~/.bashrc`. `exit` returned you to the login shell. `bash -l` (or `su - username`) starts a login shell, which reads `/etc/profile` and then `~/.bash_profile`, `~/.bash_login`, or `~/.profile`.

8. Prove that `~/.bash_profile` hides `~/.profile`. Create a one-line `~/.bash_profile`, then start a login shell and look for your `lab` alias (which only reaches login shells because `~/.profile` sources `~/.bashrc`).

```
echo 'echo "~/.bash_profile was read"' > ~/.bash_profile
bash -l
type lab
exit
```

```
~/.bash_profile was read
bash: type: lab: not found
```

Now fix it the standard way, by making `~/.bash_profile` read `~/.profile`:

```
echo '[ -f ~/.profile ] && . ~/.profile' >> ~/.bash_profile
bash -l
type lab
exit
rm ~/.bash_profile
```

```
~/.bash_profile was read
lab is aliased to `cd ~/labs/1.5'
```

What just happened: When `~/.bash_profile` exists, a login Bash reads it and **skips** `~/.profile` entirely, so Ubuntu's chain (`.profile` sources `.bashrc`) never ran and your alias was missing. Adding `[ -f ~/.profile ] && . ~/.profile` ("if the file exists, source it") restored the chain. Rocky Linux uses `~/.bash_profile` (which sources `~/.bashrc`) instead of `~/.profile`. You deleted the test file so your account is back to Ubuntu's normal layout. `type` tells you what a name is: an alias, a shell builtin, a function, or a file on disk.

9. Confirm the `PATH` change works in a fresh login. Log out (`exit`) and log back in over SSH, then:

```
echo "$PATH" | tr ':' '\n' | tail -n 3
hello-lab
```

```
/usr/local/games
/snap/bin
/home/student/labtools
Hello from ubuntu-lab, I am running as student
```

TIP

Ubuntu's `~/.profile` already adds `~/bin` and `~/.local/bin` to `PATH` at login **if those directories exist**. So the simplest way to get a personal script directory on Ubuntu is `mkdir ~/bin`, then log out and back in. This lab used `~/labtools` so you could see the whole process.

ON THE EXAM

On the exam, remember: `~/.bashrc` is for non-login interactive shells (aliases, prompt), `~/.bash_profile` or `~/.profile` is for login shells (environment variables), and `source file` (or `. file`) applies a file to the current shell. System-wide versions live in `/etc/profile`, `/etc/profile.d/*.sh`, and `/etc/bash.bashrc` (Rocky: `/etc/bashrc`).

Checkpoint: After a fresh login, your prompt starts with the time, `ll` and `lab` work, and `hello-lab` runs from any directory. `ls ~/.bash_profile` reports "No such file or directory".

Task 5: Redirect standard input, output, and error

1. Run a command that produces both normal output and an error.

```
cd ~/labs/1.5
ls /etc/hostname /nope
```

```
ls: cannot access '/nope': No such file or directory
/etc/hostname
```

Both lines appear on the screen, but they come from different streams: the error from **standard error** and the file name from **standard output**.

2. Send each stream to its own file.

```
ls /etc/hostname /nope > out.txt
cat out.txt
ls /etc/hostname /nope 2> err.txt
cat err.txt
```

```
ls: cannot access '/nope': No such file or directory
/etc/hostname
/etc/hostname
ls: cannot access '/nope': No such file or directory
```

What just happened: `>` is short for `1>`: it redirects stream 1 (stdout) to `out.txt`, so only the error still reached your screen. `2>` redirects stream 2 (stderr) to `err.txt`, so only the normal output reached the screen. A single `>` creates the file or **empties it first** if it already exists.

3. Append instead of overwrite.

```
echo "first line" > notes.txt
echo "second line" >> notes.txt
echo "oops" > notes.txt
cat notes.txt
```

```
oops
```

The third command used `>` and wiped out the first two lines. Use `>>` whenever you want to keep what is already in a file (log files, `~/.bashrc`).

> **TIP**
> `set -o noclobber` makes Bash refuse to overwrite an existing file with `>` (you then need `>|` to force it). Some admins add it to `~/.bashrc` as a safety net. Turn it off with `set +o noclobber`.

4. Send both streams to one file, and learn why order matters.

```
ls /etc/hostname /nope > all.txt 2>&1
cat all.txt
ls /etc/hostname /nope &> all2.txt
cat all2.txt
ls /etc/hostname /nope 2>&1 > wrong.txt
```

```
ls: cannot access '/nope': No such file or directory
/etc/hostname
ls: cannot access '/nope': No such file or directory
/etc/hostname
ls: cannot access '/nope': No such file or directory
```

> **What just happened:** `2>&1` means "send stream 2 to wherever stream 1 is going *right now*". The shell processes redirections left to right. In the first command, stdout was already pointing to `all.txt`, so stderr followed it there. In the last command, `2>&1` came first, while stdout still pointed at the screen, so stderr went to the screen and only stdout went to `wrong.txt`. `&>` is a Bash shortcut for "both streams to this file" (`&>>` appends both).

5. Throw away output you do not want with `/dev/null`.

```
find /etc -name "*.conf" 2> /dev/null | head -n 5
grep -r "PermitRootLogin" /etc/ssh 2>/dev/null
ls /nope &> /dev/null
echo "exit status: $?"
```

> **What just happened:** `/dev/null` is a special file that discards everything written to it (the "bit bucket"). Running `find` as a normal user produces many "Permission denied" errors; `2> /dev/null` hides them so you see only real results. The last `ls` hid all output, but you can still check whether it worked: `$?` holds the *exit status* of the previous command, `0` for success and non-zero (here `2`) for failure. Scripts use this pattern constantly: "run quietly and just tell me if it worked".

6. Feed a file to standard input with `<`, and a single string with a **here string** `<<<`.

```
wc -l < access.log
tr 'a-z' 'A-Z' < servers.csv | head -n 2
wc -w <<< "one two three"
bc <<< "2^10"
```

```
15
HOSTNAME,OS,RAM_GB,ROLE,SITE
WEB01,UBUNTU,4,WEB,LONDON
3
1024
```

What just happened: `< access.log` connects the file to the program's standard input. `wc -l access.log` would print `15 access.log`, but `wc -l < access.log` prints just `15` because `wc` never learns the file name, which is handy in scripts. `tr` only reads standard input (it cannot take a file name), so `<` is the normal way to feed it. `<<<` passes one string, followed by a newline, as standard input: a one-line version of a here doc.

7. Pipes carry only standard output.

```
ls /nope | wc -l
ls /nope 2>&1 | wc -l
```

```
ls: cannot access '/nope': No such file or directory
0
1
```

What just happened: The pipe `|` connects stdout of `ls` to stdin of `wc`. The error went to stderr, straight to your screen, so `wc` counted 0 lines. With `2>&1` before the pipe, stderr joins stdout and flows into `wc`. Bash also accepts `|&` as a shortcut for `2>&1 |`.

8. Use `tee` to save output *and* keep it flowing down the pipeline.

```
grep ' 404 ' access.log | tee 404s.txt | wc -l
cat 404s.txt
date | tee -a run.log
date | tee -a run.log
cat run.log
```

```
3
203.0.113.7 - - [10/Oct/2026:08:03:02 +0000] "GET /wp-login.php HTTP/1.1" 404 320
...
```

What just happened: `tee` is named after a T-shaped pipe fitting: it copies its standard input to a file *and* to its standard output. Here the 404 lines were saved in `404s.txt` while `wc -l` counted them. `tee -a` appends instead of overwriting.

9. Use `tee` to write a file that needs root permission.

```
sudo echo "lab 1.5 was here" > /opt/lab15.txt
echo "lab 1.5 was here" | sudo tee /opt/lab15.txt
cat /opt/lab15.txt
```

```
-bash: /opt/lab15.txt: Permission denied
lab 1.5 was here
lab 1.5 was here
```

What just happened: The first command failed even with `sudo`. The redirection `>` is performed by *your* shell, running as `student`, before `sudo` even starts; only `echo` ran as root. In the second command, `tee` itself runs as root through `sudo`, so it can write the file. This `| sudo tee file` pattern (or `| sudo tee -a file` to append) is how you write to protected files from a pipeline, and it is the pattern this book uses with here docs: `cat <<'EOF' | sudo tee /etc/...`.

Checkpoint: `cat all.txt` shows both the error and the file name; `cat wrong.txt` shows only `/etc/hostname`; `cat /opt/lab15.txt` shows `lab 1.5 was here`.

Task 6: View and filter text with cat, less, more, head, tail, grep, cut, sort, uniq, and wc

1. View files in different ways.

```
cat servers.csv
cat -n servers.csv
head -n 3 access.log
tail -n 2 access.log
tail -n +2 servers.csv
```

What just happened: `cat` (concatenate) prints whole files; with several files it joins them (`cat a b > c`). `-n` numbers the lines. `head -n 3` shows the first 3 lines (default 10), and `tail -n 2` the last 2. `tail -n +2` is special: it means "start from line 2", which is the standard way to skip a CSV header row.

2. Page through long output with `less` and `more`.

```
less /etc/services
```

Inside `less`: press Space or Page Down to go forward, `b` to go back, `g` for the top, `G` for the bottom, type `/ssh` and press Enter to search, `n` for the next match, `N` for the previous, and `q` to quit. Now try `more`:

```
more /etc/services
```

Press Space for the next page, Enter for the next line, and `q` to quit.

What just happened: `more` is the original pager; it mainly moves forward. `less` is newer and can scroll both ways and search ("less is more"). Commands like `man` and `systemctl` use `less` to show long output. Pipe any long output into it: `ps aux | less`.

3. Watch a growing file with `tail -f`. Start a background job that appends a line every 2 seconds, then follow the file:

```
(for i in 1 2 3 4 5; do echo "event $i"; sleep 2; done) >> live.log &
tail -f live.log
```

Watch the events appear, then press Ctrl+C to stop `tail`.

> **What just happened:** The parentheses run the loop in a subshell, `>> live.log` appends its output to the file, and `&` runs it in the background so you get your prompt back. `tail -f` (follow) prints the end of the file and then waits, printing new lines as they arrive. This is how admins watch logs live, for example `sudo tail -f /var/log/syslog`.

4. Search with `grep`.

```
grep '404' access.log
grep -c ' 404 ' access.log
grep -v ' 200 ' access.log | wc -l
grep -i 'post' access.log
grep -n 'reports' access.log
grep -E ' (403|404) ' access.log | wc -l
```

```
...
3
8
10.0.0.15 - - [10/Oct/2026:08:02:55 +0000] "POST /login HTTP/1.1" 302 0
10:192.168.100.20 - - [10/Oct/2026:08:07:45 +0000] "GET /reports HTTP/1.1" 500 128
15:10.0.0.22 - - [10/Oct/2026:08:15:42 +0000] "GET /reports HTTP/1.1" 500 128
4
```

> **What just happened:** `grep pattern file` prints lines that contain the pattern. `-c` counts matching lines instead of printing them. The spaces in `' 404 '` stop `404` from matching inside a byte count or timestamp. `-v` inverts the match (lines that do *not* contain `200`). `-i` ignores case. `-n` adds line numbers. `-E` enables extended regular expressions, where `(403|404)` means "403 or 404". Other useful options: `-r` searches directories recursively, `-w` matches whole words only, and `-l` lists only file names.

5. Cut out columns with `cut`.

```
cut -d, -f1,3 servers.csv | head -n 4
cut -d' ' -f1 access.log | head -n 3
```

```
hostname,ram_gb
web01,4
web02,4
db01,16
192.168.100.20
192.168.100.20
10.0.0.15
```

What just happened: `cut -d, -f1,3` splits each line at commas (`-d` sets the delimiter) and keeps fields 1 and 3. In the log, fields are separated by spaces, so `-d' '` with `-f1` extracts the client IP. `cut` treats every single delimiter as a boundary, so it is perfect for CSV but awkward when columns are separated by a varying number of spaces; that is a job for `awk` (next task). `cut -c1-10` cuts by character position instead.

6. Sort, de-duplicate, and count: the most common log analysis pipeline. Which IP address made the most requests?

```
cut -d' ' -f1 access.log | sort | uniq -c | sort -rn
```

```
      4 203.0.113.7
      3 192.168.100.20
      3 10.0.0.22
      3 10.0.0.15
      2 198.51.100.4
```

What just happened: Read the pipeline left to right. `cut` extracts the IPs. `sort` puts identical IPs next to each other. `uniq -c` collapses adjacent identical lines into one and prefixes a count; `uniq` only compares *neighboring* lines, which is why you must `sort` first. The final `sort -rn` sorts numerically (`-n`) in reverse (`-r`), so the busiest client is on top. Lines with equal counts may appear in a different order on your system. `203.0.113.7` is the client probing for `/admin`, `/.env`, and `/phpmyadmin`: an attacker scanning for weaknesses.

7. More sorting and counting.

```
tail -n +2 servers.csv | sort -t, -k3,3n | head -n 3
tail -n +2 servers.csv | cut -d, -f5 | sort -u
tail -n +2 servers.csv | cut -d, -f2 | sort | uniq -c
wc access.log
wc -l servers.csv
```

```
mon01,ubuntu,2,monitoring,paris
web01,ubuntu,4,web,london
web02,ubuntu,4,web,london
berlin
london
paris
      1 debian
      2 rocky
      4 ubuntu
  15  150 1181 access.log
8 servers.csv
```

What just happened: `sort -t, -k3,3n` sorts by field 3 only (`-k3,3`), using a comma as the field separator (`-t,`), numerically (`n`). Without `n`, `16` would sort before `2` because text is compared character by character. `sort -u` sorts and removes duplicates in one step, the same as `sort | uniq`. `wc` (word count) prints lines, words, and bytes; `-l`, `-w`, and `-c` pick one.

Checkpoint: Using one pipeline, list each HTTP status code with how many times it appears, most common first. (Hint: the status is field 9 when you split on spaces: `cut -d' ' -f9 access.log | sort | uniq -c | sort -rn`.) You should see `7 200` on the first line.

Task 7: Transform text with awk, sed, tr, printf, bc, and xargs

1. Use `awk` to work with columns and do arithmetic.

```
awk '{print $1, $9}' access.log | head -n 3
awk '$9 >= 500 {print $4, $7}' access.log
awk '{sum += $10} END {print "Total bytes:", sum}' access.log
awk -F, 'NR > 1 {ram += $3} END {print "Total RAM:", ram, "GB"}' servers.csv
```

```
192.168.100.20 200
192.168.100.20 200
10.0.0.15 200
[10/Oct/2026:08:07:45 /reports
[10/Oct/2026:08:15:42 /reports
Total bytes: 50508
Total RAM: 58 GB
```

What just happened: `awk` is a small programming language for column-based text. By default it splits each line on runs of spaces or tabs into fields `$1`, `$2`, … (`$0` is the whole line, `NF` is the number of fields, `NR` is the current line number). A program has the form `pattern {action}`: the action runs for every line that matches the pattern. `$9 >= 500` selects server errors. `sum += $10` adds up the bytes column, and the `END` block runs once after the last line. `-F,` sets the field separator to a comma, and `NR > 1` skips the header line. Ubuntu's `awk` is `mawk`; Rocky's is `gawk`. Both run these examples.

2. Use `sed` (stream editor) to edit text on the fly.

```
sed 's/ubuntu/Ubuntu/' servers.csv | head -n 3
sed -n '2,4p' servers.csv
sed '1d' servers.csv | head -n 2
sed '/rocky/d' servers.csv
```

```
hostname,os,ram_gb,role,site
web01,Ubuntu,4,web,london
web02,Ubuntu,4,web,london
web01,ubuntu,4,web,london
web02,ubuntu,4,web,london
db01,rocky,16,database,paris
web01,ubuntu,4,web,london
web02,ubuntu,4,web,london
hostname,os,ram_gb,role,site
web01,ubuntu,4,web,london
...
```

What just happened: `sed 's/old/new/'` substitutes the first match of `old` on each line; add `g` (`s/old/new/g`) to replace every match on the line. `sed -n '2,4p'` turns off automatic printing (`-n`) and prints (`p`) only lines 2 to 4. `1d` deletes line 1. `/rocky/d` deletes every line containing `rocky`. None of these changed the file: `sed` writes to standard output unless you ask it to edit in place.

3. Edit a file in place with `sed -i`, keeping a backup.

```
cp servers.csv servers-copy.csv
sed -i.bak 's/debian/ubuntu/g' servers-copy.csv
grep app01 servers-copy.csv servers-copy.csv.bak
```

```
servers-copy.csv:app01,ubuntu,8,app,berlin
servers-copy.csv.bak:app01,debian,8,app,berlin
```

What just happened: `-i` edits the file in place. `-i.bak` first saves the original with a `.bak` suffix, a good habit when changing config files with `sed` (for example `sudo sed -i.bak 's/^#Port 22/Port 2222/' /etc/ssh/sshd_config`). When `grep` searches more than one file, it prefixes each match with the file name.

4. Translate or delete characters with `tr`.

```
head -n 2 servers.csv | tr ',' '\t'
echo "Hello World" | tr 'a-z' 'A-Z'
echo "too    many     spaces" | tr -s ' '
echo "phone: 555-0142" | tr -d -c '0-9\n'
```

```
hostname    os  ram_gb  role    site
web01   ubuntu  4   web london
HELLO WORLD
too many spaces
5550142
```

What just happened: `tr set1 set2` replaces each character in set 1 with the matching character in set 2: commas become tabs (`\t`), lowercase becomes uppercase. `-s` squeezes runs of a repeated character into one. `-d` deletes characters, and `-c` complements the set, so `-d -c '0-9\n'` deletes everything that is *not* a digit or newline. `tr` works on characters, not words: `tr 'cat' 'dog'` changes every `c` to `d`, every `a` to `o`, and every `t` to `g`.

5. Format output with `printf` and compare it with `echo`.

```
echo "Server: web01"
echo -n "no newline here"; echo " <- same line"
echo -e "col1\tcol2"
printf "%-8s %-10s %3d GB\n" web01 web 4
awk -F, 'NR > 1 {printf "%-8s %-10s %3d GB\n", $1, $4, $3}' servers.csv
```

```
Server: web01
no newline here <- same line
col1    col2
web01    web          4 GB
web01    web          4 GB
web02    web          4 GB
db01     database    16 GB
db02     database    16 GB
mon01    monitoring   2 GB
app01    app          8 GB
app02    app          8 GB
```

What just happened: `echo` prints its arguments and a newline. `-n` drops the newline; `-e` makes it understand escapes like `\t` (tab) and `\n`. `printf` takes a *format string* and never adds a newline unless you write `\n`. In the format, `%s` is a string, `%d` an integer, `%-8s` a string left-aligned in 8 characters, and `%3d` a number right-aligned in 3 characters. That is how you produce neat columns. `awk` has its own `printf` with the same rules.

6. Calculate with `bc`.

```
echo "10 / 3" | bc
echo "scale=2; 10 / 3" | bc
bc <<< "scale=2; 50508 / 15"
bc -l <<< "sqrt(2)"
```

```
3
3.33
3367.20
1.41421356237309504880
```

What just happened: `bc` is an arbitrary-precision calculator that reads expressions from standard input. By default it does integer division, so `10 / 3` is `3`. `scale=2` sets two decimal places. The third line computes the average bytes per request from the total you got with `awk`. `-l` loads the math library (which also sets `scale=20`). Run `bc` alone for an interactive calculator, and type `quit` to leave. Bash itself only does integer math: `echo $((10 / 3))` prints `3`.

7. Turn lines into arguments with `xargs`.

```
tail -n +2 servers.csv | cut -d, -f1 | xargs echo
tail -n +2 servers.csv | cut -d, -f1 | head -n 3 | xargs -I {} echo "Checking {}..."
touch temp{1..5}.tmp
ls *.tmp
find . -name "*.tmp" | xargs rm -v
```

```
web01 web02 db01 db02 mon01 app01 app02
Checking web01...
Checking web02...
Checking db01...
temp1.tmp  temp2.tmp  temp3.tmp  temp4.tmp  temp5.tmp
removed './temp1.tmp'
...
```

What just happened: Many commands (`rm` , `echo` , `ping`) take *arguments*, not standard input. `xargs` bridges the gap: it reads lines from stdin and adds them as arguments to a command. The first example turned seven lines into one `echo` with seven arguments. `-I {}` runs the command once per input line, replacing `{}` with the line. `touch temp{1..5}.tmp` uses Bash *brace expansion* to create five files. `find ... | xargs rm -v` deletes every file `find` reported; `-v` makes `rm` say what it removed. If file names may contain spaces, use `find . -name "*.tmp" -print0 | xargs -0 rm` . `xargs -n 1` passes one argument per command run.

Checkpoint: Produce a report of every request that got a 404, printed as `IP -> path` using `awk` . (Answer: `awk '$9 == 404 {print $1, "->", $7}' access.log` prints three lines for `203.0.113.7` .)

Task 8: Reuse commands with history, !, and !!

1. Show your command history.

```
history | tail -n 5
history 5
echo "$HISTSIZE $HISTFILESIZE $HISTCONTROL"
```

```
  201  ...
  205  history 5
1000 2000 ignoreboth
```

What just happened: `history` lists previous commands with numbers. `history 5` shows the last five. Bash keeps the list in memory and writes it to `~/.bash_history` when the shell exits. Ubuntu's `~/.bashrc` sets `HISTSIZE=1000` (commands kept in memory), `HISTFILESIZE=2000` (lines kept in the file), and `HISTCONTROL=ignoreboth` , which skips duplicate commands and any command that starts with a space. That last trick is handy: start a command with a space and it is not saved in history. Your numbers will differ.

2. Re-run the last command with `!!` , and fix a forgotten `sudo` .

```
uname -r
!!
cat /etc/shadow | head -n 2
sudo !!
```

```
6.8.0-45-generic
uname -r
6.8.0-45-generic
cat: /etc/shadow: Permission denied
sudo cat /etc/shadow | head -n 2
root:*:19800:0:99999:7:::
daemon:*:19800:0:99999:7:::
```

What just happened: `!!` expands to the previous command. Bash prints the expanded command line first so you can see what actually ran. `sudo !!` is the classic "I forgot sudo" fix: it re-runs the last command with `sudo` in front. `/etc/shadow` holds password hashes and is readable only by root.

3. Re-run by number, by prefix, and reuse the last argument.

```
history | grep uname
```

Pick a line number from your output (for example `198`) and run it:

```
!198
```

Then try these:

```
!grep
ls -l /etc/hostname
cat !$
```

```
...
cat /etc/hostname
ubuntu-lab
```

What just happened: `!n` runs history entry number `n`. `!grep` runs the most recent command that *started with* `grep`. `!$` is the last argument of the previous command, so `cat !$` became `cat /etc/hostname`. Be careful with `!` shortcuts as root, because they run immediately; `!grep:p` prints the command without running it. You can also press Ctrl+R and start typing to search history interactively (press Ctrl+R again for older matches, Enter to run, or Ctrl+C to cancel), and the Up arrow steps back through history.

TIP

Because `!` triggers history expansion, `echo "Hello!"` can behave strangely in an interactive shell. Use single quotes (`echo 'Hello!'`) when you want a literal `!`.

Checkpoint: `history | tail -n 3` shows the commands you just ran, and you can explain what `!!`, `!n`, `!grep`, and `!$` each expand to.

Task 9: Edit files with vim and nano

1. Create a practice file for `vim`.

```
cd ~/labs/1.5
cat > vim-practice.txt <<'EOF'
The colour of the server rack is grey.
This line has a typo: teh.
Delete this whole line.
The colour of the cables is blue.
EOF
```

2. Open it in `vim`. On Ubuntu, `vi` also starts `vim`.

```
vim vim-practice.txt
```

> **What just happened:** `vim` opens in **Normal mode**, where keys are *commands*, not text. This is the part that confuses beginners. There are three modes you need:
>
> - **Normal mode** (the default): move around, delete, copy, paste. Press Esc to get back here from anywhere.
> - **Insert mode**: typing inserts text. Enter it with `i` (insert before the cursor), `a` (append after it), `o` (open a new line below), or `A` (append at end of line). The bottom line shows `-- INSERT --`.
> - **Command-line mode**: press `:` in Normal mode to type commands such as `:w` (write/save) and `:q` (quit).

3. Follow these steps exactly inside `vim`. Press Esc whenever you are unsure which mode you are in.
    1. Type `:set number` and press Enter. Line numbers appear.
    2. Move with the arrow keys or `h` (left), `j` (down), `k` (up), `l` (right). Press `G` to jump to the last line, `gg` for the first line, `0` for the start of the line, `$` for the end.
    3. Fix the typo: type `/teh` and press Enter to search. The cursor jumps to `teh`. Type `cw` (change word), type `the`, then press Esc.
    4. Delete line 3: type `3G` to go to line 3, then `dd` to delete the line.
    5. Undo and redo: press `u` (the line comes back), then Ctrl+R (it is deleted again).
    6. Add a line: press `G`, then `o`, type `Added in vim.`, and press Esc.
    7. Copy and paste: press `gg`, then `yy` (yank, which means copy, the line), then `p` (paste below). Press `dd` to remove the duplicate. `x` deletes a single character.
    8. Search and replace in the whole file: type `:%s/colour/color/g` and press Enter. `%` means every line, `s` substitute, `g` every match on the line. The status line reports `2 substitutions on 2 lines`.
    9. Save and quit: type `:wq` and press Enter.

What just happened: You used the core `vim` survival kit. The quit commands are worth memorizing: `:w` saves, `:q` quits (only if there are no unsaved changes), `:wq` or `:x` or `ZZ` saves and quits, and `:q!` quits **without** saving, throwing changes away. If you ever get stuck, press Esc twice and type `:q!`. Searching uses `/pattern` (forward), `?pattern` (backward), `n` (next), and `N` (previous). For a full interactive course, run `vimtutor` later; it takes about 30 minutes.

4. Check your edits.

```
cat vim-practice.txt
```

```
The color of the server rack is grey.
This line has a typo: the.
The color of the cables is blue.
Added in vim.
```

5. Now edit a file with `nano`.

```
nano nano-practice.txt
```

`nano` opens directly in a typing mode. The bottom two lines list shortcuts: `^` means Ctrl and `M-` means Alt. Do the following:

1. Type three lines, pressing Enter after each one: `alpha`, `beta`, `gamma`. The cursor is now on an empty fourth line.
2. Press Ctrl+O (write Out), confirm the file name with Enter. This saves.
3. Press Ctrl+W (Where is), type `beta`, press Enter. The cursor jumps to it.
4. Press Ctrl+K to cut the whole `beta` line. Press Alt+/ to jump to the last (empty) line, then press Ctrl+U to paste the line there.
5. Press Ctrl+\ (replace), type `gamma`, press Enter, type `delta`, press Enter, then press `A` to replace all.
6. Try undo and redo: press Alt+U (the replacement is undone), then Alt+E (it is redone).
7. Press Ctrl+X to exit. If there are unsaved changes, `nano` asks “Save modified buffer?”; press `Y` and then Enter.

```
cat nano-practice.txt
```

```
alpha
delta
beta
```

What just happened: `nano` is a *modeless* editor: you type text directly and use Ctrl and Alt combinations for commands. Ctrl+O saves, Ctrl+X exits, Ctrl+W searches, Ctrl+K and Ctrl+U cut and paste lines, and Ctrl+G shows help. Start it with `nano -l file` to show line numbers, or use Ctrl+_ (or Alt+G) to jump to a line number. If your result differs, you probably pasted on a different line; that is fine, as long as you saw cut, paste, and replace work.

TIP

Ubuntu asks which editor to use for tools like `crontab -e` and `visudo` the first time you run them, or uses the `EDITOR` / `VISUAL` variables. `export EDITOR=vim` in `~/.profile` sets your choice; `sudo update-alternatives --config editor` changes the system default. On Rocky minimal, `vi` is `vim-minimal`; install `vim-enhanced` for the full `vim`, and `nano` with `sudo dnf install -y nano`.

Checkpoint: `grep -c color vim-practice.txt` prints `2` and `grep -c colour vim-practice.txt` prints `0`. You can name the `vim` keys to save and quit (`:wq`) and to quit without saving (`:q!`), and the `nano` keys to save (Ctrl+O) and exit (Ctrl+X).

Challenge

CHALLENGE

1. In one pipeline, print the three most requested paths in `access.log` with their counts, formatted with `printf` as `path ... count` in aligned columns.
2. Write a here doc that creates `~/labs/1.5/report.sh`, a script that prints the date, `uname -r`, the number of 404 and 500 responses in `access.log`, and the average bytes per request (use `awk` for the total and `bc` for the division). Run it and save its output with `tee report.txt`, sending any errors to `/dev/null`.
3. Add a temporary alias `top404` that prints the IP address with the most 404 responses. Explain why the alias would disappear if you defined it in `~/.profile` but then opened a non-login shell with `bash` on a system where `.profile` did not source `.bashrc`.

Clean up

1. Restore your original startup files and remove the backups.

```
mv ~/.bashrc.lab15.bak ~/.bashrc
mv ~/.profile.lab15.bak ~/.profile
rm -f ~/.bash_profile
```

   (If you had your own `~/.bash_profile` before the lab and backed it up, restore it instead of deleting it.)

2. Remove the lab files, the tools directory, and the root-owned test file.

```
rm -rf ~/labtools ~/labs/1.5
sudo rm -f /opt/lab15.txt
```

3. If you installed `x11-apps` for the optional X forwarding test, remove it.

```
sudo apt remove -y x11-apps
```

4. Log out and log back in so your shell starts with the original prompt, `PATH`, and aliases. Verify:

```
echo "$PATH" | grep -c labtools
type lab
```

```
0
bash: type: lab: not found
```

Exam corner

- Absolute paths start with `/` (or `~`, which expands to `$HOME`). Relative paths start from the current directory; `.` is the current directory, `..` the parent, and `cd -` returns to the previous directory (`$OLDPWD`).
- `HOME`, `USER`, `SHELL`, `PATH`, `PS1`, and `DISPLAY` are the key variables. `export` makes a variable visible to child processes; `env` / `printenv` list environment variables.
- `PATH` is searched left to right; the current directory is not in it, so run local scripts as `./script`.
- Login shells read `/etc/profile` then the first of `~/.bash_profile`, `~/.bash_login`, `~/.profile`. Non-login interactive shells read `~/.bashrc`. `source file` or `. file` applies changes to the current shell.
- Streams: stdin = 0, stdout = 1, stderr = 2. `>` overwrites, `>>` appends, `2>` redirects errors, `2>&1` joins stderr to stdout (order matters), `&>` sends both, `<` feeds a file to stdin, `<<` is a here doc, `<<<` is a here string, `/dev/null` discards.
- `|` passes only stdout. `tee` writes to a file and stdout at the same time; `| sudo tee file` writes a root-owned file.
- `uniq` only removes *adjacent* duplicates, so `sort | uniq -c | sort -rn` is the classic "count and rank" pipeline.
- `cut -d, -f2` for delimited fields; `awk '{print $2}'` for whitespace-separated columns; `sed 's/a/b/g'` to substitute; `sed -i` edits in place; `tr` changes or deletes single characters.
- `xargs` turns stdin lines into command arguments; `bc` does decimal math (`scale=`); `printf` formats output without an automatic newline.
- `!!` repeats the last command (`sudo !!`), `!n` runs history entry `n`, `!string` runs the latest command starting with `string`, `!$` is the last argument.
- `vim`: `i` insert, Esc normal mode, `:w` save, `:q!` quit without saving, `:wq` save and quit, `dd` delete line, `yy` / `p` copy/paste, `u` undo, `/text` search, `:%s/old/new/g` replace. `nano`: Ctrl+O save, Ctrl+X exit, Ctrl+W search.

Review questions

1. You run `./backup.sh > backup.log`, but error messages still appear on the screen. Which command captures both output and errors in `backup.log`?
   - A. `./backup.sh 2>&1 > backup.log`
   - B. `./backup.sh > backup.log 2>&1`
   - C. `./backup.sh | backup.log`
   - D. `./backup.sh >> backup.log 2>/dev/null`

2. A user adds `export PATH="$PATH:/opt/tools/bin"` to `~/.bash_profile` on an Ubuntu system. After the next SSH login, the PATH change works, but their aliases from `~/.bashrc` no longer load in that login. Why?

3. Which pipeline lists the unique values in column 2 of `data.csv` with how many times each appears, most frequent first?
   - A. `cut -d, -f2 data.csv | uniq -c | sort -rn`
   - B. `cut -d, -f2 data.csv | sort | uniq -c | sort -rn`
   - C. `sort data.csv | cut -f2 | wc -l`
   - D. `awk '{print $2}' data.csv | uniq`

4. You typed `systemctl restart nginx` and got “Access denied”. Which is the quickest way to run the same command with root privileges?

5. Which command sends the string `scale=3; 22/7` to `bc` as standard input without using `echo` or a pipe?
   - A. `bc < "scale=3; 22/7"`
   - B. `bc << "scale=3; 22/7"`
   - C. `bc <<< "scale=3; 22/7"`
   - D. `bc > "scale=3; 22/7"`

6. In `vim`, you made changes you do not want to keep. Which command exits without saving?
   - A. `:wq`
   - B. `ZZ`
   - C. `:q!`
   - D. `:x`

PART 2 · DOMAIN 2.0

Part 2: Services and User Management

20% of the exam · 1 labs

2.1 Given a scenario, manage files and directories on a Linux system.

Lab 2.1 Manage Files and Directories

Objective: 2.1 Given a scenario, manage files and directories on a Linux system.
Domain: 2.0 Services and User Management (20% of the exam)
Time: about 90 minutes
VM: ubuntu-lab
Snapshot first: Not required. Everything happens inside `~/labs/2.1` and `/dev/shm`, and the clean-up removes it.

In this lab you build a small project directory tree and then work through every file utility on the exam the way an administrator really uses them: finding files, comparing config versions, checking timestamps, and tracking down which process is holding a file open. You will also see exactly how hard links and symbolic links differ, and take a guided tour of the device files in `/dev`. These commands are the everyday tools of any Linux job, and the exam expects you to pick the right one for a scenario.

What you'll practice

- Utilities: `cd`, `cp`, `diff`, `file`, `find`, `ln`, `locate`, `ls`, `lsof`, `mkdir`, `mv`, `pwd`, `rm`, `rmdir`, `sdiff`, `stat`, `touch`
- Links: symbolic link, hard link
- Device types in `/dev`: block devices, character devices, special character devices

How it works

Everything is a file. Linux presents almost everything as a file in one big tree that starts at `/` (the *root* directory). Regular files hold data. Directories are files that hold a list of names. Links, devices, sockets, and pipes are also files with special types. The first character of `ls -l` output tells you the type:

Character	Type
`-`	regular file
`d`	directory
`l`	symbolic link
`b`	block device
`c`	character device
`p`	named pipe (FIFO)
`s`	socket

Paths. An *absolute path* starts at `/`, like `/etc/hosts`. A *relative path* starts from your *current working directory* (the directory your shell is "in"), like `project/docs`. Two special names exist in every directory: `.` means "this directory" and `..` means "the parent directory". The `~` character is shell shorthand for your home directory (`/home/student`).

Inodes: the real identity of a file. A file on disk has two parts. The *inode* (index node) is a small record that stores everything about the file except its name: owner, permissions, size, timestamps, and where the data blocks live. Each inode has a number that is unique inside its filesystem. The *directory entry* is just a name that points to an inode number. Think of a library: the inode is the catalog card with all the facts about a book, and a directory entry is a label on a shelf that says "card number 1311234".

```
Directory entries (names)         Inode 1311234                Data blocks
------------------------          ---------------------        -----------
notes.txt       ---------+------> owner, perms, size,  ------>  "hello..."
notes-hard.txt  ---------+        timestamps, link count = 2
  (hard link: same inode)

notes-soft.txt  -> inode 1311240 (type l) contains the TEXT "notes.txt"
  (symbolic link: its own inode that stores a path)
```

This explains the two kinds of links:

- A **hard link** is a second directory entry that points to the *same inode*. Both names are equally "real". The inode keeps a *link count*. The data is only freed when the count drops to zero and no process has the file open. Hard links cannot cross filesystems (inode numbers are only unique inside one filesystem) and you cannot hard-link a directory.
- A **symbolic link** (symlink, soft link) is a tiny separate file whose content is a path to another file. It has its own inode. It can point anywhere, even to another filesystem or to a directory. If the target is deleted, the symlink becomes *dangling* (broken) because the path it stores no longer leads anywhere.

Three timestamps. Every inode records:

- **atime** (access): last time the content was read. Ubuntu mounts with `relatime`, so atime is only updated occasionally to save disk writes.
- **mtime** (modify): last time the *content* changed.
- **ctime** (change): last time the *inode* changed (content, permissions, owner, link count, or name). You cannot set ctime by hand.

Many filesystems (ext4, XFS) also record a **birth** (creation) time, which `stat` shows when available.

Finding files: `find` versus `locate`. `find` walks the directory tree live, every time, and can test almost anything (name, size, age, owner, permissions) and then run a command on each match. It is always accurate but can be slow on a big tree. `locate` searches a prebuilt database of file names, so it is nearly instant, but it only knows what existed the last time the database was built with `updatedb`. On Ubuntu 24.04 `locate` is provided by the `plocate` package.

Open files and `lsof`. When a program opens a file, the kernel gives it a *file descriptor* (a small number). `lsof` ("list open files") shows which processes have which files open. This answers classic admin questions: "Why can't I unmount this disk?" and "I deleted a huge log file, so why is the disk still full?" (Answer: a process still has the deleted

file open, so the inode and data are not freed yet.)

Device files in `/dev`. Hardware and kernel features appear as special files in `/dev`. They contain no data on disk; reading or writing them talks to a kernel driver. Each device file carries two numbers instead of a size: the **major number** identifies the driver, and the **minor number** identifies which device that driver handles.

- **Block devices** (`b`) move data in fixed-size blocks and allow random access, so you can jump to any position. Disks and partitions are block devices: `/dev/sda`, `/dev/sda1`, `/dev/nvme0n1`.
- **Character devices** (`c`) move data as a stream of bytes, one after another. Terminals and serial ports are character devices: `/dev/tty`, `/dev/ttyS0`.
- **Special character devices** are character devices that are not real hardware at all. The kernel creates them to provide useful behavior: `/dev/null` (throws away everything written to it, returns nothing when read), `/dev/zero` (returns endless zero bytes), `/dev/random` and `/dev/urandom` (return random bytes), and `/dev/full` (always reports "disk full" when written).

Tasks

Task 1: Navigate and build a project tree

You are setting up a working area for a small web application. You will use `pwd`, `cd`, `mkdir`, `ls`, and `touch`.

1. Create the lab directory and move into it.

```
mkdir -p ~/labs/2.1 && cd ~/labs/2.1
pwd
```

```
/home/student/labs/2.1
```

> **What just happened:** `mkdir` makes a directory. The `-p` (parents) option creates any missing parent directories along the way (`~/labs` and then `~/labs/2.1`) and does not complain if they already exist. `&&` runs the second command only if the first succeeded. `cd` (change directory) moves your shell into the new directory, and `pwd` (print working directory) shows the absolute path of where you are now.

2. Practice moving around. Run each line and watch the output of `pwd`.

```
cd /etc
pwd
cd ..
pwd
cd -
cd
pwd
cd ~/labs/2.1
```

> **What just happened:** `cd /etc` uses an absolute path. `cd ..` moves up one level to `/`. `cd -` jumps back to the previous directory (`/etc`) and prints its name. `cd` with no argument takes you home (`/home/student`). The last line returns you to the lab directory using `~` for home.

3. Build the whole project tree in one command using *brace expansion*.

```
mkdir -p webapp/{src,docs,config,logs,backup}
ls
ls webapp
```

```
webapp
backup  config  docs  logs  src
```

> **What just happened:** The shell expands `webapp/{src,docs,config,logs,backup}` into five separate arguments (`webapp/src`, `webapp/docs`, and so on) before `mkdir` runs. `mkdir -p` then creates `webapp` and the five subdirectories. Brace expansion is a shell feature, so it works with any command.

4. Create some empty files with `touch`, then add content to a few.

```
touch webapp/src/{app.py,utils.py,models.py}
touch webapp/docs/README.md webapp/logs/app.log
echo "print('hello')" > webapp/src/app.py
echo "Project notes" > webapp/docs/notes.txt
```

> **What just happened:** `touch` creates a file if it does not exist (and updates its timestamps if it does). `echo "text" > file` writes text into a file, replacing anything already there. You will learn more about `touch` and timestamps in Task 4.

5. List the tree in several useful ways.

```
ls -l webapp
ls -la webapp/src
ls -lh webapp/docs
ls -R webapp
ls -ld webapp
ls -lt webapp/src
```

```
total 20
drwxrwxr-x 2 student student 4096 Oct 10 10:02 backup
drwxrwxr-x 2 student student 4096 Oct 10 10:02 config
drwxrwxr-x 2 student student 4096 Oct 10 10:03 docs
drwxrwxr-x 2 student student 4096 Oct 10 10:03 logs
drwxrwxr-x 2 student student 4096 Oct 10 10:03 src
```

What just happened: `ls -l` gives the *long listing*: type and permissions (`drwxrwxr-x`), link count, owner, group, size, modification time, and name. `-a` shows *all* files, including hidden ones whose names start with a dot (you will see `.` and `..`). `-h` prints sizes in human-readable units (K, M, G). `-R` lists recursively into every subdirectory. `-d` lists the directory itself instead of its contents. `-t` sorts by modification time, newest first.

Checkpoint: `ls -R ~/labs/2.1/webapp` shows five subdirectories, three `.py` files in `src` , `README.md` and `notes.txt` in `docs` , and `app.log` in `logs` .

Task 2: Copy, move, rename, and remove

Now you reorganize the project with `cp` , `mv` , `rm` , and `rmdir` .

1. Create a config file and make a backup copy of it.

```
cd ~/labs/2.1/webapp
printf 'port=8080\nworkers=4\ndebug=false\nlog_level=info\n' > config/app.conf
cp config/app.conf backup/app.conf.bak
cp -p config/app.conf backup/app.conf.preserved
ls -l config backup
```

What just happened: `printf` writes text and turns each `\n` into a new line, so `app.conf` gets four lines. `cp SOURCE DEST` copies a file. A plain copy gets a new modification time (now). `cp -p` *preserves* the mode, ownership, and timestamps of the original, which matters for backups because you want to know when the original last changed.

2. Copy a whole directory. `cp` refuses to copy a directory unless you ask for recursion.

```
cp src backup/
cp -r src backup/src-copy
cp -a docs backup/docs-archive
ls backup
```

```
cp: -r not specified; omitting directory 'src'
app.conf.bak  app.conf.preserved  docs-archive  src-copy
```

What just happened: The first `cp` failed with a clear message. `-r` (recursive) copies a directory and everything inside it. `-a` (archive) is the same as `-dR --preserve=all` : recursive, keeps symlinks as symlinks, and preserves permissions, ownership, and timestamps. Use `-a` when you want an exact copy.

3. Use `mv` to rename a file and to move files into another directory.

```
mv docs/notes.txt docs/project-notes.txt
mv src/models.py src/utils.py backup/src-copy/
ls docs src
```

> **What just happened:** `mv` both renames and moves. With two names in the same directory it renames. With a directory as the last argument it moves all the earlier arguments into that directory. Inside one filesystem `mv` is instant because it only rewrites directory entries; the inode and data do not move. The original `utils.py` already existed in `backup/src-copy` from the `cp -r`, so `mv` silently replaced it. Add `-i` (interactive) to be asked before overwriting.

4. Try the safe, interactive versions. Answer `n` to each prompt.

```
cp -i config/app.conf backup/app.conf.bak
mv -i backup/app.conf.preserved backup/app.conf.bak
```

```
cp: overwrite 'backup/app.conf.bak'? n
mv: overwrite 'backup/app.conf.bak'? n
```

5. Remove files and directories. Start with an empty directory, then a non-empty one.

```
mkdir -p tmp/a/b/c
rmdir tmp/a/b/c
rmdir -p tmp/a/b
rmdir backup
```

```
rmdir: failed to remove 'backup': Directory not empty
```

> **What just happened:** `rmdir` only removes *empty* directories, which makes it a safe choice: it can never delete files by accident. `rmdir -p tmp/a/b` removes `b`, then `a`, then `tmp`, as long as each becomes empty. It refused to remove `backup` because it still has contents.

6. Use `rm` for files and non-empty directories.

```
rm -i backup/app.conf.preserved
rm -r backup/docs-archive
ls backup
```

> **What just happened:** `rm` removes files (it deletes directory entries; the data is freed when the link count reaches zero). `-i` asks for confirmation (answer `y`). `-r` removes a directory and everything under it. You will often see `rm -rf`, where `-f` (force) never prompts and ignores missing files. There is no recycle bin on the command line, so double-check paths before using `-rf`.

CAUTION

`rm -rf` with a typo can destroy a system. A classic mistake is a stray space: `rm -rf / home/olduser` deletes from `/`. Modern GNU `rm` refuses to operate on `/` itself unless you add `--no-preserve-root`, but it will happily delete any other directory you name. Run `ls` on the exact path first.

Checkpoint: `ls ~/labs/2.1/webapp/backup` shows `app.conf.bak` and `src-copy`. `ls ~/labs/2.1/webapp/src` shows only `app.py`, and `~/labs/2.1/webapp/tmp` no longer exists.

Task 3: Identify files with `file` and inspect them with `stat`

File names in Linux do not have to have extensions, and an extension can lie. The `file` command looks inside a file to see what it really is.

1. Run `file` on several kinds of objects.

```
cd ~/labs/2.1
cp /usr/bin/ls ./mystery
gzip -k webapp/config/app.conf
file mystery webapp/config/app.conf webapp/config/app.conf.gz
file /bin /etc /dev/sda /dev/null webapp/src/app.py
```

```
mystery:                  ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV),
                          dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, ...
webapp/config/app.conf:   ASCII text
webapp/config/app.conf.gz: gzip compressed data, was "app.conf", ...
/bin:                     symbolic link to usr/bin
/etc:                     directory
/dev/sda:                 block special (8/0)
/dev/null:                character special (1/3)
webapp/src/app.py:        ASCII text
```

What just happened: `file` reads the first bytes of each file and compares them to a database of known patterns called *magic numbers*. The file `mystery` has no extension, but `file` recognizes it as an ELF executable (the Linux program format). `gzip -k` compressed a copy while keeping (`-k`) the original. On Ubuntu 24.04, `/bin` is a symbolic link to `usr/bin`. For devices, `file` shows the type and the major/minor numbers. If your disk is `vda` or `nvme0n1`, `/dev/sda` will say "cannot open"; that is fine. `app.py` is reported as plain text because it has no `#!` line; with `#!/usr/bin/env python3` as its first line, `file` would call it a Python script.

2. Look at the full inode details of a file with `stat`.

```
stat webapp/config/app.conf
```

```
  File: webapp/config/app.conf
  Size: 46          Blocks: 8          IO Block: 4096   regular file
Device: 252,0   Inode: 1311290     Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/ student)   Gid: ( 1000/ student)
Access: 2026-10-10 10:05:12.418263510 +0000
Modify: 2026-10-10 10:05:12.418263510 +0000
Change: 2026-10-10 10:05:12.418263510 +0000
 Birth: 2026-10-10 10:05:12.418263510 +0000
```

> **What just happened:** `stat` prints the inode contents. **Size** is in bytes; **Blocks** is the number of 512-byte units allocated. **Device** is the filesystem the file lives on (major,minor). **Inode** is the inode number and **Links** is the hard link count. **Access** shows permissions in octal (`0664`) and symbolic form, plus the owner UID and group GID. Then the three timestamps (atime, mtime, ctime) and the birth time. Your inode number and device will differ.

3. `stat` can also print just the fields you want, which is handy in scripts.

```
stat -c '%n  inode=%i  links=%h  size=%s  mode=%a  owner=%U' webapp/config/app.conf
stat -f /
```

> **What just happened:** `-c` (format) prints a custom line: `%n` name, `%i` inode, `%h` hard links, `%s` size, `%a` octal permissions, `%U` owner name. `stat -f` shows information about the *filesystem* instead of a file: type (for example `ext2/ext3` for ext4), block size, and free blocks and inodes.

Checkpoint: `file ~/labs/2.1/mystery` reports an ELF executable, and `stat -c %h ~/labs/2.1/webapp/config/app.conf` prints `1`.

Task 4: Control timestamps with `touch`

Backup and clean-up scripts often decide what to do based on file age. To test them you need files with old timestamps, and `touch` can create those.

1. Create a set of log files and give them different ages.

```
cd ~/labs/2.1/webapp/logs
touch -d "2024-01-15 09:30" old-jan.log
touch -d "10 days ago" ten-days.log
touch -t 202510010800 oct-first.log
touch today.log
ls -l --time-style=long-iso
```

```
total 0
-rw-rw-r-- 1 student student 0 2026-10-10 10:03 app.log
-rw-rw-r-- 1 student student 0 2024-01-15 09:30 old-jan.log
-rw-rw-r-- 1 student student 0 2025-10-01 08:00 oct-first.log
-rw-rw-r-- 1 student student 0 2026-09-30 10:10 ten-days.log
-rw-rw-r-- 1 student student 0 2026-10-10 10:10 today.log
```

> **What just happened:** `touch -d STRING` sets the atime and mtime to a date described in almost plain English (`"10 days ago"`, `"yesterday"`, `"2024-01-15 09:30"`). `touch -t` uses the compact format `[[CC]YY]MMDDhhmm[.ss]`, so `202510010800` means 2025-10-01 08:00. `--time-style=long-iso` makes `ls` show full dates. Your dates will be relative to today.

2. Change only one timestamp, and observe that ctime cannot be faked.

```
touch -a -d "2020-06-01" today.log
stat today.log | grep -E 'Access: [0-9]|Modify|Change'
```

```
Access: 2020-06-01 00:00:00.000000000 +0000
Modify: 2026-10-10 10:10:41.120093114 +0000
Change: 2026-10-10 10:11:02.551876020 +0000
```

What just happened: `touch -a` changes only the access time; `touch -m` changes only the modification time. Changing a timestamp is itself a change to the inode, so **ctime** jumped to *now*. There is no option to set ctime, which is why investigators trust it more than mtime.

Checkpoint: `ls -lt ~/labs/2.1/webapp/logs` lists `old-jan.log` last (oldest modification time).

Task 5: Find files with `find` and `locate`

1. Find by name. `find` takes a starting directory, then tests, then an optional action.

```
cd ~/labs/2.1
find . -name "*.log"
find webapp -iname "readme*"
find webapp -type d
find webapp -type f -name "*.py"
```

What just happened: `find .` searches from the current directory down. `-name "*.log"` matches names with a shell-style pattern; always quote the pattern so your shell does not expand `*` first. `-iname` is case-insensitive. `-type d` matches directories and `-type f` regular files (`-type l` matches symlinks). Tests placed one after another are joined with AND.

2. Find by time. Use the log files you aged in Task 4.

```
find webapp/logs -mtime +7
find webapp/logs -mtime -1
find webapp -mmin -30 -type f
```

What just happened: `-mtime` counts in 24-hour days. `+7` means "modified more than 7 days ago", `-1` means "modified less than 1 day ago". `-mmin` works the same way in minutes. `-atime` and `-ctime` test the other two timestamps. This is how log clean-up jobs pick old files.

3. Find by size. Make a large file first.

```
dd if=/dev/zero of=webapp/backup/big.img bs=1M count=50 status=none
find . -size +10M
find . -type f -size 0
find . -type f -empty
```

What just happened: `dd` copied 50 blocks of 1 MiB from `/dev/zero`, making a 50 MiB file (more on `/dev/zero` in Task 8). `-size +10M` finds files bigger than 10 MiB; suffixes are `k`, `M`, and `G`. `-size 0` and `-empty` both find empty files (`-empty` also matches empty directories, but `-type f` limits it to files here).

4. Find by permissions and owner. Search system directories as well; send errors to `/dev/null`.

```
chmod 777 webapp/config/app.conf
find webapp -perm 777
find webapp -perm -o=w
find /usr/bin -perm -4000 2>/dev/null
find /etc -user student 2>/dev/null
find /var/log -group adm -name "*.log" 2>/dev/null | head
```

What just happened: `-perm 777` matches *exactly* mode 777. `-perm -o=w` (with a leading dash) matches files that have *at least* the "others can write" bit, which is the usual way to hunt for world-writable files. `-perm -4000` finds *setuid* programs (like `passwd`), a common security audit. `-user` and `-group` match the owner. `2>/dev/null` hides "Permission denied" errors from directories you cannot read. The `/etc -user student` search probably prints nothing, which is the expected result.

5. Act on the results with `-exec`, `-delete`, and `-ok`.

```
find webapp -name "*.py" -exec ls -l {} \;
find webapp -name "*.py" -exec wc -l {} +
find webapp/logs -name "*.log" -mtime +7 -ok rm {} \;
find webapp -perm 777 -exec chmod 644 {} \;
find webapp/logs -mtime +7
```

What just happened: `-exec COMMAND {} \;` runs the command once per match, replacing `{}` with the file name; the `\;` (escaped semicolon) marks the end of the command. Ending with `+` instead passes many file names to a single command, which is faster. `-ok` is like `-exec` but asks `y/n` for each file (answer `y` to delete the old logs). The last line confirms that no logs older than 7 days remain. `-delete` also exists and removes every match without asking, so test the `find` without it first.

6. Install `plocate` and compare `locate` with `find`.

```
sudo apt update
sudo apt install -y plocate
locate app.conf
sudo updatedb
locate app.conf
locate -c "*.conf"
locate -i readme.md | head -5
```

```
/home/student/labs/2.1/webapp/backup/app.conf.bak
/home/student/labs/2.1/webapp/config/app.conf
/home/student/labs/2.1/webapp/config/app.conf.gz
```

What just happened: On Ubuntu 24.04 the `locate` command comes from the `plocate` package, which also provides `updatedb`. The first `locate app.conf` probably found nothing in your lab directory because the database was built before you created those files. `sudo updatedb` rebuilds the database (stored in `/var/lib/plocate/plocate.db`). After that, `locate` finds the files instantly. `-c` counts matches instead of listing them, and `-i` ignores case. A systemd timer (`plocate-updatedb.timer`) rebuilds the database daily. `locate` only shows you files you have permission to see.

7. Prove that `locate` can be out of date.

```
rm webapp/config/app.conf.gz
locate app.conf.gz
find ~ -name app.conf.gz
```

What just happened: `locate` still lists the deleted file because the database is stale, while `find` correctly finds nothing. Rule of thumb: use `locate` for a fast "where is that file?" search, and `find` when you need accurate, current results or tests beyond the name.

TIP

On Rocky Linux 9, `locate` comes from the `mlocate` package (`sudo dnf install -y mlocate`), and you run `sudo updatedb` the same way.

Checkpoint: `find ~/labs/2.1 -size +10M` prints `big.img`, `find ~/labs/2.1/webapp -perm 777` prints nothing, and `locate -c app.conf.bak` prints `1`.

Task 6: Compare files with `diff` and `sdiff`

A colleague edited the application config, and you need to see exactly what changed.

1. Create a second version of the config file.

```
cd ~/labs/2.1/webapp/config
printf 'port=8080\nworkers=4\ndebug=false\nlog_level=info\n' > app.conf
printf 'port=8443\nworkers=4\nlog_level=debug\ntls=on\n' > app.conf.new
```

2. Compare them with plain `diff`.

```
diff app.conf app.conf.new
```

```
1c1
< port=8080
---
> port=8443
3,4c3,4
< debug=false
< log_level=info
---
> log_level=debug
> tls=on
```

What just happened: `diff` prints instructions for turning the first file into the second. `1c1` means "line 1 of the first file must be **c**hanged into line 1 of the second". `3,4c3,4` means lines 3 to 4 change into lines 3 to 4. Other letters are `a` (add) and `d` (delete). Lines starting with `<` come from the first file, lines with `>` from the second. If the files are identical, `diff` prints nothing and exits with status 0.

3. Use the *unified* format, which is what patches and `git` use.

```
diff -u app.conf app.conf.new
diff -q app.conf app.conf.new
diff -r ../src ../backup/src-copy
```

```
--- app.conf    2026-10-10 10:20:01.000000000 +0000
+++ app.conf.new    2026-10-10 10:20:01.000000000 +0000
@@ -1,4 +1,4 @@
-port=8080
+port=8443
 workers=4
-debug=false
-log_level=info
+log_level=debug
+tls=on
```

What just happened: `-u` shows changes with a few lines of context: `-` lines are removed, `+` lines are added, and lines starting with a space are unchanged. The `@@ -1,4 +1,4 @@` header gives the line ranges. `-q` (quiet) only reports *whether* files differ. `-r` compares two directory trees recursively and reports files that exist in only one of them ("Only in ...").

4. View the two versions side by side with `sdiff`.

```
sdiff -w 70 app.conf app.conf.new
sdiff -s -w 70 app.conf app.conf.new
```

```
port=8080                       | port=8443
workers=4                         workers=4
debug=false                     | log_level=debug
log_level=info                  | tls=on
```

What just happened: `sdiff` prints the first file on the left and the second on the right. The middle column marks each line: `|` means the lines differ, `<` means the line is only in the left file, `>` only in the right file, and a blank means identical. `-w 70` sets the total width in characters. `-s` suppresses identical lines so you only see differences. (`sdiff -o merged.conf` can even merge interactively.)

Checkpoint: `diff -q app.conf app.conf.new` prints `Files app.conf and app.conf.new differ`, and `diff app.conf app.conf; echo $?` prints `0`.

Task 7: Hard links and symbolic links

1. Create a file and one link of each type, then compare inode numbers.

```
mkdir -p ~/labs/2.1/links && cd ~/labs/2.1/links
echo "original content" > notes.txt
ln notes.txt notes-hard.txt
ln -s notes.txt notes-soft.txt
ls -li
```

```
total 8
1311301 -rw-rw-r-- 2 student student 17 Oct 10 10:25 notes-hard.txt
1311302 lrwxrwxrwx 1 student student  9 Oct 10 10:25 notes-soft.txt -> notes.txt
1311301 -rw-rw-r-- 2 student student 17 Oct 10 10:25 notes.txt
```

What just happened: `ln TARGET LINKNAME` creates a hard link; `ln -s TARGET LINKNAME` creates a symbolic link. `ls -i` adds the inode number in the first column. `notes.txt` and `notes-hard.txt` share inode `1311301`, and the link count (third column) is `2` for both: they are two names for the same file. `notes-soft.txt` has its own inode, type `l`, permissions `rwxrwxrwx` (symlink permissions are never used; the target's permissions apply), and a size of 9, which is the length of the text `notes.txt` it stores.

2. Confirm with `stat`, and edit through each name.

```
stat -c '%n inode=%i links=%h type=%F' notes.txt notes-hard.txt notes-soft.txt
echo "added via hard link" >> notes-hard.txt
echo "added via soft link" >> notes-soft.txt
cat notes.txt
readlink notes-soft.txt
```

```
notes.txt inode=1311301 links=2 type=regular file
notes-hard.txt inode=1311301 links=2 type=regular file
notes-soft.txt inode=1311302 links=1 type=symbolic link
original content
added via hard link
added via soft link
```

What just happened: Writing through either link changes the same data, so `cat notes.txt` shows both new lines. Writing through a symlink follows it to the target. `>>` appends instead of overwriting. `readlink` prints the path a symlink stores.

3. Delete the original and see what happens to each link.

```
rm notes.txt
ls -li
cat notes-hard.txt
cat notes-soft.txt
```

```
1311301 -rw-rw-r-- 1 student student 57 Oct 10 10:26 notes-hard.txt
1311302 lrwxrwxrwx 1 student student  9 Oct 10 10:25 notes-soft.txt -> notes.txt
original content
added via hard link
added via soft link
cat: notes-soft.txt: No such file or directory
```

What just happened: Removing `notes.txt` only removed one name. The inode's link count dropped from 2 to 1, so the data survives and `notes-hard.txt` still works. The symlink still contains the text `notes.txt`, but nothing has that name any more, so it is now *dangling* (`ls` shows it in red on a color terminal). Recreate the target and the symlink works again:

```
echo "brand new file" > notes.txt
cat notes-soft.txt
find . -xtype l
rm notes.txt
find . -xtype l
```

What just happened: The symlink now points at the *new* `notes.txt`, a completely different inode. `find . -xtype l` lists broken symlinks, a useful clean-up check (the first run prints nothing, the second prints `./notes-soft.txt`).

4. Try links to a directory and across filesystems. `/dev/shm` is a separate in-memory filesystem (tmpfs).

```
mkdir projdir
ln projdir projdir-hard
ln -s ~/labs/2.1/webapp/config confdir
ls -l confdir/
ln notes-hard.txt /dev/shm/cross-hard.txt
ln -s ~/labs/2.1/links/notes-hard.txt /dev/shm/cross-soft.txt
cat /dev/shm/cross-soft.txt
df -h . /dev/shm | awk '{print $1, $6}'
```

```
ln: projdir: hard link not allowed for directory
ln: failed to create hard link '/dev/shm/cross-hard.txt' => 'notes-hard.txt':
Invalid cross-device link
```

What just happened: You cannot hard-link a directory (it could create loops in the tree). You can symlink a directory: `confdir` acts like a shortcut to the config directory. The trailing `/` in `ls -l confdir/` makes `ls` follow the link and list the contents. A hard link to `/dev/shm` fails with "Invalid cross-device link" because inode numbers only make sense inside one filesystem. A symlink across filesystems works fine because it just stores a path. `df` shows the two paths are on different filesystems.

5. Notice relative versus absolute symlinks, and how `pwd` reports a path reached through a symlink.

```
cd confdir
pwd
pwd -P
cd ~/labs/2.1/links
```

```
/home/student/labs/2.1/links/confdir
/home/student/labs/2.1/webapp/config
```

What just happened: `pwd` shows the *logical* path, including the symlink you walked through. `pwd -P` shows the *physical* path with all symlinks resolved. A symlink created with a relative target (like `notes.txt`) is resolved relative to the directory the link lives in, so it breaks if you move the link somewhere else. An absolute target (like `confdir`'s) keeps working wherever the link is moved.

ON THE EXAM

On the exam: a hard link shares the inode and survives deletion of the original; it cannot span filesystems or point to a directory. A symbolic link has its own inode, can span filesystems and point to directories, and breaks when its target is removed. `ls -li` and `stat` show inode numbers and link counts.

Checkpoint: `cat ~/labs/2.1/links/notes-hard.txt` still shows three lines, `find ~/labs/2.1/links -xtype l` prints `notes-soft.txt`, and `cat /dev/shm/cross-soft.txt` prints the same content.

Task 8: Find open files with `lsof`

1. Install `lsof` (it is usually already present) and look at the files your own shell has open.

```
sudo apt install -y lsof
lsof -p $$
```

```
COMMAND  PID    USER   FD   TYPE DEVICE SIZE/OFF    NODE NAME
bash    1532 student  cwd    DIR  252,0     4096 1311300 /home/student/labs/2.1/links
bash    1532 student  rtd    DIR  252,0     4096       2 /
bash    1532 student  txt    REG  252,0  1446024  786563 /usr/bin/bash
bash    1532 student    0u   CHR  136,0      0t0       3 /dev/pts/0
bash    1532 student    1u   CHR  136,0      0t0       3 /dev/pts/0
bash    1532 student    2u   CHR  136,0      0t0       3 /dev/pts/0
...
```

What just happened: `$$` is a shell variable holding the PID (process ID) of your current shell, and `-p` limits `lsof` to that process. Columns: **COMMAND** and **PID** identify the process. **FD** is the file descriptor: `cwd` is the current working directory, `rtd` the root directory, `txt` the program file, and numbers like `0u`, `1u`, `2u` are standard input, output, and error (`u` means open for read and write). **TYPE** shows `DIR`, `REG` (regular file), or `CHR` (character device; your terminal `/dev/pts/0`).

2. Scenario: "I can't remove or unmount this directory because something is using it." Start a background process that sits inside a directory, then find it.

```
(cd ~/labs/2.1/webapp/logs && sleep 600) &
lsof +D ~/labs/2.1/webapp
```

```
COMMAND  PID    USER  FD   TYPE DEVICE SIZE/OFF    NODE NAME
sleep   1688 student cwd    DIR  252,0     4096 1311279 /home/student/labs/2.1/webapp/logs
```

What just happened: The parentheses run a *subshell* that changes into `logs` and then runs `sleep 600` in the background (`&`). `lsof +D DIR` searches the whole directory tree for anything open, including processes whose current directory is inside it. This is exactly how you find what is keeping a filesystem busy before `umount`. `lsof DIRNAME` without `+D` checks only that one directory.

3. Scenario: "I deleted a huge log file but the disk space did not come back." Create a 200 MB log file and hold it open with `tail -f`.

```
cd ~/labs/2.1
dd if=/dev/zero of=huge.log bs=1M count=200 status=none
tail -f huge.log > /dev/null &
df -h /
rm huge.log
df -h /
```

What just happened: `tail -f` keeps the file open, waiting for new lines. After `rm`, the name is gone, but the second `df -h /` shows the *same* used space as before, because the inode still has an open file descriptor. The kernel frees the data only when the link count is 0 **and** no process has it open.

4. Find the deleted-but-open file and release the space.

```
lsof +L1
sudo lsof -u student | grep deleted
kill %2
df -h /
lsof +L1
```

```
COMMAND  PID    USER  FD  TYPE DEVICE  SIZE/OFF NLINK    NODE NAME
tail    1701 student  3r   REG  252,0 209715200     0 1311320
/home/student/labs/2.1/huge.log (deleted)
```

What just happened: `lsof +L1` lists open files with a link count less than 1, which means files that were deleted but are still open. The **NLINK** column is `0` and the name ends with `(deleted)`. `-u student` shows all files opened by a user. `kill %2` stops background job number 2 (the `tail`; job control is covered in Lab 2.3). After that, `df` shows the 200 MB freed. On a real server the usual fix is to restart the service that holds the log.

5. Two more lookups you will use constantly.

```
sudo lsof -i :22
lsof /dev/null | head -5
```

What just happened: `-i :22` lists network sockets using port 22 (the SSH server and your SSH session). `lsof FILE` lists every process that has a particular file open.

Checkpoint: `lsof +L1` prints nothing (the deleted file was released), and `jobs` shows only the `sleep 600` job still running.

Task 9: Tour the device files in `/dev`

1. List block and character devices side by side.

```
lsblk
ls -l /dev/sd* /dev/vd* /dev/nvme* 2>/dev/null
ls -l /dev/null /dev/zero /dev/random /dev/urandom /dev/full /dev/tty
```

```
brw-rw---- 1 root disk 8,  0 Oct 10 09:01 /dev/sda
brw-rw---- 1 root disk 8,  1 Oct 10 09:01 /dev/sda1
brw-rw---- 1 root disk 8, 16 Oct 10 09:01 /dev/sdb
brw-rw---- 1 root disk 8, 32 Oct 10 09:01 /dev/sdc
crw-rw-rw- 1 root root 1, 7 Oct 10 09:01 /dev/full
crw-rw-rw- 1 root root 1, 3 Oct 10 09:01 /dev/null
crw-rw-rw- 1 root root 1, 8 Oct 10 09:01 /dev/random
crw-rw-rw- 1 root tty  5, 0 Oct 10 09:01 /dev/tty
crw-rw-rw- 1 root root 1, 9 Oct 10 09:01 /dev/urandom
crw-rw-rw- 1 root root 1, 5 Oct 10 09:01 /dev/zero
```

What just happened: `lsblk` lists your disks so you know their names (they may be `vda` or `nvme0n1` instead of `sda`). In `ls -l`, disks start with `b` (**block device**) and are owned by group `disk`. The others start with `c` (**character device**). Where the file size normally appears you see two numbers: **major, minor**. All the SCSI/SATA disks share major `8` (the `sd` driver); the minor number picks the disk and partition (each disk gets 16 minors: `sda`=0, `sdb`=16). Major `1` is the kernel's "memory" driver, which provides the special character devices: minor `3` is null, `5` zero, `7` full, `8` random, `9` urandom.

2. See which drivers own which major numbers, and count device types.

```
grep -A8 'Character devices' /proc/devices
grep -A8 'Block devices' /proc/devices
find /dev -maxdepth 1 -type b | wc -l
find /dev -maxdepth 1 -type c | wc -l
```

What just happened: `/proc/devices` lists registered drivers and their major numbers in two sections (for example `1 mem`, `5 /dev/tty`, `136 pts`, `8 sd`). `find -type b` and `-type c` match block and character device files. Most of `/dev` is character devices.

3. Use `/dev/null`, the "black hole".

```
echo "nobody will see this" > /dev/null
cat /dev/null | wc -c
ls /root 2>/dev/null; echo "exit status: $?"
cp /dev/null ~/labs/2.1/webapp/logs/app.log
```

What just happened: Anything written to `/dev/null` is discarded, and reading it returns end-of-file immediately (0 bytes). Its main use is hiding output you do not want, like the error from `ls /root` (you still get the exit status). Copying `/dev/null` over a file empties it, a quick way to truncate a log.

4. Use `/dev/zero` and `/dev/full`.

```
head -c 16 /dev/zero | od -An -tx1
dd if=/dev/zero of=~/labs/2.1/zeros.bin bs=1M count=5 status=none
ls -lh ~/labs/2.1/zeros.bin
echo test > /dev/full
```

```
 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
-rw-rw-r-- 1 student student 5.0M Oct 10 10:40 /home/student/labs/2.1/zeros.bin
-bash: echo: write error: No space left on device
```

What just happened: `/dev/zero` returns as many zero bytes as you read. `head -c 16` takes 16 bytes and `od -An -tx1` dumps them as hex. With `dd` it creates files of a known size or wipes a disk. `/dev/full` always fails writes with "No space left on device", which developers use to test how programs handle a full disk.

5. Use `/dev/urandom` and `/dev/random`.

```
head -c 16 /dev/urandom | od -An -tx1
head -c 12 /dev/random | base64
tr -dc 'A-Za-z0-9' < /dev/urandom | head -c 20; echo
```

What just happened: Both devices return random bytes from the kernel's cryptographic random generator. The last line keeps only letters and digits from the random stream and takes 20 of them, a quick random password. Historically `/dev/random` *blocked* (paused) when the kernel estimated it was low on randomness, while `/dev/urandom` never blocks. Since kernel 5.6 `/dev/random` only blocks very early at boot, before the generator is ready; on a running system both behave the same.

6. Use `/dev/tty`, which always means "the terminal of the current process".

```
tty
echo "hello terminal" > /dev/tty
echo "hidden" > /dev/null; echo "shown anyway" 2>/dev/null > /dev/tty
```

```
/dev/pts/0
hello terminal
shown anyway
```

What just happened: `tty` prints the device file of your terminal. Over SSH this is a *pseudo-terminal* like `/dev/pts/0` (character device, major 136). `/dev/tty` is a special alias that always points to whatever terminal the current process is attached to. Scripts write to `/dev/tty` (or read passwords from it) when they must reach the user even if normal output is redirected to a file.

TIP

Explore/simulate: Physical serial ports appear as `/dev/ttyS0`, `/dev/ttyS1` (character devices, major 4) and USB serial adapters as `/dev/ttyUSB0`. Run `ls -l /dev/ttyS*` to see them even though no cable is attached. Virtual consoles (the screens you get on the VM console with `Ctrl+Alt+F1` … `F6`) are `/dev/tty1` to `/dev/tty6`.

Checkpoint: `ls -l /dev/null` starts with `c` and shows `1, 3`. `ls -l` on your first disk starts with `b`. `stat -c '%F' /dev/zero` prints `character special file`.

Challenge

CHALLENGE

1. Using one `find` command, list every file under `/etc` that is larger than 100 KB and was modified in the last 30 days, and show each with `ls -lh`.
2. Create a file in `~/labs/2.1`, make three hard links to it in different subdirectories, then use `find ~/labs/2.1 -samefile FILE` (or `-inum`) to list all four names. What link count does `stat` report?
3. A colleague says "the disk is full but `du` doesn't show any big files." Reproduce the situation with `dd` and `less`, explain why `df` and `du` disagree, and fix it without rebooting.

Clean up

1. Stop the background `sleep` (and anything else left from this lab) and confirm no jobs remain.

```
jobs
kill %1 2>/dev/null
pkill -u student -x sleep
pkill -u student -x tail
jobs
```

2. Remove the lab files, including the symlink in `/dev/shm`.

```
cd ~
rm -f /dev/shm/cross-soft.txt /dev/shm/cross-hard.txt
rm -rf ~/labs/2.1
```

3. Rebuild the `locate` database so it forgets the deleted files.

```
sudo updatedb
locate app.conf.bak
```

The last command should print nothing. You can leave `plocate` and `lsof` installed; later labs may use them.

Exam corner

- `ls -l` first character: `-` file, `d` directory, `l` symlink, `b` block device, `c` character device, `p` pipe, `s` socket. `ls -i` shows inode numbers.
- `cp -r` copies directories; `cp -a` copies recursively and preserves everything; `cp -p` preserves mode, ownership, and timestamps. `mv` renames or moves. `rmdir` only removes empty directories; `rm -r` removes non-empty ones.
- `mkdir -p` creates parent directories as needed. `pwd -P` resolves symlinks. `cd -` returns to the previous directory.

- `touch` creates empty files or updates timestamps; `touch -d "date"` or `-t YYYYMMDDhhmm` sets a specific time. ctime cannot be set manually.
- `stat` shows the inode: size, blocks, inode number, link count, permissions, UID/GID, and atime/mtime/ctime/birth.
- `file` identifies content by magic numbers, not by extension.
- `find PATH TESTS ACTION`: `-name`, `-iname`, `-type`, `-size +10M`, `-mtime +7`, `-mmin -30`, `-perm -4000`, `-user`, `-group`, `-exec cmd {} \;`, `-exec cmd {} +`, `-ok`, `-delete`.
- `locate` searches a database built by `updatedb` (package `plocate` on Ubuntu 24.04, `mlocate` on Rocky 9). It is fast but can be stale.
- `diff` shows changes (`<` first file, `>` second, `c` / `a` / `d`); `diff -u` is unified format; `sdiff` shows side by side with `|`, `<`, `>`.
- Hard link: same inode, link count increases, survives deletion of the original, no directories, no cross-filesystem. Symbolic link (`ln -s`): own inode, stores a path, can cross filesystems and point to directories, breaks if the target is removed.
- `lsof +D dir` finds what is using a directory; `lsof +L1` finds deleted-but-open files; `lsof -i :port`; `lsof -u user`; `lsof -p PID`.
- Block devices (disks, partitions) use random block access; character devices (terminals, serial ports) are byte streams. Special character devices: `/dev/null`, `/dev/zero`, `/dev/random`, `/dev/urandom`, `/dev/full`, `/dev/tty`. Device files show major (driver) and minor (instance) numbers.

Review questions

1. An administrator deletes `/data/report.csv`, which had a hard link at `/archive/report.csv` on the same filesystem. What happens to `/archive/report.csv`? A. It becomes a broken link. B. It is deleted as well. C. It still works and its link count drops by one. D. It is converted to a symbolic link.

2. Which command finds all files under `/var` larger than 500 MB? A. `locate -size +500M /var` B. `find /var -size +500M` C. `find /var -mtime +500` D. `ls -lhR /var --size 500M`

3. `df` shows `/var` is 95% full, but you removed a 10 GB log file an hour ago. Which command best identifies the cause? A. `lsof +L1` B. `stat /var` C. `sdiff /var /tmp` D. `updatedb`

4. You run `ls -l /dev/sdb` and the line begins with `b` and shows `8, 16`. What do the `b` and the numbers mean?

5. A user runs `locate nginx.conf` immediately after installing nginx, and nothing is found. What should they do?

6. Which command shows two versions of a file side by side, marking differing lines with `|`? A. `diff -u` B. `cmp` C. `sdiff` D. `stat`

PART 5 · DOMAIN 5.0

Part 5: Troubleshooting

22% of the exam · 1 labs

5.1 Summarize monitoring concepts and configurations in a Linux system.

Lab 5.1 Monitoring Concepts and Configurations

Objective: 5.1 Summarize monitoring concepts and configurations in a Linux system.
Domain: 5.0 Troubleshooting (22% of the exam)
Time: about 100 minutes
VM: ubuntu-lab and rocky-lab
Snapshot first: Recommended. This lab installs several services and changes logging on both VMs.

You cannot fix what you cannot see. In this lab you build a small but real monitoring setup: a service level objective for a web server with a health check that measures it, threshold checks that raise alerts and send a webhook, an agent (node_exporter) compared with agentless polling, SNMP polling and traps between your two VMs, and central log aggregation with rsyslog. Every piece is a concept the exam names, and every piece is something you will meet at work.

What you'll practice

- Service monitoring: service level agreement (SLA), service level indicator (SLI), service level objective (SLO), error budgets
- Data acquisition: SNMP (`snmpwalk`, `snmpget`, community strings, SNMPv3 users, traps with `snmptrap` / `snmptrapd`, MIBs and OIDs)
- Data acquisition: agent versus agentless monitoring (`prometheus-node-exporter` versus SSH polling)
- Data acquisition: webhooks, health checks, and log aggregation (rsyslog forwarding over TCP 514)
- Configurations: thresholds, alerts, events, notifications, and logging (`logger`, `journalctl`, `wall`)

How it works

Monitoring means collecting data about systems over time and reacting when something looks wrong. It has three layers: *what you promise*, *what you measure*, and *how you react*.

What you promise: SLA, SLO, SLI. Picture a pizza shop.

- The **SLA (service level agreement)** is the contract with the customer: "Delivery in 45 minutes or the pizza is free." It is a business document with consequences (refunds, credits, penalties).
- The **SLO (service level objective)** is the internal target the shop aims for, usually stricter than the SLA so there is a safety margin: "We aim to deliver 99% of pizzas in 30 minutes."
- The **SLI (service level indicator)** is the actual measurement: "Last month, 98.7% of pizzas arrived within 30 minutes."

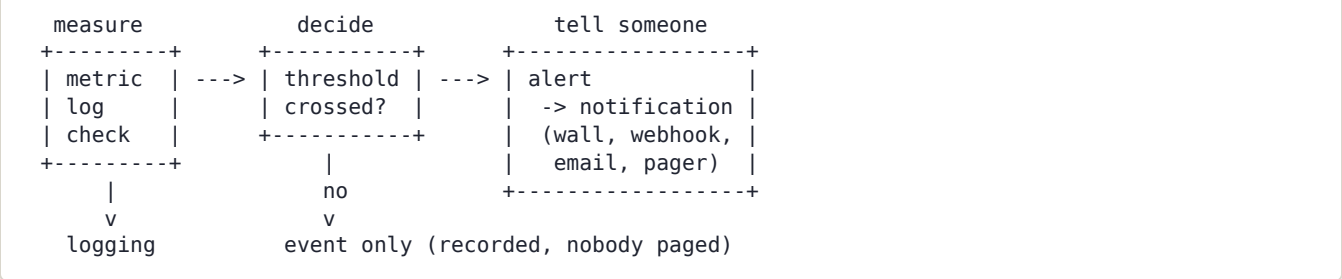
For a web server, a typical SLI is *availability* (the percentage of health checks that succeeded) or *latency* (the percentage of requests answered in under 300 ms). The SLO might be "99.9% available per month." The gap between 100% and the SLO is the **error budget**: 0.1% of a 30-day month is 43.2 minutes of allowed downtime. When the budget is used up, the team stops shipping risky changes.

What you measure: data acquisition. There are several ways to get data out of a system:

- A **health check** is a small, repeated test that asks "are you working?" For a web server, that means requesting a page and checking for HTTP status `200`. Load balancers use health checks to remove broken servers from rotation.
- An **agent** is software you install on the monitored host that collects data and exposes or sends it. Prometheus `node_exporter`, the Zabbix agent, and the Datadog agent are examples. Agents give rich, detailed data but you must install, update, and secure them on every host.
- **Agentless** monitoring uses something the host already has, usually SSH (run `uptime` remotely) or SNMP. Nothing new to install, but the data is limited to what those protocols expose, and the monitoring server needs credentials.
- **SNMP (Simple Network Management Protocol)** is the classic protocol for polling network devices and servers. A manager *polls* an SNMP agent on UDP port 161 and asks for values. Each value has an **OID (object identifier)**, a dotted number such as `1.3.6.1.2.1.1.5.0` (the host name). A **MIB (Management Information Base)** is a text file that gives names and descriptions to those numbers, so `1.3.6.1.2.1.1.5.0` can be shown as `SNMPv2-MIB::sysName.0`. A **trap** is the opposite direction: the agent sends an unrequested message to the manager on UDP port 162 when something happens (link down, disk full). SNMPv1 and v2c use a plain-text password called a **community string**; SNMPv3 adds real users with authentication and encryption.
- A **webhook** is an HTTP POST, usually with a JSON body, that one system sends to a URL on another system when an event happens. Chat tools, ticketing systems, and paging services all accept webhooks.
- **Log aggregation** means shipping logs from many hosts to one central place, so you can search them together and keep them safe if a host dies or is compromised. On Linux the simplest form is rsyslog forwarding to a central rsyslog server.

How you react: configurations. These words are related but not the same:

- **Logging** is recording what happened (to the journal or a log file).
- An **event** is any notable thing that happened ("nginx restarted", "check failed").
- A **threshold** is a limit that turns a measurement into a problem ("disk usage above 80%").
- An **alert** is the decision that a threshold was crossed and a human should care.
- A **notification** is how the alert reaches a human or system: email, `wall` message, chat webhook, pager.



A common mistake is alerting on everything. When people get dozens of meaningless alerts a day they start ignoring all of them, including the important one. This is called **alert fatigue**. Good alerts are actionable (someone can do something), tied to user impact (the SLO), and tuned so they rarely fire falsely.

Tasks

Task 1: Define an SLO and build a health check

You will run nginx on ubuntu-lab, define an SLO for it, and measure the SLI with a health check that a systemd timer runs every 15 seconds.

1. Create the lab directory and install nginx and curl.

```
mkdir -p ~/labs/5.1 && cd ~/labs/5.1
sudo apt update
sudo apt install -y nginx curl
curl -s -o /dev/null -w '%{http_code} %{time_total}\n' http://localhost/
```

```
200 0.001412
```

> **What just happened:** nginx installs and starts automatically on Ubuntu. The `curl` options: `-s` is silent (no progress bar), `-o /dev/null` throws away the page body, and `-w` (write-out) prints chosen variables after the transfer. `%{http_code}` is the HTTP status code and `%{time_total}` is the total time in seconds. This one line is a complete health check: the service is up (`200`) and fast (1.4 ms).

2. Write down your service levels. Create a short file so you can refer to it. A **heredoc** (`cat <<'EOF' > file` … `EOF`) sends all the lines between the markers into the file; the quotes around `EOF` stop the shell from changing `$` signs inside.

```
cat <<'EOF' > ~/labs/5.1/service-levels.txt
Service: lab web site (nginx on ubuntu-lab)
SLA (promise to customers): 99.5% monthly availability, else service credit
SLO (internal target):      99.9% of health checks return HTTP 200
SLI (what we measure):      successful checks / total checks, every 15 s
Latency SLI:                checks answered in under 0.300 s
EOF
cat ~/labs/5.1/service-levels.txt
```

> **What just happened:** You wrote the three levels in the order a team would. The SLO (99.9%) is stricter than the SLA (99.5%) so you get warned before you break the contract. The SLI is defined precisely enough that a script can calculate it.

3. Create the health check script. Open it with `sudo nano /usr/local/bin/web-healthcheck` or use the heredoc below. `tee` writes its input to the file; `sudo tee` lets you write to a root-owned path.

```
cat <<'EOF' | sudo tee /usr/local/bin/web-healthcheck > /dev/null
#!/bin/bash
# web-healthcheck: probe the web service once and record the result
URL="http://localhost/"
LOG=/var/log/web-health.log
result=$(curl -s -o /dev/null -w '%{http_code} %{time_total}' \
         --max-time 2 "$URL")
code=${result%% *}
secs=${result##* }
if [ "$code" = "200" ]; then state=UP; else state=DOWN; fi
echo "$(date -Is) $state $code $secs" >> "$LOG"
if [ "$state" = "DOWN" ]; then
    logger -t healthcheck -p daemon.err "web check FAILED code=$code url=$URL"
fi
EOF
sudo chmod 755 /usr/local/bin/web-healthcheck
sudo /usr/local/bin/web-healthcheck
cat /var/log/web-health.log
```

```
2026-10-10T10:02:11+00:00 UP 200 0.001377
```

What just happened: The script runs one check and appends one line: timestamp, state, status code, response time. `--max-time 2` makes curl give up after 2 seconds, so a hung server counts as a failure instead of hanging the check. `${result%% *}` keeps the text before the first space (the code) and `${result##* }` keeps the text after the last space (the time). If nginx is down, curl prints code `000`. On failure, `logger` writes an **event** into the system log with tag `healthcheck` and priority `daemon.err` (facility `daemon`, severity `err`).

4. Run the check every 15 seconds with a systemd service and timer. The service says *what* to run; the timer says *when*.

```
cat <<'EOF' | sudo tee /etc/systemd/system/web-healthcheck.service
[Unit]
Description=Health check for the lab web site

[Service]
Type=oneshot
ExecStart=/usr/local/bin/web-healthcheck
EOF
cat <<'EOF' | sudo tee /etc/systemd/system/web-healthcheck.timer
[Unit]
Description=Run the web health check every 15 seconds

[Timer]
OnBootSec=30s
OnUnitActiveSec=15s
AccuracySec=1s

[Install]
WantedBy=timers.target
EOF
sudo systemctl daemon-reload
sudo systemctl enable --now web-healthcheck.timer
systemctl list-timers web-healthcheck.timer
```

```
NEXT                         LEFT LAST                         PASSED UNIT
Sat 2026-10-10 10:03:01 UTC  9s  Sat 2026-10-10 10:02:46 UTC 5s ago web-healthcheck.timer
```

What just happened: `Type=oneshot` means the service runs once and exits. In the timer, `OnUnitActiveSec=15s` starts the service 15 seconds after it last started, `OnBootSec=30s` starts the first run 30 seconds after boot, and `AccuracySec=1s` stops systemd from batching the run up to a minute late (the default accuracy). `enable --now` starts the timer immediately and at every boot.

5. Create some downtime so the SLI has something to measure. Wait about a minute, stop nginx for about one minute, then start it again.

```
sleep 60; sudo systemctl stop nginx; sleep 60; sudo systemctl start nginx
tail -n 8 /var/log/web-health.log
journalctl -t healthcheck --since "10 minutes ago"
```

```
2026-10-10T10:04:16+00:00 UP 200 0.001290
2026-10-10T10:04:31+00:00 DOWN 000 0.000214
2026-10-10T10:04:46+00:00 DOWN 000 0.000198
2026-10-10T10:05:01+00:00 DOWN 000 0.000202
2026-10-10T10:05:16+00:00 DOWN 000 0.000187
2026-10-10T10:05:31+00:00 UP 200 0.001544
Oct 10 10:04:31 ubuntu-lab healthcheck[2210]: web check FAILED code=000
url=http://localhost/
```

6. Calculate the availability SLI and the latency SLI with `awk`.

```
awk '{ total++ } $2 == "UP" { up++ }
     END { printf "checks=%d up=%d availability=%.2f%%\n", total, up, 100*up/total }' \
     /var/log/web-health.log
awk '{ total++ } $2 == "UP" && $4 < 0.300 { fast++ }
     END { printf "latency SLI=%.2f%% under 300 ms\n", 100*fast/total }' \
     /var/log/web-health.log
```

```
checks=14 up=10 availability=71.43%
latency SLI=71.43% under 300 ms
```

What just happened: `awk` reads the log line by line. Field `$2` is the state and `$4` is the response time. It counts all lines, counts the good ones, and prints the percentage at the `END`. Your numbers will differ. Over a short window, one minute of downtime destroys the SLO; over a month, that same minute would use only about 2% of a 99.9% error budget (1 of 43.2 minutes). This is why SLOs are always stated over a time window.

Checkpoint: `systemctl list-timers web-healthcheck.timer` shows the timer, `/var/log/web-health.log` grows by one line every 15 seconds, it contains some `DOWN` lines, and `journalctl -t healthcheck` shows the failure events.

Task 2: Thresholds, events, alerts, and notifications

Now build a threshold check for disk usage and load average. It records every run as an event, and only raises an alert (with a notification) when a threshold is crossed.

1. Look at the two raw numbers you will check.

```
df --output=pcent / | tail -n 1
cat /proc/loadavg
nproc
```

```
 23%
0.08 0.05 0.01 1/187 2380
2
```

What just happened: `df --output=pcent /` prints only the "use percent" column for the root filesystem; `tail -n 1` drops the header. `/proc/loadavg` holds the 1, 5, and 15-minute load averages (the average number of processes running or waiting). A load equal to the number of CPUs (`nproc`, here 2) means the CPUs are fully busy, so `nproc` is a sensible default load threshold.

2. Create the threshold script.

```
cat <<'EOF' | sudo tee /usr/local/bin/threshold-check > /dev/null
#!/bin/bash
# threshold-check: compare disk and load against limits and alert
DISK_LIMIT=${DISK_LIMIT:-80}          # percent
LOAD_LIMIT=${LOAD_LIMIT:-$(nproc)}    # 1-minute load average
WEBHOOK=${WEBHOOK:-}                  # optional URL for notifications

disk=$(df --output=pcent / | tail -n 1 | tr -dc '0-9')
load=$(cut -d ' ' -f 1 /proc/loadavg)

alert() {
    logger -t threshold -p user.warning "ALERT $1"
    wall "ALERT on $(hostname): $1"
    if [ -n "$WEBHOOK" ]; then
        curl -s -X POST -H 'Content-Type: application/json' \
            -d "{\"host\":\"$(hostname)\",\"severity\":\"warning\",\"message\":\"$1\"}" \
            "$WEBHOOK" > /dev/null
    fi
}

if [ "$disk" -ge "$DISK_LIMIT" ]; then
    alert "disk / at ${disk}% (limit ${DISK_LIMIT}%)"
fi
if awk -v l="$load" -v m="$LOAD_LIMIT" 'BEGIN { exit !(l > m) }'; then
    alert "load ${load} above ${LOAD_LIMIT}"
fi
logger -t threshold -p user.info "event: check done disk=${disk}% load=${load}"
EOF
sudo chmod 755 /usr/local/bin/threshold-check
```

What just happened: `${DISK_LIMIT:-80}` means "use the variable if it is set, otherwise 80", so you can change limits without editing the script. `tr -dc '0-9'` deletes every character that is not a digit, turning `23%` into `23`. Bash cannot compare decimals like `0.08`, so `awk` does the load comparison and sets the exit status. Every run logs an informational **event**; only a crossed threshold calls `alert`, which logs a warning and sends **notifications** with `wall` (a message to every logged-in terminal) and, if configured, a webhook.

3. Run it with normal limits, then with a deliberately tiny disk limit to force an alert.

```
threshold-check
DISK_LIMIT=1 threshold-check
journalctl -t threshold --since "5 minutes ago"
```

```
Broadcast message from student@ubuntu-lab (pts/0) (Sat Oct 10 10:12:40 2026):

ALERT on ubuntu-lab: disk / at 23% (limit 1%)

Oct 10 10:12:31 ubuntu-lab threshold[2401]: event: check done disk=23% load=0.02
Oct 10 10:12:40 ubuntu-lab threshold[2412]: ALERT disk / at 23% (limit 1%)
Oct 10 10:12:40 ubuntu-lab threshold[2415]: event: check done disk=23% load=0.02
```

4. Filter the journal by severity. Severity (priority) levels go from `0` (emerg) to `7` (debug); `-p warning` shows warning and anything more severe.

```
journalctl -t threshold -p warning --since "5 minutes ago"
```

> **What just happened:** Only the alert line appears, not the routine events. Monitoring systems work the same way: everything is logged, but only high-severity items reach people. If you set `DISK_LIMIT=1` permanently, you would get a `wall` message every run for a condition nobody needs to fix. That is how alert fatigue starts. A good threshold is high enough to be meaningful and low enough to leave time to act (for example, warn at 80%, page at 95%).

> **Checkpoint:** `journalctl -t threshold -p warning` shows exactly one `ALERT` line from the forced test, and you saw the `Broadcast message` from `wall` in your terminal.

Task 3: Send alerts with a webhook

Real alerts usually go to a chat or paging system through a webhook. You will run a tiny webhook receiver on ubuntu-lab and point the threshold script at it. (Note: `python3 -m http.server` only answers GET requests, so it cannot receive a webhook. You need a listener that handles POST.)

1. Create a 15-line Python listener.

```
cat <<'EOF' > ~/labs/5.1/webhook-listener.py
#!/usr/bin/env python3
# Minimal webhook receiver: prints the JSON body of every POST
import json
from http.server import BaseHTTPRequestHandler, HTTPServer

class Hook(BaseHTTPRequestHandler):
    def do_POST(self):
        length = int(self.headers.get("Content-Length", 0))
        body = json.loads(self.rfile.read(length))
        print("WEBHOOK RECEIVED:", body, flush=True)
        self.send_response(200)
        self.end_headers()
        self.wfile.write(b"ok\n")

HTTPServer(("127.0.0.1", 8099), Hook).serve_forever()
EOF
python3 ~/labs/5.1/webhook-listener.py
```

> **What just happened:** `BaseHTTPRequestHandler` calls `do_POST` for each POST request. The handler reads exactly `Content-Length` bytes, parses the JSON, prints it, and answers `200 ok`. It listens only on `127.0.0.1` port 8099, so nothing outside the VM can reach it. Leave it running in this terminal.

2. Open a **second** SSH session to ubuntu-lab and send a test webhook by hand.

```
curl -s -X POST -H 'Content-Type: application/json' \
    -d '{"host":"ubuntu-lab","severity":"info","message":"test"}' \
    http://127.0.0.1:8099/alert
```

In the first terminal you see:

```
WEBHOOK RECEIVED: {'host': 'ubuntu-lab', 'severity': 'info', 'message': 'test'}
127.0.0.1 - - [10/Oct/2026 10:20:05] "POST /alert HTTP/1.1" 200 -
```

What just happened: `-X POST` sets the HTTP method, `-H` adds a header telling the receiver the body is JSON, and `-d` supplies the body. This is exactly what Slack, Microsoft Teams, PagerDuty, or a ticketing system would receive.

3. Trigger a real threshold alert that notifies through the webhook.

```
DISK_LIMIT=1 WEBHOOK=http://127.0.0.1:8099/alert threshold-check
```

```
WEBHOOK RECEIVED: {'host': 'ubuntu-lab', 'severity': 'warning', 'message': 'disk / at 23%
(limit 1%)'}
```

4. Stop the listener in the first terminal with `Ctrl+C`.

Checkpoint: The listener printed a `WEBHOOK RECEIVED` line with `severity: warning` that came from `threshold-check`.

Task 4: Agent versus agentless monitoring

1. Install the Prometheus node exporter, an **agent** that exposes hundreds of metrics over HTTP.

```
sudo apt install -y prometheus-node-exporter
systemctl is-active prometheus-node-exporter
ss -tlnp | grep 9100
curl -s http://localhost:9100/metrics | grep -E '^node_load1
|^node_memory_MemAvailable_bytes'
```

```
active
LISTEN 0  4096   *:9100   *:*
node_load1 0.04
node_memory_MemAvailable_bytes 3.412783104e+09
```

What just happened: The package installs and starts the `prometheus-node-exporter` service on TCP port 9100. `ss -tlnp` lists listening TCP sockets with numbers and process names. The `/metrics` page is plain text, one metric per line; a Prometheus server would *scrape* (fetch) it every 15 to 60 seconds and store the history. `grep -E` picks out the 1-minute load and the available memory in bytes.

2. Count how much data the agent gives you.

```
curl -s http://localhost:9100/metrics | grep -vc '^#'
```

```
1247
```

3. Now do **agentless** monitoring from rocky-lab: nothing extra is installed on ubuntu-lab, you just use SSH, which is already there. Enter the student password when asked.

```
# on rocky-lab
ssh student@ubuntu-lab 'uptime; df -h / | tail -n 1; free -m | grep Mem'
```

```
 10:31:02 up  2:11,  2 users,  load average: 0.03, 0.04, 0.01
/dev/mapper/ubuntu--vg-ubuntu--lv   24G  5.4G   17G  25% /
Mem:            3915     560    2780       1     810    3354
```

What just happened: Tools like Ansible and many monitoring systems work this way: they log in over SSH, run commands, and parse the output. Compare the two styles:

	Agent (node_exporter)	Agentless (SSH, SNMP)
Install on host	Yes, on every host	No, uses existing services
Data detail	Very rich (1,000+ metrics)	Whatever commands or MIBs expose
Credentials	Usually none to read metrics	Monitoring server needs SSH keys or SNMP credentials
Overhead	Small always-running process	A login or poll each interval
Good for	Servers you control	Network gear, appliances, locked-down hosts

Checkpoint: `curl -s localhost:9100/metrics | grep node_load1` on ubuntu-lab and the SSH one-liner on rocky-lab both show the load average of ubuntu-lab.

Task 5: Poll a host with SNMP and learn MIBs

You will make ubuntu-lab an SNMP agent and poll it from rocky-lab with SNMPv2c and SNMPv3.

1. Install the SNMP agent (`snmpd`), the client tools (`snmp`), and the MIB downloader on ubuntu-lab. `snmp-mibs-downloader` comes from the `multiverse` component, which Ubuntu 24.04 enables by default.

```
sudo apt install -y snmpd snmp snmp-mibs-downloader
sudo cp /etc/snmp/snmpd.conf /etc/snmp/snmpd.conf.orig
```

2. Replace the agent configuration. The default only listens on localhost and only shows the `system` group. Substitute your lab subnet if it is not `192.168.100.0/24` .

   The file is `/etc/snmp/snmpd.conf` :

```
cat <<'EOF' | sudo tee /etc/snmp/snmpd.conf
# Listen on all IPv4 addresses, UDP port 161
agentaddress udp:161
sysLocation  Lab rack 1
sysContact   student <student@ubuntu-lab>
# A view is the part of the OID tree a client may read
view labview included .1.3.6.1.2.1.1
view labview included .1.3.6.1.2.1.25
view labview included .1.3.6.1.4.1.2021
# SNMPv2c: read-only community "labcomm" from the lab subnet only
rocommunity labcomm 192.168.100.0/24 -V labview
rocommunity labcomm 127.0.0.1 -V labview
# SNMPv3: read-only user that must authenticate and encrypt
rouser labv3 priv -V labview
EOF
```

What just happened: `agentaddress udp:161` makes snmpd listen on every interface. A `view` limits which branches of the OID tree are visible: `.1.3.6.1.2.1.1` is the `system` group (name, uptime, location), `.1.3.6.1.2.1.25` is HOST-RESOURCES (storage, processes), and `.1.3.6.1.4.1.2021` is the UCD-SNMP group (load averages, memory). `rocommunity` creates a read-only community string that only works from the listed source addresses. `rouser labv3 priv` allows the SNMPv3 user `labv3` read-only access, but only with authentication **and** privacy (encryption).

3. Create the SNMPv3 user. snmpd reads `createUser` lines from its persistent file at start-up, replaces them with hashed keys, and removes the clear-text passwords.

```
sudo systemctl stop snmpd
echo 'createUser labv3 SHA "AuthPass123" AES "PrivPass123"' | \
    sudo tee -a /var/lib/snmp/snmpd.conf
sudo systemctl start snmpd
sudo grep -c createUser /var/lib/snmp/snmpd.conf
ss -ulnp | grep :161
```

```
0
UNCONN 0  0   0.0.0.0:161   0.0.0.0:*
```

What just happened: `SHA` is the authentication hash and `AES` is the encryption cipher. The count of `createUser` lines is now `0` because snmpd converted it into a `usmUser` line with localized keys. `ss -ulnp` shows snmpd listening on UDP 161 on all addresses. If `ufw` is active on your VM, also run `sudo ufw allow from 192.168.100.0/24 to any port 161 proto udp`.

4. Turn on MIB names for the Ubuntu client. Debian and Ubuntu disable MIB loading by default for licensing reasons, so you only see numbers until you comment out the `mibs :` line.

```
snmpget -v2c -c labcomm localhost 1.3.6.1.2.1.1.5.0
sudo sed -i 's/^mibs :/#mibs :/' /etc/snmp/snmp.conf
snmpget -v2c -c labcomm localhost 1.3.6.1.2.1.1.5.0
ls /usr/share/snmp/mibs | head -n 5
```

```
iso.3.6.1.2.1.1.5.0 = STRING: "ubuntu-lab"
SNMPv2-MIB::sysName.0 = STRING: ubuntu-lab
GNOME-SMI.txt
HOST-RESOURCES-MIB.txt
...
```

What just happened: The first answer is just a number (`iso.3.6.1.2.1.1.5.0`). After enabling the MIBs downloaded by `snmp-mibs-downloader` , the same OID is translated to `SNMPv2-MIB::sysName.0` . The MIB changed only how the *client* displays the answer; the agent sent the same data. `snmptranslate -Td SNMPv2-MIB::sysName.0` would print the MIB description of that object.

5. Poll ubuntu-lab from rocky-lab. Install the client tools there; Rocky ships the standard MIBs with them and loads them by default.

```
# on rocky-lab
sudo dnf install -y net-snmp-utils
snmpwalk -v2c -c labcomm ubuntu-lab system
snmpget -v2c -c labcomm ubuntu-lab SNMPv2-MIB::sysUpTime.0 UCD-SNMP-MIB::laLoad.1
```

```
SNMPv2-MIB::sysDescr.0 = STRING: Linux ubuntu-lab 6.8.0-45-generic #45-Ubuntu SMP ...
SNMPv2-MIB::sysObjectID.0 = OID: NET-SNMP-MIB::netSnmpAgentOIDs.10
DISMAN-EVENT-MIB::sysUpTimeInstance = Timeticks: (791265) 2:11:52.65
SNMPv2-MIB::sysContact.0 = STRING: student <student@ubuntu-lab>
SNMPv2-MIB::sysName.0 = STRING: ubuntu-lab
SNMPv2-MIB::sysLocation.0 = STRING: Lab rack 1
...
DISMAN-EVENT-MIB::sysUpTimeInstance = Timeticks: (791402) 2:11:54.02
UCD-SNMP-MIB::laLoad.1 = STRING: 0.02
```

What just happened: `-v2c` selects SNMP version 2c and `-c labcomm` gives the community string. `snmpwalk` walks every OID under a starting point (`system` is a MIB name for `.1.3.6.1.2.1.1`). `snmpget` fetches exact OIDs; the `.0` suffix means "the single instance" of a scalar value and `.1` in `laLoad.1` is the first row of the load table (the 1-minute load). Try `snmpwalk -v2c -c wrong ubuntu-lab system` : it times out, because a wrong community gets no reply at all.

6. Poll with SNMPv3 and compare what a sniffer would see.

```
# on rocky-lab
snmpget -v3 -l authPriv -u labv3 -a SHA -A 'AuthPass123' \
    -x AES -X 'PrivPass123' ubuntu-lab SNMPv2-MIB::sysName.0
```

```
SNMPv2-MIB::sysName.0 = STRING: ubuntu-lab
```

What just happened: `-l authPriv` is the security level (authentication and privacy), `-u` the user, `-a` / `-A` the auth protocol and passphrase, `-x` / `-X` the privacy protocol and passphrase. With v2c, the community string and every value cross the network in clear text; with v3 authPriv, nobody can read or forge them. Use SNMPv3 in production.

Checkpoint: From rocky-lab, `snmpwalk -v2c -c labcomm ubuntu-lab system` shows `sysLocation.0 = STRING: Lab rack 1`, and the SNMPv3 `snmpget` returns `sysName.0`.

Task 6: Send and receive SNMP traps

Polling asks "how are you?" every few minutes. A trap lets the device shout "something happened!" immediately.

1. Install and configure the trap receiver on ubuntu-lab. The file is `/etc/snmp/snmptrapd.conf`.

```
sudo apt install -y snmptrapd
echo 'authCommunity log,execute,net labtrap' | sudo tee /etc/snmp/snmptrapd.conf
sudo systemctl enable snmptrapd
sudo systemctl restart snmptrapd
ss -ulnp | grep :162
```

```
UNCONN 0  0   0.0.0.0:162   0.0.0.0:*
```

What just happened: `snmptrapd` listens on UDP port 162. `authCommunity log,execute,net labtrap` says: accept traps that use community `labtrap`, and allow them to be logged, passed to scripts (`execute`), and forwarded (`net`). Traps with any other community are dropped.

2. Watch the receiver's log in one terminal on ubuntu-lab.

```
sudo journalctl -u snmptrapd -f
```

3. Send a trap from rocky-lab. This uses an example notification defined in `NET-SNMP-EXAMPLES-MIB` with one variable attached.

```
# on rocky-lab
snmptrap -v2c -c labtrap ubuntu-lab '' \
    NET-SNMP-EXAMPLES-MIB::netSnmpExampleHeartbeatNotification \
    netSnmpExampleHeartbeatRate i 123456
```

   On ubuntu-lab you see something like:

```
snmptrapd[3105]: 2026-10-10 10:48:12 rocky-lab [UDP: [192.168.100.20]:46125->
[192.168.100.10]:162]:
DISMAN-EVENT-MIB::sysUpTimeInstance = Timeticks: (1234567) 3:25:45.67
SNMPv2-MIB::snmpTrapOID.0 = OID: NET-SNMP-EXAMPLES-MIB::netSnmpExampleHeartbeatNotification
NET-SNMP-EXAMPLES-MIB::netSnmpExampleHeartbeatRate = INTEGER: 123456
```

What just happened: The arguments are: version, community, destination, then the uptime value (`''` means "use the current uptime"), the trap OID (what kind of event this is), and then variable bindings written as *OID type value* (`i` means integer, `s` would mean string). Real devices send traps like `IF-MIB::linkDown` or vendor traps for fan failures. Stop the journal view with `Ctrl+C`.

TIP

Traps use UDP and are fire-and-forget: if the packet is lost, the event is lost. SNMPv2c and v3 also support **informs**, which are traps the receiver must acknowledge (`snmpinform`). This is one reason monitoring systems combine traps with regular polling.

Checkpoint: `sudo journalctl -u snmptrapd --since "10 minutes ago"` on ubuntu-lab shows the trap from rocky-lab with `netSnmpExampleHeartbeatRate = INTEGER: 123456`.

Task 7: Aggregate logs from rocky-lab on ubuntu-lab

You will make ubuntu-lab a central log server and forward all rocky-lab logs to it over TCP port 514.

1. On ubuntu-lab, create a directory for remote logs owned by the `syslog` user (Ubuntu's rsyslog drops root privileges to that user), then add a receiver configuration in `/etc/rsyslog.d/10-remote-in.conf`.

```
sudo mkdir -p /var/log/remote
sudo chown syslog:adm /var/log/remote
cat <<'EOF' | sudo tee /etc/rsyslog.d/10-remote-in.conf
# Receive syslog over TCP 514 and write one file per sending host
template(name="PerHost" type="string" string="/var/log/remote/%HOSTNAME%.log")
ruleset(name="remote") {
    action(type="omfile" dynaFile="PerHost")
}
module(load="imtcp")
input(type="imtcp" port="514" ruleset="remote")
EOF
sudo rsyslogd -N1
sudo systemctl restart rsyslog
ss -tlnp | grep :514
```

```
rsyslogd: version 8.2312.0, config validation run (level 1), master config /etc/rsyslog.conf
rsyslogd: End of config validation run. Bye.
LISTEN 0  25   0.0.0.0:514   0.0.0.0:*
```

What just happened: `imtcp` is the TCP input module. The `input` binds it to port 514 and sends everything it receives into the ruleset `remote`, so remote messages are kept apart from local ones (they do not end up in `/var/log/syslog`). The `template` builds a file name from the sender's host name, and `dynaFile` uses that template, so each host gets its own file. `rsyslogd -N1` checks the syntax without starting anything; always run it before a restart. If `ufw` is active, run `sudo ufw allow from 192.168.100.0/24 to any port 514 proto tcp`.

2. On rocky-lab, forward everything to ubuntu-lab. The file is `/etc/rsyslog.d/90-forward.conf`.

```
# on rocky-lab
sudo dnf install -y rsyslog
cat <<'EOF' | sudo tee /etc/rsyslog.d/90-forward.conf
# Forward all messages to the central log server over TCP
*.* action(type="omfwd" target="ubuntu-lab" port="514" protocol="tcp"
           queue.type="linkedList" queue.size="10000"
           action.resumeRetryCount="-1")
EOF
sudo rsyslogd -N1
sudo systemctl restart rsyslog
logger -t labtest "hello from rocky-lab to the central log server"
```

What just happened: `*.*` selects every facility and every severity. `omfwd` is the forwarding module; `protocol="tcp"` is more reliable than UDP because lost packets are re-sent. The `queue` settings give the action its own memory queue, and `action.resumeRetryCount="-1"` means retry forever, so messages are held, not lost, while the server is unreachable. (In legacy syntax the same rule is `*.* @@ubuntu-lab:514`, where `@@` means TCP and a single `@` means UDP.)

3. On ubuntu-lab, check that the logs arrived.

```
ls -l /var/log/remote/
sudo tail -n 3 /var/log/remote/rocky-lab.log
```

```
-rw-r----- 1 syslog adm 1873 Oct 10 11:02 rocky-lab.log
2026-10-10T11:02:15.112345+00:00 rocky-lab systemd[1]: Started System Logging Service.
2026-10-10T11:02:16.004211+00:00 rocky-lab labtest[4012]: hello from rocky-lab to the
central log server
```

TIP

If nothing arrives: on rocky-lab run `journalctl -u rsyslog -n 20` (connection errors) and `sudo ausearch -m avc -ts recent` (SELinux denials). On ubuntu-lab check `journalctl -u rsyslog -n 20`, that port 514 is listening, and that `/var/log/remote` is owned by `syslog`. Test the network path with `nc -zv ubuntu-lab 514` from rocky-lab (`sudo dnf install -y nmap-ncat`).

4. Prove why central logs matter: search both hosts' events from one place.

```
sudo grep -h -i "fail\|error" /var/log/remote/*.log | tail -n 5
```

What just happened: With log aggregation, one search covers every host that forwards to the server. It also protects evidence: an attacker who gains root on rocky-lab can delete its local logs, but the copies already sent to ubuntu-lab remain. Larger shops use the same idea with tools such as the Elastic stack, Graylog, Loki, or Splunk, which add indexing, dashboards, and alerting on top of the collected logs.

Checkpoint: `/var/log/remote/rocky-lab.log` exists on ubuntu-lab and contains your `hello from rocky-lab` line.

Challenge

CHALLENGE

1. Create `threshold-check.service` and `threshold-check.timer` units that run the threshold script every 5 minutes, and make the webhook URL part of the service with `Environment=WEBHOOK=...`.
2. Change the health check so it also checks that the page body contains the word `nginx` (a content check), not just the status code.
3. Configure rocky-lab as a second SNMP agent (`sudo dnf install -y net-snmp`), open UDP 161 in firewalld, and poll it from ubuntu-lab with SNMPv3.

Clean up

1. On ubuntu-lab, stop and remove the health check timer, scripts, and logs.

```
sudo systemctl disable --now web-healthcheck.timer
sudo rm -f /etc/systemd/system/web-healthcheck.service \
    /etc/systemd/system/web-healthcheck.timer
sudo systemctl daemon-reload
sudo rm -f /usr/local/bin/web-healthcheck /usr/local/bin/threshold-check \
    /var/log/web-health.log
```

2. On rocky-lab, stop forwarding logs.

```
# on rocky-lab
sudo rm -f /etc/rsyslog.d/90-forward.conf
sudo systemctl restart rsyslog
```

3. On ubuntu-lab, remove the log receiver and the remote logs.

```
sudo rm -f /etc/rsyslog.d/10-remote-in.conf
sudo systemctl restart rsyslog
sudo rm -rf /var/log/remote
```

4. Remove the lab services and restore the SNMP client configuration on ubuntu-lab.

```
sudo apt purge -y snmpd snmptrapd prometheus-node-exporter nginx nginx-common
sudo apt autoremove -y
sudo sed -i 's/^#mibs :/mibs :/' /etc/snmp/snmp.conf
sudo rm -rf /var/lib/snmp/snmpd.conf /etc/snmp/snmpd.conf.orig
rm -rf ~/labs/5.1
```

5. If you opened any `ufw` ports, remove them with `sudo ufw status numbered` and `sudo ufw delete <number>`.

Exam corner

- **SLA** = contract with customers (with penalties); **SLO** = internal target (stricter than the SLA); **SLI** = the actual measurement (for example, % of successful requests).
- Error budget = 100% minus the SLO. 99.9% per 30 days allows about 43 minutes of downtime.
- SNMP agents listen on **UDP 161**; traps go to the manager on **UDP 162**. `snmpget` reads one OID, `snmpwalk` reads a subtree, `snmptrap` sends a trap, `snmptrapd` receives traps.
- A **MIB** maps numeric **OIDs** to names and descriptions. On Ubuntu, `snmp-mibs-downloader` and commenting out `mibs :` in `/etc/snmp/snmp.conf` enable names.
- SNMPv1/v2c use clear-text **community strings**; SNMPv3 adds users with authentication (`-a SHA`) and privacy/encryption (`-x AES`), security level `authPriv`.
- **Agent** monitoring installs software on each host (node_exporter on port 9100, Zabbix agent); **agentless** uses existing protocols (SSH, SNMP, WMI).
- A **webhook** is an HTTP POST (usually JSON) sent to a URL when an event occurs.
- A **health check** is a repeated probe (for example `curl` expecting HTTP 200) used by monitors and load balancers.
- **Log aggregation**: rsyslog `imtcp` / `imudp` on the server, `omfwd` (or `@@host:514` for TCP, `@host:514` for UDP) on clients.
- Threshold crossed leads to an alert, which is delivered as a notification. Too many non-actionable alerts cause **alert fatigue**.
- `logger -t tag -p facility.severity "msg"` writes events to the log; `journalctl -t tag -p warning` filters them.

Review questions

1. A company promises customers 99.5% uptime with service credits if it is missed, and internally aims for 99.9%. What is the 99.9% figure? A. SLA B. SLI C. SLO D. MIB
2. Which port and protocol does an SNMP manager use to receive traps?
3. You run `snmpget` on Ubuntu and see `iso.3.6.1.2.1.1.5.0` instead of `SNMPv2-MIB::sysName.0`. What is missing? A. The SNMP agent B. MIB files loaded by the client C. An SNMPv3 user D. A trap receiver

4. Which rsyslog client line forwards all messages to `logs.example.com` over TCP? A. `*.* @logs.example.com:514` B. `*.* @@logs.example.com:514` C. `*.* >logs.example.com` D. `*.* tcp://logs.example.com`
5. An administrator is paged 40 times a day for CPU spikes that go away on their own, and starts ignoring pages. What is this called, and what is one fix?
6. What is the main trade-off of agent-based monitoring compared with agentless monitoring?

REFERENCE

Appendices

Answer key · Exam corner review sheet · Coverage map · Lab VM troubleshooting · Acronyms

Appendix A: Answer Key

Answers to the review questions at the end of every lab.

Lab 1.1: Explore Basic Linux Concepts

1. **B.** Changes to `/etc/default/grub` only reach the bootloader after `update-grub` (or `grub-mkconfig`) regenerates `/boot/grub/grub.cfg`.
2. **C.** The initrd/initramfs is a temporary root filesystem in RAM that carries the drivers and tools (such as `lvm`) needed to find and mount `/`.
3. **PXE** (Preboot eXecution Environment); the bootloader is downloaded with **TFTP** after DHCP supplies the server address and file name.
4. **C.** `/proc` is a virtual filesystem whose contents are generated by the kernel on demand.
5. **B.** AArch64 is called `arm64` in Debian and Ubuntu package names; `amd64` is x86_64.
6. **B.** Copyleft requires anyone who distributes a modified version to provide the source under the same license.

Lab 1.5: Use Common Shell Operations

1. **B.** Redirections are processed left to right, so stdout must point to the file before `2>&1` copies stderr to the same place; option A sends errors to the screen.
2. Because `~/.bash_profile` now exists, a login Bash reads it instead of `~/.profile`, and on Ubuntu it is `~/.profile` that sources `~/.bashrc`; the fix is to add `[ -f ~/.profile ] && . ~/.profile` (or source `~/.bashrc`) to `~/.bash_profile`.
3. **B.** `uniq -c` only counts adjacent duplicates, so the values must be sorted first, then `sort -rn` ranks by count.
4. `sudo !!`, which re-runs the previous command with `sudo` in front.
5. **C.** `<<<` is a here string that feeds one string to standard input; `<` expects a file name and `<<` expects a here doc delimiter.
6. **C.** `:q!` quits and discards changes; `:wq`, `ZZ`, and `:x` all save first.

Lab 2.1: Manage Files and Directories

1. **C.** A hard link is another name for the same inode; deleting one name only lowers the link count, and the data remains while any name (or open descriptor) exists.
2. **B.** `find /var -size +500M` walks `/var` and matches files bigger than 500 MiB; `locate` only searches names.
3. **A.** `lsof +L1` lists deleted files that a process still holds open, which keeps their disk space allocated until the process closes them or restarts.
4. `b` means it is a block device; `8` is the major number (the `sd` disk driver) and `16` is the minor number (the second disk, `sdb`).

5. Run `sudo updatedb` to rebuild the locate database (or use `find`), because `locate` only knows files that existed when the database was last built.
6. **C.** `sdiff` prints the files in two columns and marks changed lines with `|`, left-only lines with `<`, and right-only lines with `>`.

Lab 5.1: Monitoring Concepts and Configurations

1. **C.** The SLO is the internal target; the SLA is the 99.5% contract and the SLI is the measured value.
2. **UDP 162.** Agents are polled on UDP 161 and send traps to the manager on UDP 162.
3. **B.** The data arrived; the client just has no MIB to translate the OID into a name (on Ubuntu, install `snmp-mibs-downloader` and comment out `mibs :`).
4. **B.** `@@` means TCP; a single `@` means UDP.
5. **Alert fatigue.** Fix it by raising the threshold or requiring the condition to last several minutes, and by alerting only on user-impacting, actionable conditions.
6. Agents give much richer data but must be installed, updated, and secured on every host; agentless needs nothing installed but is limited to what SSH or SNMP expose and needs credentials on the monitoring server.

Appendix B: Exam Corner Review Sheet

Every lab's key facts in one place. Read this section in the days before your exam.

Lab 1.1: Explore Basic Linux Concepts

- Boot order: firmware (BIOS/UEFI) then bootloader (GRUB 2) then kernel then initrd/initramfs then systemd (PID 1) then the default target.
- Edit `/etc/default/grub`, never `grub.cfg`. Rebuild with `update-grub` or `grub-mkconfig -o /boot/grub/grub.cfg` (Debian/Ubuntu) or `grub2-mkconfig -o /boot/grub2/grub.cfg` (RHEL/Rocky). `grubby` changes kernel parameters on RHEL-family systems.
- `GRUB_CMDLINE_LINUX` applies to all entries; `GRUB_CMDLINE_LINUX_DEFAULT` to normal boots only. `GRUB_TIMEOUT` and `GRUB_DEFAULT` control the menu.
- The running kernel's parameters are in `/proc/cmdline`. Press `e` at the GRUB menu for a one-time change.
- The initrd/initramfs holds drivers needed to mount `/`. Ubuntu: `update-initramfs`, `lsinitramfs`. Rocky: `dracut`, `lsinitrd`.
- PXE = network boot: DHCP gives an address and the boot file name, TFTP delivers the bootloader, then the kernel and initrd.
- FHS: `/etc` config, `/var` variable data and logs, `/proc` virtual kernel and process info, `/dev` device files, `/boot` kernel and initrd, `/usr` installed software, `/tmp` temporary (sticky bit), `/home` user files, `/bin` and `/sbin` and `/lib` essential commands and libraries.
- x86_64 = AMD64 = `amd64` (Debian). AArch64 = ARM64 = `arm64` (Debian). RISC-V is an open instruction set. `uname -m` shows the architecture.
- dpkg-based: `.deb`, `dpkg`, `apt` (Debian, Ubuntu). RPM-based: `.rpm`, `rpm`, `dnf` / `yum` / `zypper` (RHEL, Rocky, Fedora, SUSE).
- Display manager = graphical login (GDM, SDDM, LightDM). Window manager = places and decorates windows. X Server (Xorg) and Wayland are display servers; Wayland is the modern default.
- Copyleft (GPL) requires derivative works to keep the same license. Permissive (MIT, BSD, Apache) does not. Proprietary = closed source, restricted redistribution. Free software = the four freedoms.

Lab 1.5: Use Common Shell Operations

- Absolute paths start with `/` (or `~`, which expands to `$HOME`). Relative paths start from the current directory; `.` is the current directory, `..` the parent, and `cd -` returns to the previous directory (`$OLDPWD`).
- `HOME`, `USER`, `SHELL`, `PATH`, `PS1`, and `DISPLAY` are the key variables. `export` makes a variable visible to child processes; `env` / `printenv` list environment variables.
- `PATH` is searched left to right; the current directory is not in it, so run local scripts as `./script`.
- Login shells read `/etc/profile` then the first of `~/.bash_profile`, `~/.bash_login`, `~/.profile`. Non-login interactive shells read `~/.bashrc`. `source file` or `. file` applies changes to the current shell.
- Streams: stdin = 0, stdout = 1, stderr = 2. `>` overwrites, `>>` appends, `2>` redirects errors, `2>&1` joins stderr to stdout (order matters), `&>` sends both, `<` feeds a file to stdin, `<<` is a here doc, `<<<` is a here string, `/dev/null` discards.
- `|` passes only stdout. `tee` writes to a file and stdout at the same time; `| sudo tee file` writes a root-owned file.
- `uniq` only removes *adjacent* duplicates, so `sort | uniq -c | sort -rn` is the classic "count and rank" pipeline.
- `cut -d, -f2` for delimited fields; `awk '{print $2}'` for whitespace-separated columns; `sed 's/a/b/g'` to substitute; `sed -i` edits in place; `tr` changes or deletes single characters.

- `xargs` turns stdin lines into command arguments; `bc` does decimal math (`scale=`); `printf` formats output without an automatic newline.
- `!!` repeats the last command (`sudo !!`), `!n` runs history entry `n`, `!string` runs the latest command starting with `string`, `!$` is the last argument.
- `vim`: `i` insert, Esc normal mode, `:w` save, `:q!` quit without saving, `:wq` save and quit, `dd` delete line, `yy` / `p` copy/paste, `u` undo, `/text` search, `:%s/old/new/g` replace. `nano`: Ctrl+O save, Ctrl+X exit, Ctrl+W search.

Lab 2.1: Manage Files and Directories

- `ls -l` first character: `-` file, `d` directory, `l` symlink, `b` block device, `c` character device, `p` pipe, `s` socket. `ls -i` shows inode numbers.
- `cp -r` copies directories; `cp -a` copies recursively and preserves everything; `cp -p` preserves mode, ownership, and timestamps. `mv` renames or moves. `rmdir` only removes empty directories; `rm -r` removes non-empty ones.
- `mkdir -p` creates parent directories as needed. `pwd -P` resolves symlinks. `cd -` returns to the previous directory.
- `touch` creates empty files or updates timestamps; `touch -d "date"` or `-t YYYYMMDDhhmm` sets a specific time. ctime cannot be set manually.
- `stat` shows the inode: size, blocks, inode number, link count, permissions, UID/GID, and atime/mtime/ctime/birth.
- `file` identifies content by magic numbers, not by extension.
- `find PATH TESTS ACTION`: `-name`, `-iname`, `-type`, `-size +10M`, `-mtime +7`, `-mmin -30`, `-perm -4000`, `-user`, `-group`, `-exec cmd {} \;`, `-exec cmd {} +`, `-ok`, `-delete`.
- `locate` searches a database built by `updatedb` (package `plocate` on Ubuntu 24.04, `mlocate` on Rocky 9). It is fast but can be stale.
- `diff` shows changes (`<` first file, `>` second, `c` / `a` / `d`); `diff -u` is unified format; `sdiff` shows side by side with `|`, `<`, `>`.
- Hard link: same inode, link count increases, survives deletion of the original, no directories, no cross-filesystem. Symbolic link (`ln -s`): own inode, stores a path, can cross filesystems and point to directories, breaks if the target is removed.
- `lsof +D dir` finds what is using a directory; `lsof +L1` finds deleted-but-open files; `lsof -i :port`; `lsof -u user`; `lsof -p PID`.
- Block devices (disks, partitions) use random block access; character devices (terminals, serial ports) are byte streams. Special character devices: `/dev/null`, `/dev/zero`, `/dev/random`, `/dev/urandom`, `/dev/full`, `/dev/tty`. Device files show major (driver) and minor (instance) numbers.

Lab 5.1: Monitoring Concepts and Configurations

- **SLA** = contract with customers (with penalties); **SLO** = internal target (stricter than the SLA); **SLI** = the actual measurement (for example, % of successful requests).
- Error budget = 100% minus the SLO. 99.9% per 30 days allows about 43 minutes of downtime.
- SNMP agents listen on **UDP 161**; traps go to the manager on **UDP 162**. `snmpget` reads one OID, `snmpwalk` reads a subtree, `snmptrap` sends a trap, `snmptrapd` receives traps.
- A **MIB** maps numeric **OIDs** to names and descriptions. On Ubuntu, `snmp-mibs-downloader` and commenting out `mibs :` in `/etc/snmp/snmp.conf` enable names.
- SNMPv1/v2c use clear-text **community strings**; SNMPv3 adds users with authentication (`-a SHA`) and privacy/encryption (`-x AES`), security level `authPriv`.
- **Agent** monitoring installs software on each host (node_exporter on port 9100, Zabbix agent); **agentless** uses existing protocols (SSH, SNMP, WMI).

- A **webhook** is an HTTP POST (usually JSON) sent to a URL when an event occurs.
- A **health check** is a repeated probe (for example `curl` expecting HTTP 200) used by monitors and load balancers.
- **Log aggregation**: rsyslog `imtcp` / `imudp` on the server, `omfwd` (or `@@host:514` for TCP, `@host:514` for UDP) on clients.
- Threshold crossed leads to an alert, which is delivered as a notification. Too many non-actionable alerts cause **alert fatigue**.
- `logger -t tag -p facility.severity "msg"` writes events to the log; `journalctl -t tag -p warning` filters them.

Appendix C: Exam Objective Coverage Map

This map shows where each topic from the official XK0-006 objectives is practiced. Topics are listed under the part of the lab where they first appear.

1.1 Explain basic Linux concepts.

Practiced in Lab 1.1.

Where in the lab	Exam topics practiced
Lab introduction	`bootloader`, `kernel`, `parameter`
What you'll practice	`boot process`, `configuration file`, `initrd`, `PXE`, `Filesystem Hierarchy Standard`, `/bin`, `/boot`, `/dev`, `/etc`, `/home`, `/lib`, `/proc`, `/sbin`, `/tmp`, `/usr`, `/var`, `AArch64`, `RISC-V`, `x86`, `x86_64`, `AMD64`, `RPM`, `dpkg`, `display manager`, `window manager`, `X Server`, `Wayland`, `open source`, `free software`, `proprietary`, `copyleft`

1.5 Given a scenario, manage a Linux system using common shell operations.

Practiced in Lab 1.5.

Where in the lab	Exam topics practiced
Lab introduction	`HOME`, `PATH`, `SHELL`, `awk`, `grep`, `history`, `sed`, `tr`, `vim`, `nano`
What you'll practice	`DISPLAY`, `PS1`, `USER`, `absolute`, `relative`, `.bashrc`, `.bash_profile`, `.profile`, `standard output`, `standard error`, `standard input`, `here doc`, `alias`, `bc`, `cat`, `cut`, `echo`, `head`, `less`, `more`, `printf`, `sort`, `source`, `tail`, `tee`, `uname`, `uniq`, `wc`, `xargs`

2.1 Given a scenario, manage files and directories on a Linux system.

Practiced in Lab 2.1.

Where in the lab	Exam topics practiced
Lab introduction	`diff`, `file`, `find`, `ls`, `symbolic link`, `hard link`
What you'll practice	`cd`, `cp`, `ln`, `locate`, `lsof`, `mkdir`, `mv`, `pwd`, `rm`, `rmdir`, `sdiff`, `stat`, `touch`, `block device`, `character device`, `special character device`

5.1 Summarize monitoring concepts and configurations in a Linux system.

Practiced in Lab 5.1.

Where in the lab	Exam topics practiced
Lab introduction	`SLO`, `SNMP`, `trap`, `agent`, `agentless`, `webhook`, `health check`, `log aggregation`, `threshold`, `alert`, `logging`
What you'll practice	`SLA`, `SLI`, `MIB`, `event`, `notification`

Appendix E: Acronym List

The acronyms from the official exam objectives. Know what each one stands for.

ACL	Access Control List
ACME	Automated Certificate Management Environment
AI	Artificial Intelligence
AIDE	Advanced Intrusion Detection Environment
API	Application Programming Interface
ARM	Advanced RISC (Reduced Instruction Set Computer) Machine
BIOS	Basic Input/Output System
CA	Certificate Authority
CI/CD	Continuous Integration/Continuous Deployment
CIFS	Common Internet File System
CIS	Center for Internet Security
CMS	Content Management System
CPU	Central Processing Unit
CSV	Comma-Separated Values
CUPS	Common UNIX Printing System
CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
DHCP	Dynamic Host Configuration Protocol
DNAT	Destination Network Address Translation
DNS	Domain Name System
EGID	Effective Group Identifier
EPEL	Extra Packages for Enterprise Linux
EUID	Effective User Identifier
FEC	Forward Error Correction
FHS	Filesystem Hierarchy Standard
FQDN	Fully Qualified Domain Name
FTP	File Transfer Protocol
FUSE	Filesystem in Userspace
GDPR	General Data Protection Regulation
GID	Group Identifier
GNU	GNU's Not Unix
GPG2	GNU Privacy Guard 2
GPG	GNU Privacy Guard
GPT	GUID (Globally Unique Identifier) Partition Table
GPU	Graphics Processing Unit
GRUB	Grand Unified Bootloader
GUI	Graphical User Interface
GUID	Globally Unique Identifier
HMAC	Hashed Message Authentication Code
HTTP	HyperText Transfer Protocol
HTTPD	HyperText Transfer Protocol Daemon
IaC	Infrastructure as Code
ICMP	Internet Control Message Protocol
IFS/OFS	Internal Field Separator/Output Field Separator
IMAP4	Internet Message Access Protocol 4
initrd	Initial RAM Disk
I/O	Input/Output
IoC	Indicators of Compromise
IOPS	Input/Output Operations Per Second
IP	Internet Protocol
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
KRB5	Kerberos 5
KVM	Kernel-based Virtual Machine
LAN	Local Area Network
LDAP	Lightweight Directory Access Protocol
LLM	Large Language Model
LUKS	Linux Unified Key Setup
LUKS2	Linux Unified Key Setup 2
LVM	Logical Volume Manager
MAC	Media Access Control
MBR	Master Boot Record
MFA	Multifactor Authentication
MIB	Management Information Base
MTU	Maximum Transmission Unit
NAS	Network-Attached Storage
NAT	Network Address Translation

NFS	Network File System
NIC	Network Interface Card
NTP	Network Time Protocol
NVMe	Non-Volatile Memory Express
OOM	Out of Memory
OpenSCAP	Open Security Content Automation Protocol
OpenSSL	Open Secure Sockets Layer
OS	Operating System
PAM	Pluggable Authentication Modules
PAT	Port Address Translation
PEP	Python Enhancement Proposal
PHP	PHP: Hypertext Preprocessor
PID	Process Identification Number
PKI	Public Key Infrastructure
PPID	Parent Process Identification Number
PTP	Precision Time Protocol
PV	Physical Volume
PXE	Preboot Execution Environment
qcow	QEMU Copy On Write
QEMU	Quick Emulator
RAID	Redundant Array of Independent Disks
RAM	Random Access Memory
RISC	Reduced Instruction Set Computer
RPM	Red Hat Package Manager (RPM Package Manager)
SAN	Storage Area Network
SELinux	Security-Enhanced Linux
setGID	Set Group Identity
setUID	Set User Identity
SFTP	Secure File Transfer Protocol
SLA	Service-Level Agreement
SLES	SUSE Linux Enterprise Server
SLI	Service-Level Indicator
SLO	Service-Level Objective
SMB	Server Message Block
SMTP	Simple Mail Transfer Protocol
SNAT	Source Network Address Translation
SNMP	Simple Network Management Protocol
SR-IOV	Single Root Input/Output Virtualization
SSD	Solid-State Drive
SSH	Secure Shell
SSHD	Secure Shell Daemon
SSL	Secure Sockets Layer
SSO	Single Sign-On
SSSD	System Security Services Daemon
TCP	Transmission Control Protocol
TFTP	Trivial File Transfer Protocol
TLS	Transport Layer Security
TOTP	Time-based One-Time Password
TTL	Time to Live
UDP	User Datagram Protocol
UEFI	Unified Extensible Firmware Interface
UFW	Uncomplicated Firewall
UID	User Identifier
USB	Universal Serial Bus
UUID	Universally Unique Identifier
Vim	Vi Improved
VM	Virtual Machine
WAN	Wide Area Network
XML	Extensible Markup Language
YAML	YAML Ain't Markup Language